软件工程与方法

第4讲 软件开发

本章内容

- 系统工程
- 需求工程
- 软件设计
- 软件实现

软件开发面临的严重问题

- 认知问题——理解要做的软件
  - 需求模糊、知识孤岛、系统复杂、认知负荷
- 过程问题——有序地开发软件
  - 计划与现实的鸿沟、质量与速度的冲突、沟通不畅、风险失控
- 演化问题——保持系统持续生
  - 技术债累积、系统腐化、适应变化、知识流失

软件开发的关键活动（essence）

规划解决方案
（建模和软件设计）
Plan a solution
(modeling and software design)
系统设计：“蓝图”

执行规划
（代码生成）
Carry out the plan
(code generation)
编码实现：“执行层”

理解问题
（通信和分析）
Understand the problem
(communication and analysis)
需求管理：“方向标”

为了准确性检查结果
（测试和质量保证）
Examine the result for accuracy
(testing and quality assurance)
测试验证：“质量关”

软件开发

1 系统工程

主要内容

① 系统分析方法

② 可行性研究

③ 成本-效益分析

④ 技术分析

⑤ 系统结构的表示

1.1 系统分析方法

系统分析的目标：

- 识别用户要求
- 评价系统的可行性
- 进行经济分析和技术分析
- 把功能分配给硬件、软件、人、数据库和其它系统元素
- 建立成本和进度限制
- 生成系统规格说明，形成所有后续工程的基础

系统分析所要回答的问题：

所期望的功能和性能是什么？

可靠性和质量问题是哪些？

总的系统目标是什么？

成本与进度限制如何？

制造的需求是什么？

市场与竞争情况怎样？

有效的技术有哪些？

将来可能有哪些扩充？

基于计算机的系统

- 系统工程（system engineering）关注于将客户的需求、期望以及提出的限制条件转化到产品的解决方案中，并且在产品的整个生命周期中对解决方案给予始终的支持。
- 系统工程涵盖了一个系统所有的开发工作，其中不一定包括软件。
- 计算机系统工程
  - 通常把基于计算机的系统定义为某些元素的一个集合，并把这些元素组织起来以实现某种方法、过程或借助处理信息进行控制。
  - 一个问题求解活动，目的是揭示、分析所期望的功能，并把它们分配到系统元素中去。
  - 它**一定包括计算机，并且包括软件**。

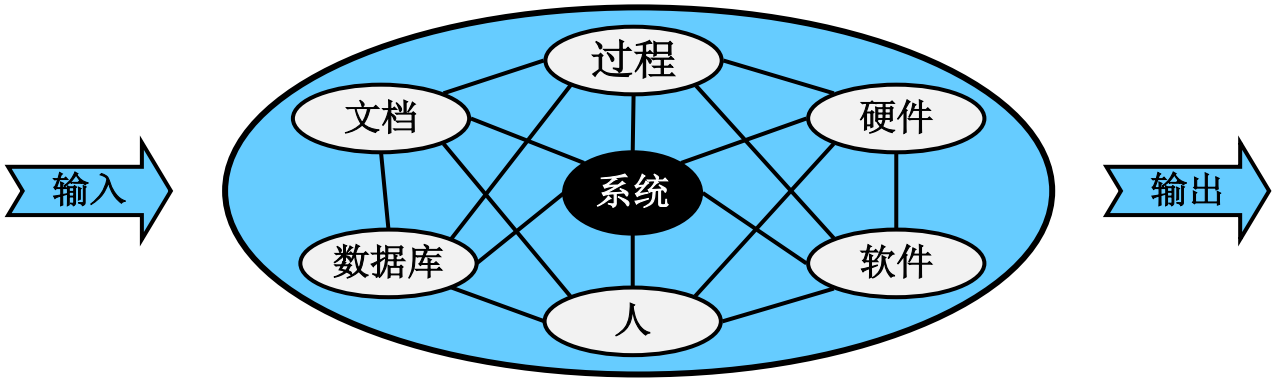
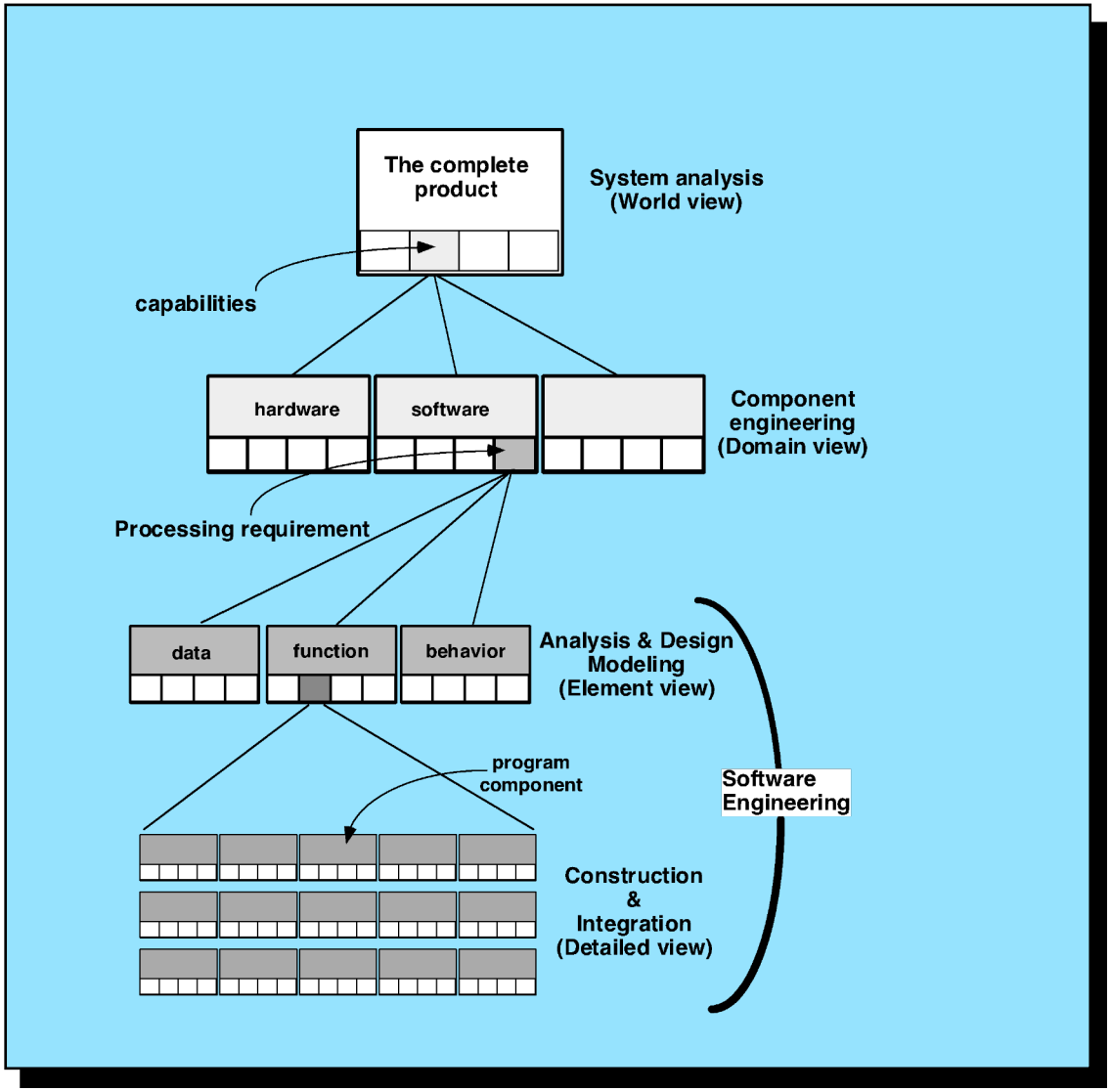
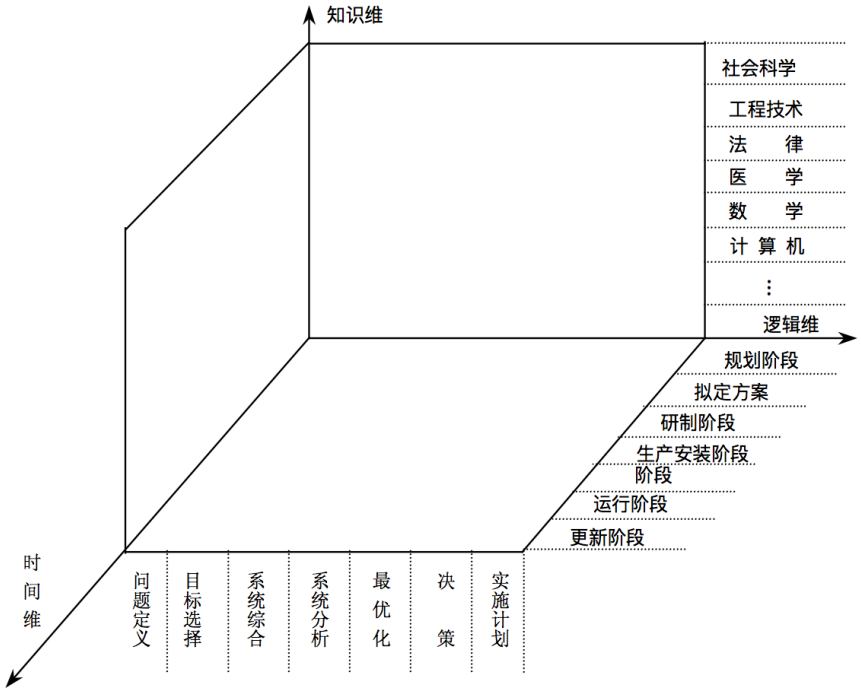


图 基于计算机系统的系统元素

The system engineering of hierarchy



A. D. Hall提出的系统工程方法的三维模型



系统工程的三维模型

描述了复杂系统建设方法的一个总体结构框架，其中：

- 时间维：表示系统建设在时间上的阶段划分
- 逻辑维：表示处理问题的逻辑步骤
- 知识维：表示系统建设所涉及的知识领域

1.2 可行性研究

- 在错误的道路上奔跑，速度越快（AI加持下），死得越惨。可行性研究就是在那张地图画出来之前，先确认这条路通不通。
- 可行性研究是用最小的成本，为项目的未来购买一份"确定性"。 它让我们在充满不确定性的软件世界中，做出更理性、更可靠的决策。

可行性研究焦点问题：

- 系统是否符合机构的总体目标？
- 系统是否可能在现有技术条件、预算和时间限制内完成？
- 系统能否与已经存在的其它系统集成？
- 组织是否能够承受开发所面临的风险？

五大核心评估维度：

1. 技术可行性 (Technical Feasibility)
2. 经济可行性 (Economic Feasibility)
3. 法律与合规可行性 (Legal & Compliance Feasibility)
4. 操作可行性 (Operational Feasibility)
5. 时间可行性 (Schedule Feasibility)

可行性研究报告的主要内容

1. 执行摘要：一页纸讲清楚结论和建议。
2. 项目背景与目标：为什么要做？解决什么痛点？
3. 备选方案对比：各方案的优缺点矩阵。
4. 详细可行性分析：
   - 技术架构图与风险评估。
   - 财务模型（NPV净现值, IRR内部收益率, ROI）。
   - 法律合规清单。
   - 组织变革影响分析。
5. 实施路线图：初步的时间表和里程碑。
6. 结论与建议：明确的“行”或“不行”。

1.3 软件工程经济学

- 经济学是关于选择的科学，而不是关于金钱的科学。
- 软件工程必须以价值为基础。
- 软件工程经济学使软件技术决策与组织的业务目标保持一致。

- 成本是管理者最关心的问题，它是项目成败的关键。
- 追求效益是项目的目标。
- 效益分类：

经济效益 vs 社会效益

有形的 vs 无行的

短期的 vs 长期的

1.4 技术分析

- 技术分析的核心如下：
  1. 架构模式选择 (Architecture Pattern)
  2. 技术栈选型 (Tech Stack Selection)
  3. 数据策略 (Data Strategy)
  4. 非功能性需求 (NFRs) 保障
  5. 成本分析 (Cost Analysis / FinOps)

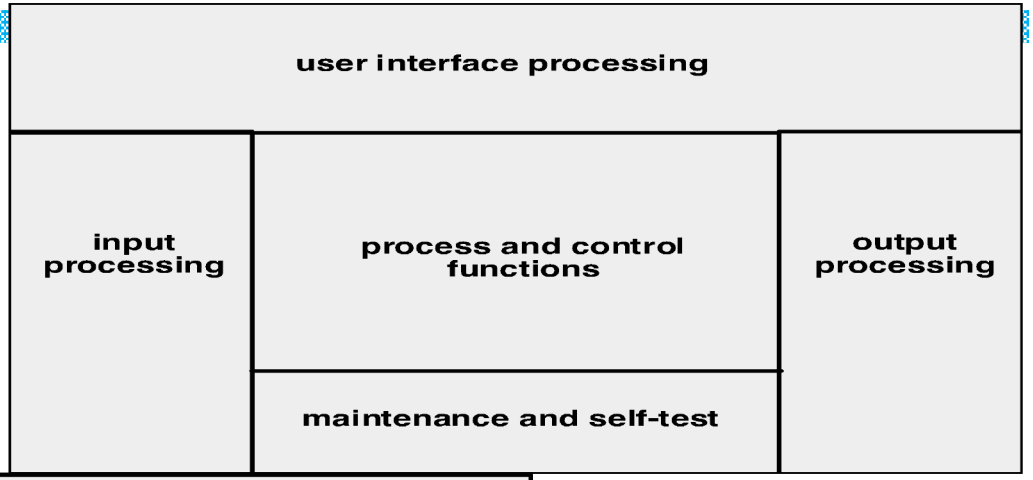
- 技术分析是在**业务需求、技术能力、成本预算和未来风险**之间找到那个**最优平衡点**。它不是一次性的任务，而是一个随着项目演进不断迭代的动态过程。

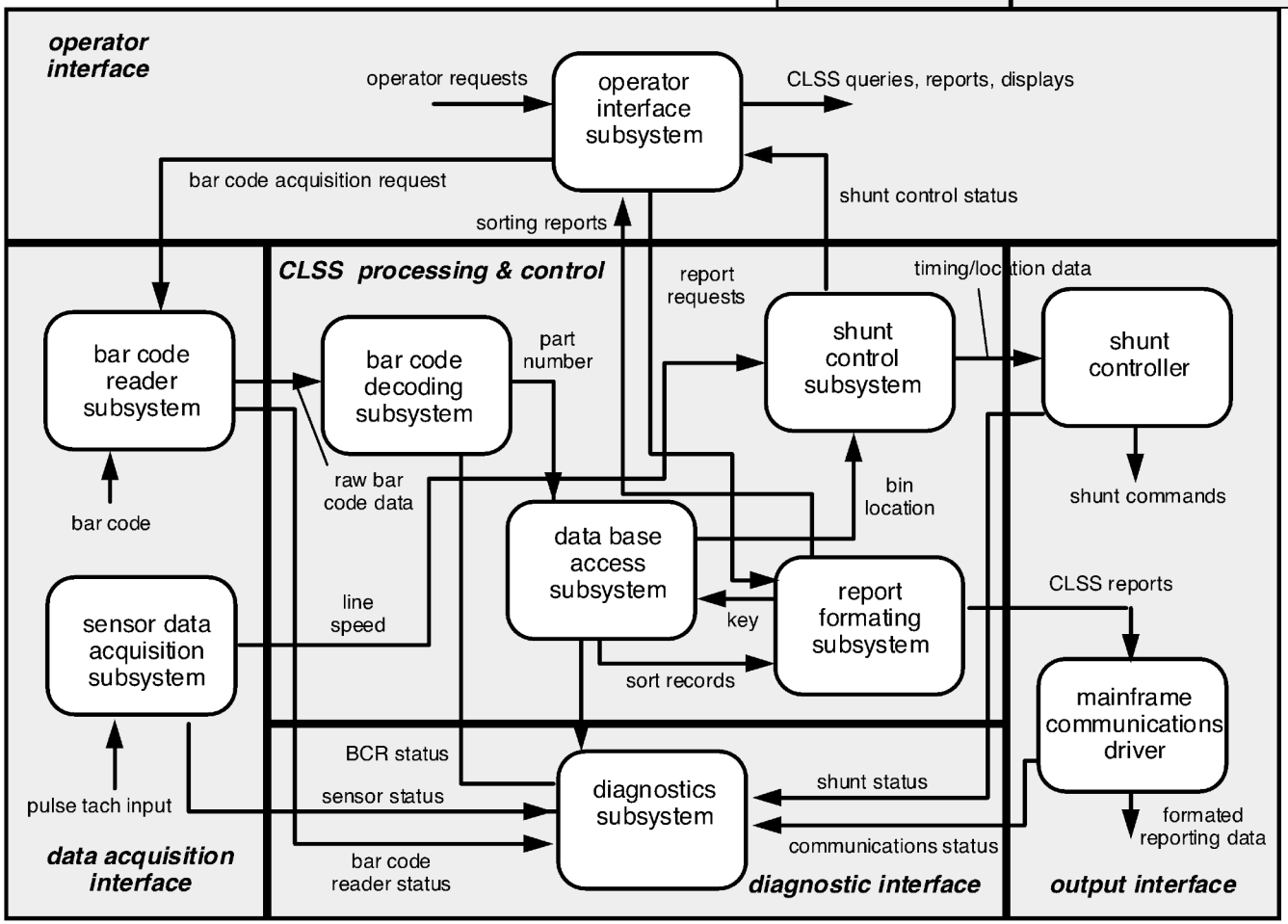
1.5 系统结构的表示

- 一旦把基于计算机系统的功能分配到各个元素，就能够据此建立一个模型，用以表达系统元素之间的相互关系，并为需求分析和设计奠定基础。
- 软件结构图（Software Structure Diagram）是描述系统组件及层次关系的静态可视化工具，其核心是展示系统的“物理”或“逻辑”结构。
- 强调层次结构，如：表现层→业务层→数据层
- 常见类型：
  - 模块结构图：用于描述软件的模块化设计，展示各模块的功能及依赖
  - 组件结构图：用于描述软件的组件化设计，展示各组件的交互
  - 类图：属于UML结构图的子类，用于描述对象-oriented系统的静态结构

产品体系结构模板

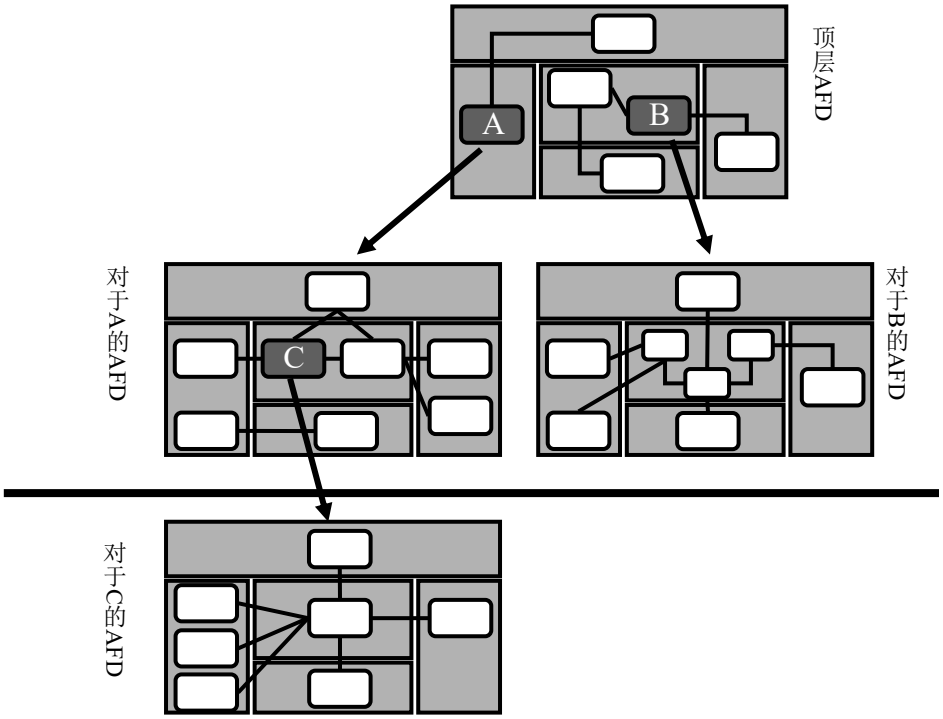
ACD(Architecture Context Diagram)



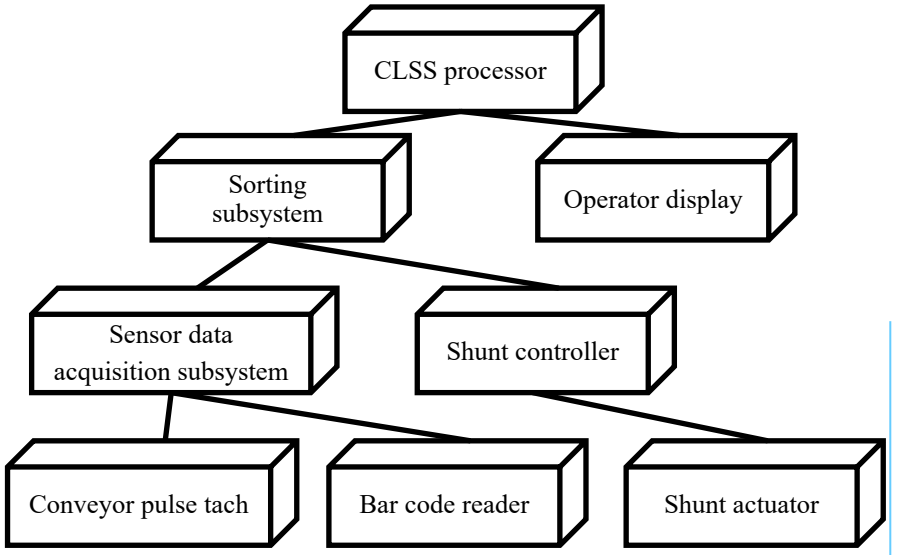


模型图

- ACD(Architecture Context Diagram)
- AFD (Architecture Flow Diagram)
- AID(Architecture Interconnect Diagram)
- ADS(Architecture Diagram Specification)



AFD的层次结构图



CLSS的UML布置图

系统规格说明书的主要内容：

1. 引言
   - 文档的范围和目的
   - 概述：目标；限制条件
2. 功能和数据描述
   - 系统结构：ACD图；ACD描述
3. 子系统描述
   - 对于子系统n的ADS；AFD；系统模块描述；性能问题；设计限制条件；系统部件的分配
   - 结构词典
   - AID及其描述
4. 系统模型化和模拟结果
   - 用于模拟的系统模型
   - 模拟结果
   - 特殊的性能问题
5. 项目问题
   - 项目开发成本
   - 项目进度安排
6. 附录

软件开发

2 需求工程

主要内容

① 需求工程过程

② 需求采集

③ 需求分析方法

④ 需求评审

⑤ 需求管理

一个天文学家、一个物理学家和一个数学家(据说)正在苏格兰度假。当他们从火车车厢的窗口向外瞭望时，观察到田地中央有一只黑色的羊。“多么有趣，”天文学家评论道，“所有的苏格兰羊都是黑色的！”物理学家对此反驳说，“不，不！某些苏格兰羊是黑色的！”数学家祈求地凝视着天空，然后吟诵起来，“在苏格兰至少存在着一块田地，至少有一只羊，这只羊至少有一侧是黑色的。”

需求案例：F-22飞机软件故障

- 现象：
  - 2004年12月21日，一架F-22战斗机在内利斯空军基地进行战术训练的过程中，飞机坠毁。
  - 2007年2月11日，12架F-22战斗机从夏威夷飞往日本，但在穿越国际日期变更线的时候，飞机失去了所有的导航和通信资料，被迫目视返航。
- 原因：分别是飞行控制软件故障、通信导航软件没有考虑日期变更线，导致软件瘫痪。

需求案例：血淋淋的 BUG

两起空难

- 2018年10月29日，一架波音737 Max 8客机撞向印度尼西亚的爪哇海，造成189名乘客和机组人员死亡。调查人员称该飞机的飞行控制软件出现“故障”。
- 2019年3月10日，埃塞俄比亚航空公司一架波音737 MAX 8客机在飞往肯尼亚首都内罗毕途中坠毁。飞机上载有157乘客和机组人员。怀疑埃航这架飞机发生了同样的“软件故障”。

迟迟修正不了的 BUG

MCAS，自动纠正失速系统，是波音737 MAX的一种操纵辅助系统。它有几个特点：

① 发现失速后，程序只相信主传感器，不与备份传感器核实。（同样的情况空客的飞机则会交给飞行员处理。）

② 一旦相信，不通知飞行员，直接操纵机翼。

③ 飞行员手动操作后，仍旧会每五秒自动执行，让飞行员不得不与飞机较劲。

④ 程序开关非常隐蔽。

THE WALL STREET JOURNAL.

Home World U.S. Politics Economy Business Tech Markets Opinion Life & Arts Real Estate WSJ. Magazine

BUSINESS

Boeing to Make Key Change in 737 MAX Cockpit Software

MAX software change marks shift for Boeing design of anti-stall system

从技术角度看，这是个很容易修正的错误。延误的原因，其中之一是联邦官员和波音公司安全专家对于需要进行多大改动，有意见分歧：

- 波音公司认为，更新这个BUG很简单，减少下压幅度就行。
- 联邦官员认为，189条人命是大事，必须大改，确保单个传感器错误信号不会触发MCAS系统。
- 加上波音公司和美国政府的低效，这个性命攸关的 BUG，硬生生被拖到四个月后。

- 波音 737 MAX 的交付
  - 波音要求每个需求都能追溯到相应的代码行。反之亦然。也就是说每行代码发生变化，必须能追溯到软件的设计文档。
- 软件外包和成本压缩
  - 裁掉了90%熟练员工，用“外包”来代替他们。
  - 2015年，波音公司给印度的外包业务翻了一番，达到了5亿美元。
- 使用Ada语言达到99%
  - 属于80年代的编程语言。
  - 缺少合适数量精通Ada的程序员。

软件需求的重要性

1. 决定项目成败的“第一性原理”
2. 多方沟通的“唯一权威基准”
3. 控制成本与进度的“核心杠杆”
4. 确保软件质量的“底层逻辑”
5. 知识传递与后期维护的“救命稻草”

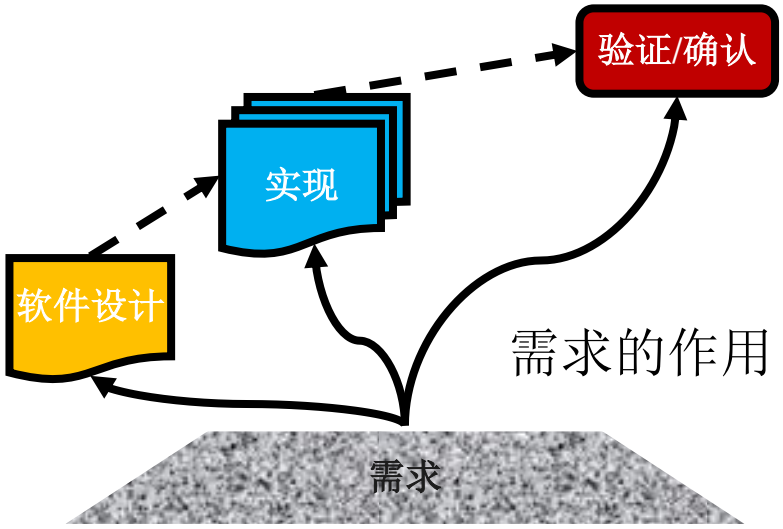
软件所有开发、测试、运维工作全部建立在需求之上；需求错一步，后期成本放大数十至百倍，轻则项目延期超支，重则资金报废、安全事故、经济赔偿。

2.1 什么是需求工程

- 需求工程是一个涵盖创建和维护系统需求文档所必需的一切活动的过程。
- 需求工程是指应用已证实有效的技术、方法进行需求分析，确定客户需求，帮助分析人员**理解问题**并**定义目标系统**的所有外部特征的一门学科。它通过合适的工具和记号系统地描述待开发系统及其行为特征和相关约束，形成**需求文档**，并对用户不断变化的**需求演进**给予支持。
- 需求工程包括：
  - **需求开发：**

    指需求的获取、分析、规格说明和验证。典型的需求开发结果是需求规格说明书及相关的分析模型。
  - **需求管理：**

    在软件开发过程中对需求开发结果的控制、跟踪和管理。



需求工程的任务

需求模型定义系统的行为空间，每个行是为了实现系统目标。

- **引导：**决定客户需要什么
- **分析和协商：**理解各种客户需求之间的关系，整理这些关系成为一个高效的解决方案。
- **需求规格书：**建立一个明确的需求模型。
- **系统模型：**建立需求表示，这个表示能用于评估需求的正确性、完整性和一致性。
- **确认：**检查模型。
- **管理：**识别、控制和追踪需求和需求变更。

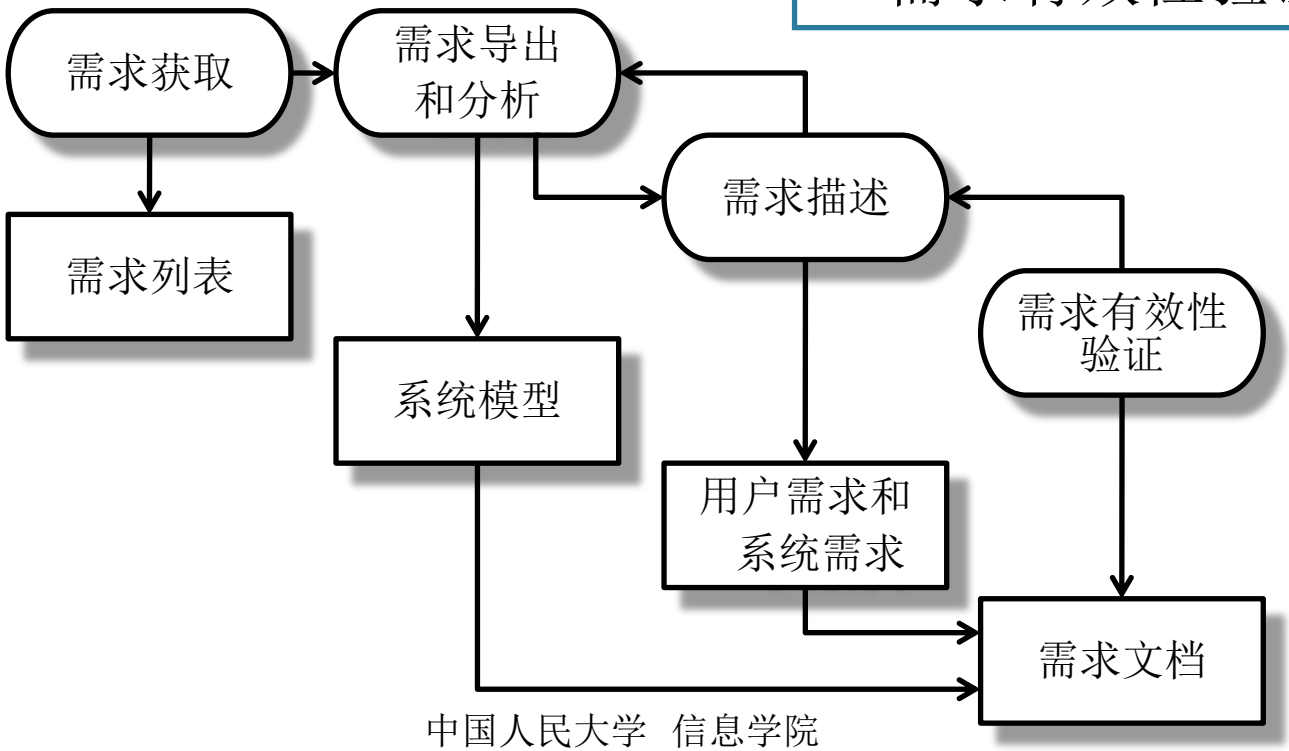
需求工程的主要活动

需求管理的主要活动包括：

- 需求基线的建立
- 需求变更控制
- 需求跟踪

需求开发的主要活动：

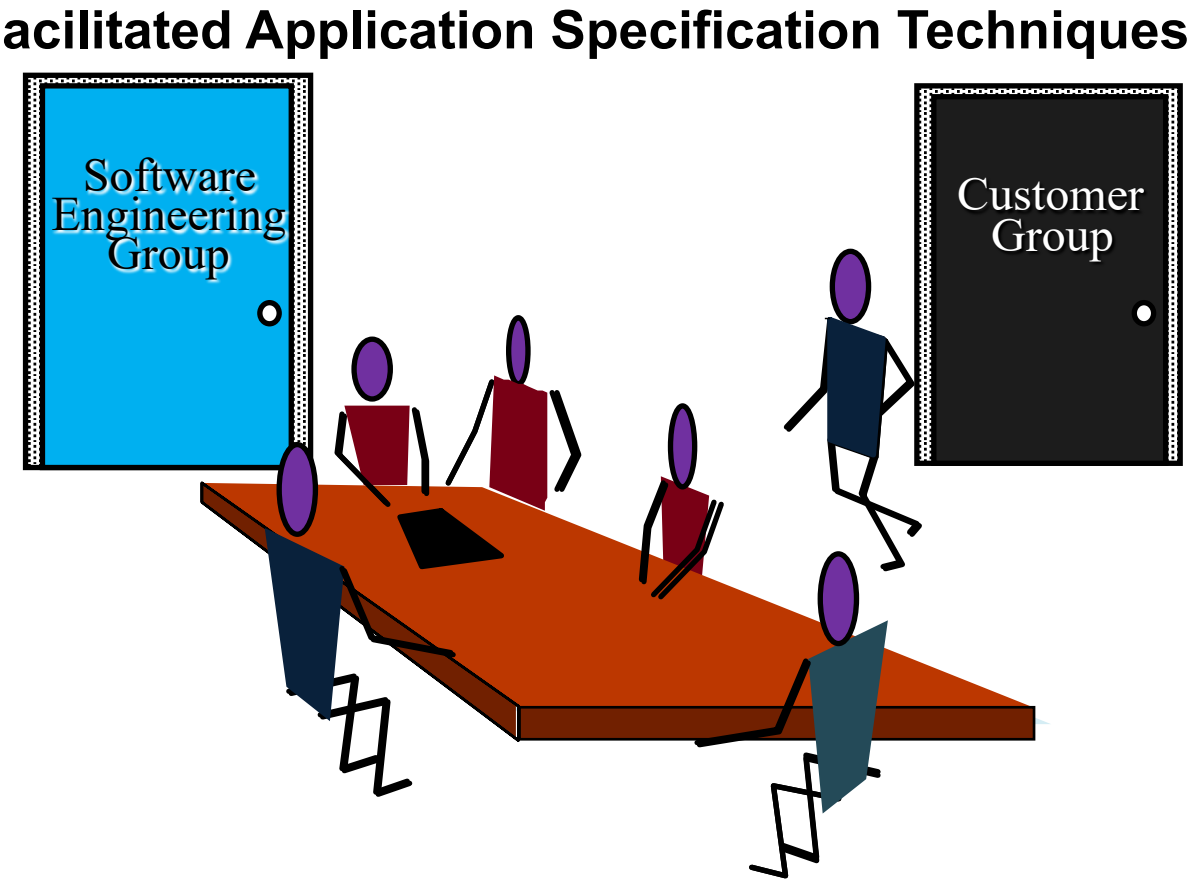
- 需求获取
- 需求导出和分析
- 需求描述和文档编写
- 需求有效性验证



2.2 需求采集

系统分析员、又称**系统工程师**，他的主要责任是：

- 引导用户，启发用户，挖掘需求。
- 将需求以用户和开发人员都易于理解的方式表达出来。



FAST: basic guidelines

- **A meeting** is conducted at a neutral site and attended by both software engineering analysts and customers.
- **Rules** for preparation and participation are established.
- **An agenda** is suggested that is formal enough to cover all important point but informal enough to encourage the free flow of ideas.
- **A facilitator** (can be a customer, a developer or an outsider) is assigned to control the meeting.
- **A definition** mechanism (can be work sheets flip charts or wall stickers or chat room) is used where how meeting will be conducted.
- **The goal** of this meeting is to identify the problem, propose elements of the solution, negotiate different approaches and specify a preliminary set of solution requirement in an atmosphere for the achievement of the goal.

FAST: the steps activities

- In the beginning a formal meeting is conducted between developer and customer, Q&A session is started to have the general idea about the problem, desired solution, customer and benefits.
- Then, customer and developer both write a separate one or two page. Product request document. This is just like a small report with brief description.
- Next, step is to set a meeting place, time and date for FAST. Parallel a facilitator and attendees of the FAST (both the development and customer user organization sides) are selected.
- After that the product request reports are distributed to all the attendees before the meeting starts.
- Additionally before the meeting each attendee is asked to make a list of three objects or instances:
  - Objects that will be used by the system to perform its functions (Objects of input).
  - Objects that will be produced by the system (objects of output) and,
  - Objects that will be part of the environment that surrounds the system.

Guidelines & Principles

FAST Guidelines

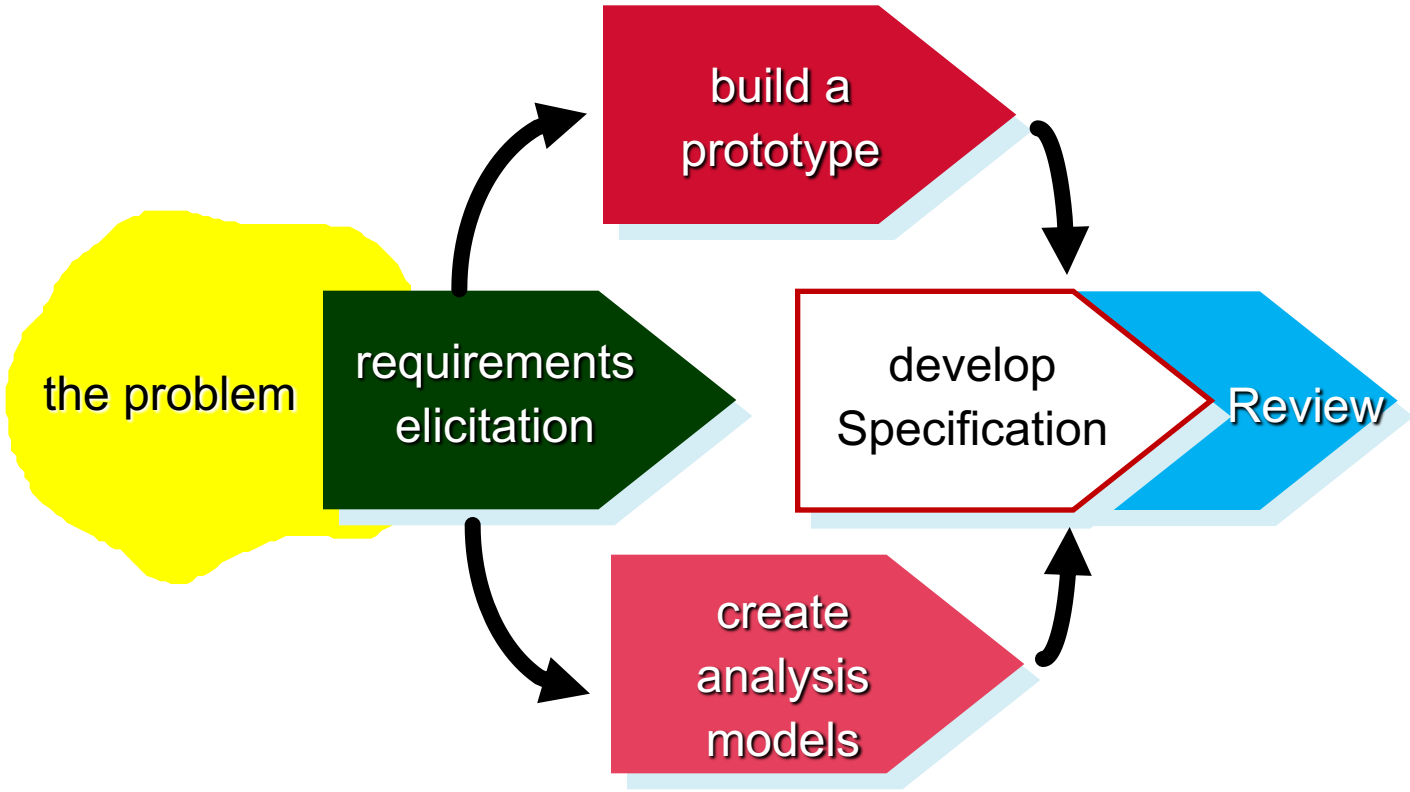
- participants must attend entire meeting
- all participants are equal
- preparation is as important as meeting
- all pre-meeting documents are to be viewed as “proposed”
- off-site meeting location is preferred
- set an agenda and maintain it
- don’t get mired in technical detail

J. Wood & D. Silver

Davis’ Principles

- Understand the problem before you begin to create the analysis model.
- Develop prototypes that enable a user to understand how human-machine interaction will occur.
- Record the origin of and the reason for every requirement.
- Use multiple views of requirements.
- Prioritize requirements.
- Work to eliminate ambiguity.

需求分析过程


build a prototype
the problem
requirements elicitation
develop Specification
Review
create analysis models

开发原型（Prototype）

- 在开发的早期，用户和开发者往往对系统的认识模糊，很难完全准确地表达系统的全面要求。
- 原型是一个可实际运行、反复修改、可以不断完善的系统。

- 原型主要有3种不同的作用类型：

① **探索型**
  - 目的是要弄清对目标系统的要求，确定所希望的特性，并探讨多种方案的可行性。
  - 它主要针对开发目标模糊，用户和开发者对项目都缺乏经验的情况。

② **实验型**
  - 用于大规模开发和实现之前，考核方案是否合适，规格说明是否可靠。

③ **进化型**
  - 目的不在于改进规格说明，而是将系统建造得易于变化，在改进原型的过程中，逐步将原型进化成最终系统。

- 运用原型的2种不同策略：

① **废弃策略**
  - 先构造一个功能简单而且质量要求不高的模型系统。
  - 系统构造完成后，原来的模型系统被废弃。
  - 探索型和实验原型属于这种策略。

② **追加策略**
  - 先构造一个功能简单而且质量要求不高的模型系统，作为最终系统的核心。
  - 它对应于进化原型。

原型开发法的特点分析

- 原型法从基本需求出发，快速构建系统的原型，通过原型确认需求，改进原型，最终达到建立系统的目的。
- 软件原型化开发方法通常有3个基本步骤：
  - ① 获得基本需求
  - ② 开发工作模型
  - ③ 模型验证
- 原型开发技术的特点：
  - ① 加快需求的确定
  - ② 简化项目管理
  - ③ 加强用户参与与决策

原型法优点：

1）增进用户与开发人员之间的沟通

2）用户在系统开发过程中起主导作用

3）辨认动态的用户需求

4）启迪衍生式的用户需求

5）缩短开发周期，降低开发风险

原型法缺点：

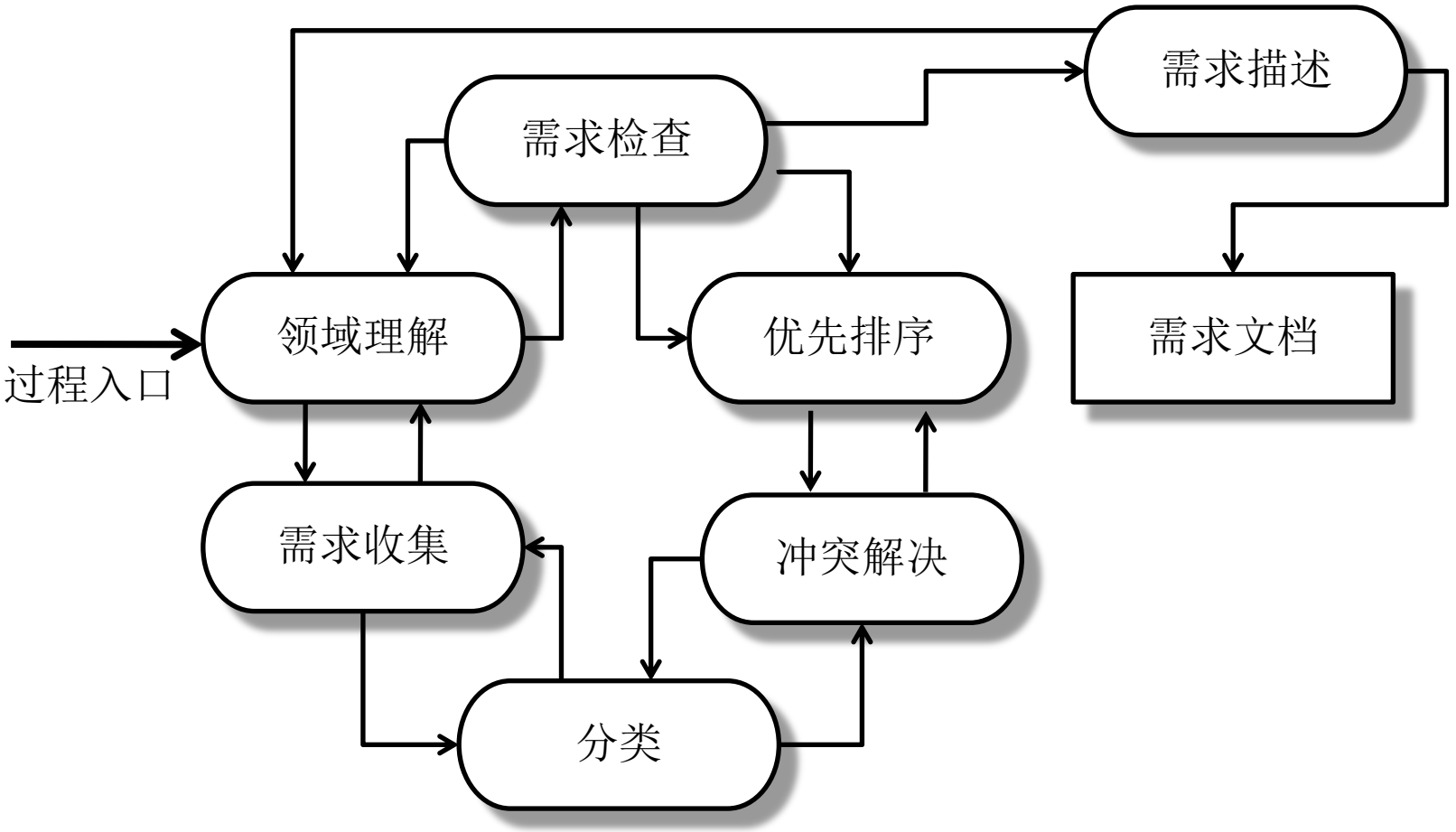
1）原型开发时难以管理，无法备全全部资料，导致存在质量隐患。

2）难以确定原型，原型目标偏离太多，则会导致系统失效。

3）比较费劲的是要把所有子系统相互融合的形式放到整个信息系统中。

4）由于多次反复，导致用户失去开始时的耐心，在不耐烦的情况下选择了不合格的原型当做最终系统。

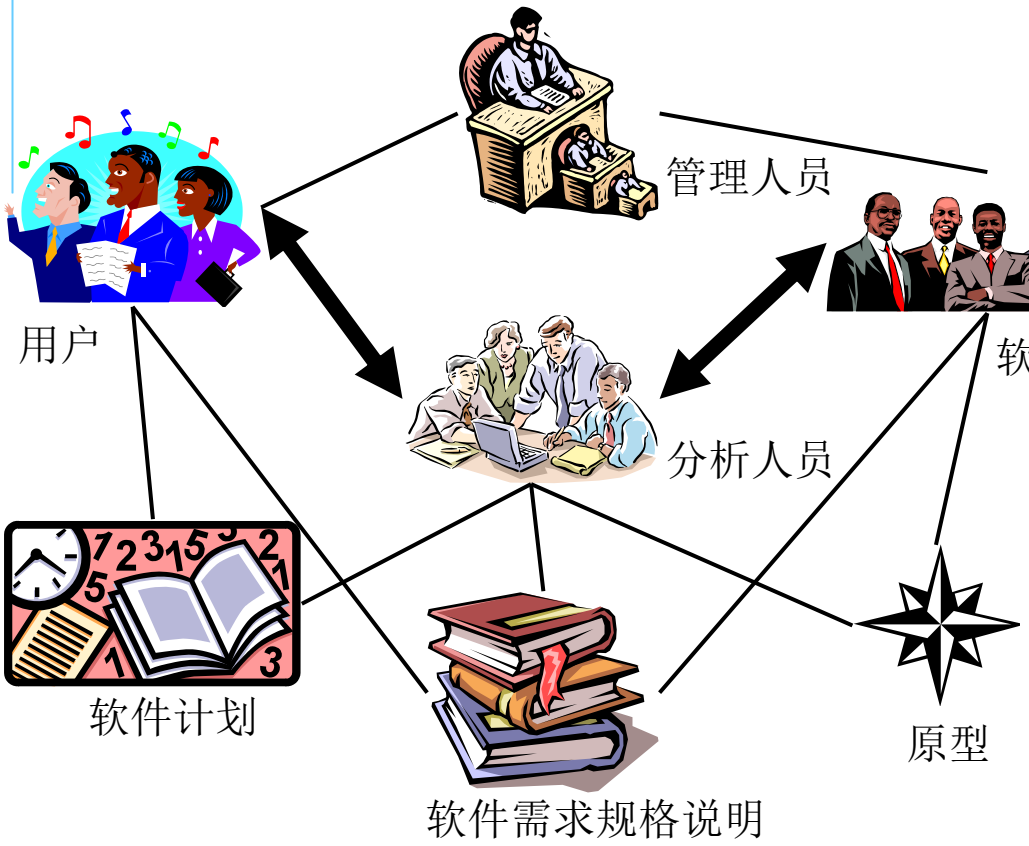
需求导出和分析过程



问题识别

- 系统分析人员要研究可行性报告和软件项目实施计划。
- 从系统角度来理解软件并评审用于产生计划估算的软件范围是否恰当。
- 确定对目标系统的综合需求，并提出这些需求实现条件，以及需求应达到的标准。
- 问题识别的另一项工作是建立分析所需要的通信途径，以保证能顺利地对问题进行分析。
- 分析员必须与用户、软件开发机构的管理部门、软件开发组成员建立联系。
- 项目负责人在此过程中起协调人的作用。

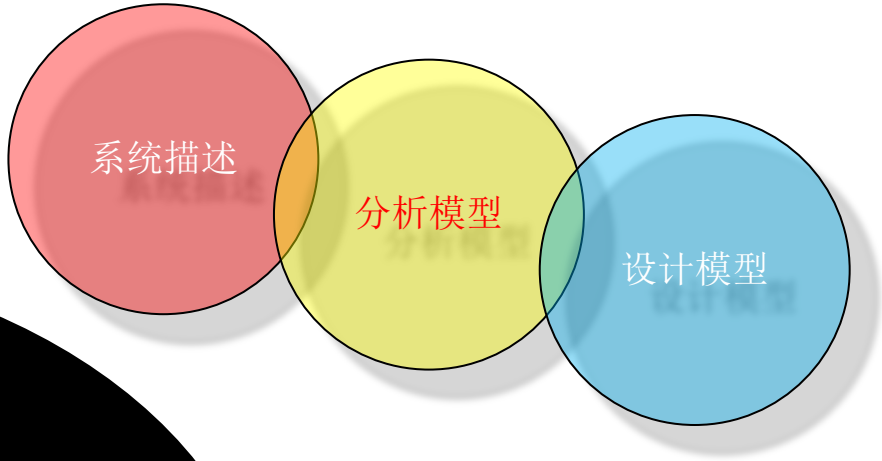
软件需求分析的通信途径

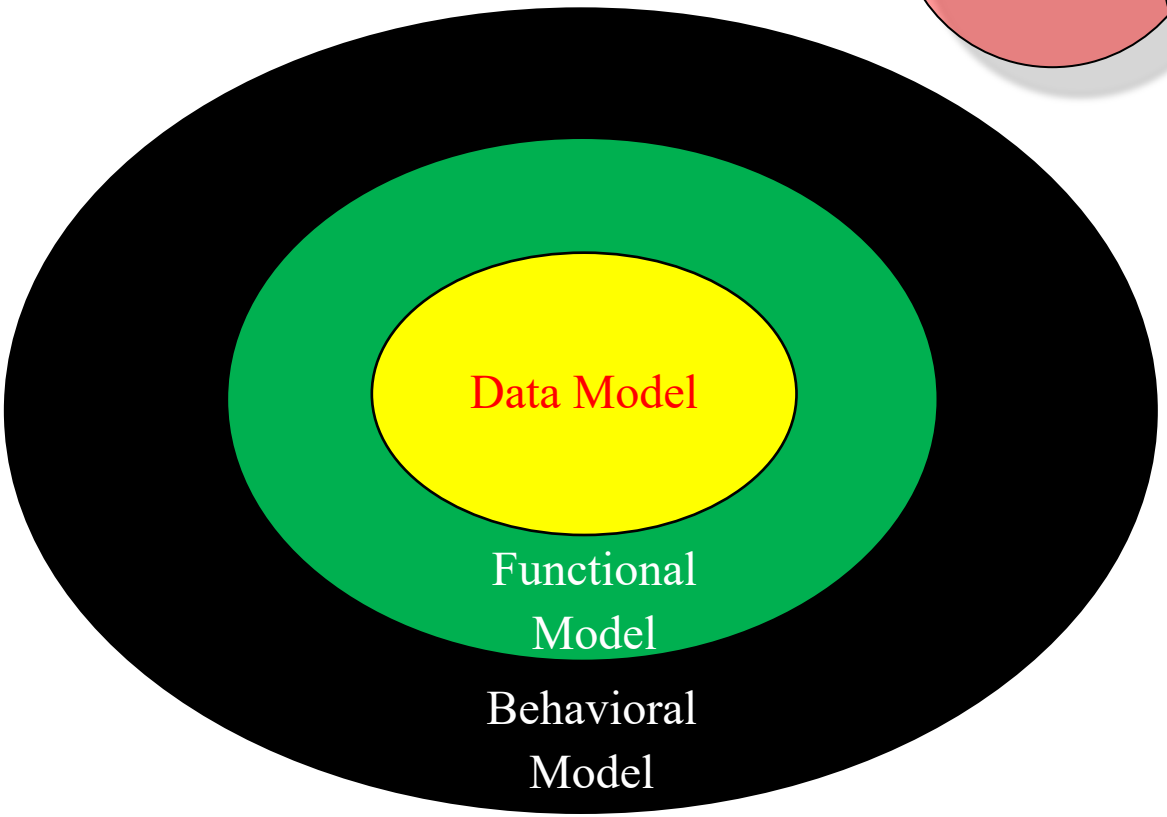


例：12306订票系统的需求项

- **终端用户方面的:**
  - 访问用户数量、总体注册用户数量、平时访问用户的峰值、平时访问用户的谷值、大假期访问用户的峰值、大假期访问用户 的谷值、小假期访问用户的峰值、小假期访问用户的谷值; 用户的地域分布性、用户可能介入的设备、用户接入的网络状况统计分析；
- **安全方面：**
  - 内部安全：主要关注资金以及交易的安全，特别是防止内部人员尤其是系统管理员；
  - 外部安全：主要关注如何确保拒绝外部恶意入侵与攻击成为一个核心，特别是类似DDOS之类的攻击；
- **后台服务方面的:**
  - 关键购票流程业务分析: 购票的基本流程分析、一次购票的TPS数量分析、一次购票的用户流量分析、一次购票的用户静态流量分析、一次购票的用户动态流量分析；
  - 这其中又分为初次购票与再次购票两种情况，流程稍微有些不同; 系统提供的其他服务统计分析; 节假日中长假只有春节与国庆,小假有元旦、清明、五一、端午、中秋等；

分析模型


系统描述
分析模型
设计模型


Data Model
Functional Model
Behavioral Model

软件需求分析的八大核心原则

原则	核心思想	要解决的问题	关键实践
以用户为中心	关注用户问题，而非技术实现	价值错位、隐含需求	用户访谈、观察、画像
问题与方案分离	先理解问题，再想解决方案	过早决策、错过更好选择	5个为什么、问题陈述
分解与抽象	分而治之，逐层细化	软件复杂度、认知负担	功能分解、用户故事地图
优先级排序	有限资源做最有价值的事	范围蔓延、资源浪费	MoSCoW法、Kano模型
可视化与原型化	让需求看得见、摸得着	无形性、理解偏差	原型、用例图、界面草图
验证与确认	每个需求都有验收标准	主观判断、"做没做完"扯皮	行为驱动开发、实例化需求
团队共识	需求是团队的，不是个人的	知识孤岛、理解不一致	需求评审、跨职能协作
持续管理	需求是活的，需要持续跟进	需求变化、蔓延失控	变更控制、需求追踪

软件需求分析方法

- **结构化分析方法**（SA 方法）
  - 面向数据流、自顶向下、逐步分解。
  - 工具：数据流图 DFD、数据字典 DD、判定表、判定树。
- **面向对象分析方法**（OOA）
  - 把系统抽象成对象、类、继承、消息。
  - 工具：用例图、类图、时序图、状态图等 UML 图。
- **原型化方法**
  - 快速构造可运行原型，让用户试用、反馈，反复迭代完善需求。
  - 适合：需求模糊、用户不确定的项目。
- **面向数据结构的分析方法**
  - 以数据结构为中心导出程序结构。
  - 典型：Jackson 方法、Warnier 方法。

软件需求分析技术

- **需求获取：从源头捕捉需求**
  - 用户访谈、问卷调查、现场观察、文档分析、头脑风暴、原型启发
- **需求建模：结构化地表达需求**
  - 结构化分析方法（传统/经典）:数据流图、实体关系图、状态转换图、数据字典
  - 面向对象分析方法（OOA，当前主流）:通常使用UML作为表示工具
  - 敏捷分析方法（轻量级/当前流行）:用户故事、用户故事地图、实例化需求、行为驱动开发
- **需求项分析：从不同角度分析需求**
  - 业务规则分析、非功能性需求分析、用户界面/体验分析

2.3 需求有效性验证

- 需求有效性验证是要检验需求能否反映客户的意愿。
- 在需求有效性验证过程中，要对需求文档中定义的需求执行多种类型的检查。

- 有效性检查的内容
  - 一致性检查
  - 完备性检查
  - 现实性检查
  - 可检验性检查

- 有效性验证技术
  - 需求评审
  - 原型建立
  - 测试-用例生成
  - 自动的一致性分析

- 软件需求的可测试性要符合以下的条件：
  - 一致性
  - 完整性
  - 明确不含糊
  - 可量化（像“反应时间快”的需求是无法被验证的），可被度量的
  - 实务上的软件验证及确认（不只是理论上可行，在有限资源下也是可实现的

确认和验证软件需求

- 确认（Validation）：
  - 是通过检查和提供客观证据，确定需求制品（artifact）是否能够反映干系人（stakeholder）真实需求的方法和过程。
  - 需求确认要确保需求制品能够追溯到其来源，并真实表达了用户需求。
  - 确认过程提供的证据表明分配给软件的系统需求能够满足用户需求，并解决了相应问题。
  - 需求确认往往是通过干系人参与完成的。
- 验证（Verification）：
  - 是通过检查和提供客观证据，证明需求制品是否符合相关质量标准（如正确性、完整性、一致性）的方法和过程。
  - 需求验证往往可以借助一些辅助验证工具来完成。
- 两者的区别：
  - 验证：是否正确做事（过程合规性）。
  - 确认：是否做了正确的事（结果有效性）。

需求评审

- 需求评审是指由项目各方利益相关者（包括客户、产品经理、开发人员、测试人员等）共同对软件需求规格说明进行系统化审查，以发现其中存在的错误、歧义、遗漏或不一致，并达成共识。
- 需求评审是一项质量控制活动，可以是正式的也可以是非正式的。

需求评审检查的内容

① 完整性
② 正确性
③ 一致性
④ 无二义性
⑤ 可行性
⑥ 可测试性
⑦ 可追溯性
⑧ 优先级合理
⑨ 界面与交互合理性
⑩ 非功能需求完备

评审的类型

- 正规检视 (Inspection)
- 技术评审 (Technical Review)
- 走读 (Walkthrough)

技术评审

- **技术评审**（technical review）的主要特点是由一组评审者按照规范的步骤对软件需求、设计、代码或其他技术文档进行仔细地检查，以找出和消除其中的缺陷。
- 正规技术评审（Formal Technical Review），也称作“正规审查”（Formal Inspection）或简称“审查”（Inspection），是一种的同行审查技术。

- 正规技术评审目的
  1. 发现软件在功能、逻辑、实现上的错误；
  2. 验证软件符合它的需求规格；
  3. 确认软件符合预先定义的开发规范和标准；
  4. 保证软件在统一的模式下进行开发；
  5. 便于项目管理。

- 正规技术评审活动过程
  1. 计划
  2. 预备会
  3. 会前准备
  4. 评审会
  5. 修正错误
  6. 复审
  7. 复核

高质量需求的特征

- 完整性
- 正确性
- 可行性
- 必要性
- 划分等级
- 无二义性
- 可验证性

影响需求质量的因素

- 用户需求不断增加
- 模棱两可的需求
- 用户不配合
- 过于精简的需求说明
- 忽略了用户的分类
- 不准确的计划
- 不必要的特性

2.4 需求管理

需求管理

为什么要管理需求?

① 大型软件系统的需求总是在变化的
② 大型软件系统总是要不断地完善
③ 大型软件系统通常拥有不同的用户群
④ 业务和技术环境的变化肯定会反映到系统本身上来

什么是需求管理？

- **需求管理**是一个对系统需求变更了解和控制的过程。
- 需求管理的过程与其它需求工程过程相互关联的。
- 初始需求导出的同时就启动了需求管理规划，一旦形成了需求文档的草稿版本，需求管理活动就开始了。
- 持久的需求
  - 这一类需求相对平稳，一般起源于一个机构的核心活动，与该系统的领域直接相关。
- 易变的需求
  - 这是一类很不稳定的需求，在整个开发过程中以及系统投入运行以后都会不断变更。

易变需求的分类

需求类型	描述
易变的需求	需求的变化源于机构所处环境的变化。
浮现的需求	在系统的开发过程中，客户对系统有了更深入的理解，于是有新的需求浮现出来。设计过程要反映这种需求。
引发的需求	需求源于计算机系统的引进，引进的计算机系统会改变机构中的业务过程，可能产生新的工作方式，这种工作方式产生了新的需求
兼容性需求	需求依赖于机构中特别的系统或业务过程。当这些在改变时，正在使用中的系统或交付的系统兼容性需求变发生了。

需求管理的活动

1. **识别与获取**：多维度挖掘真实诉求
2. **建模与文档化**：从“碎片化诉求”到“结构化需求”，需求规格说明：让“口头约定”成为“可追溯的契约”
3. **需求验证**：需求验证与确认，确保“做的事”="要的事”
4. **优先级管理**：资源永远有限，必须决定“先做什么，后做什么，甚至不做什么”。
5. **变更控制**：在“灵活响应”与“项目失控”间找平衡
6. **跟踪与维护**：看住需求，维护“需求-设计-实现-测试”的可追溯性

需求跟踪

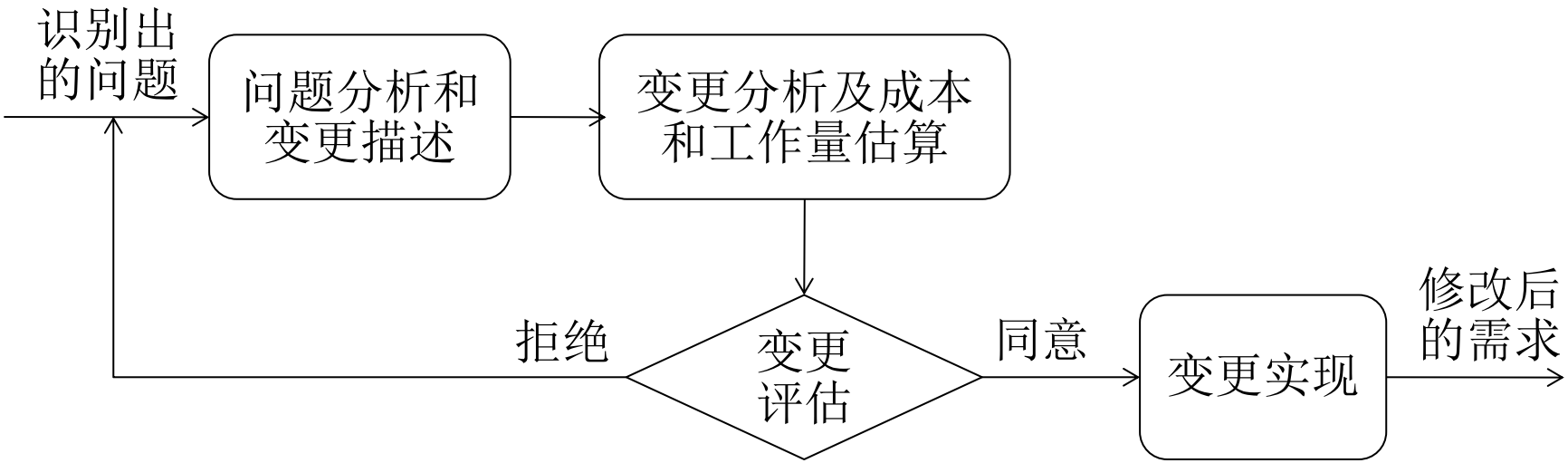
- **需求跟踪**是指建立需求与其前后两个方向相关元素之间关联关系的活动，其目的是建立与维护“需求来源-需求规约-设计文档-程序代码-测试用例”之间的一致性，确保所有的工作成果符合用户的需求。
- 需求跟踪又可以分为：
  - 前向需求跟踪(Pre-Requirements Traceability)
  - 后向需求跟踪(Post-Requirements Traceability)
- 需求跟踪表示技术：
  - 需求跟踪矩阵
  - 需求跟踪关系表
  - 需求跟踪图
- 需求跟踪的作用如下：
  - 严控项目范围，防止需求蔓延与偏离
  - 提升变更管理效率，精准评估变更影响
  - 保障软件质量，确保需求实现的完整性
  - 满足合规审计要求，提供可验证的证据链

需求维护的3类可追溯性信息

1. 源可追溯性信息
   - 连接需求到提出需求的项目相关人员和这些需求的基本原则。
   - 以便变更发生时，与相关人员商讨变更事宜。
2. 依赖可追溯性信息
   - 连接需求文档中彼此依赖的需求。
   - 这个信息用来评估一个变更会对多少需求产生影响以及引发的需求变更的范围和程度。
3. 设计可追溯性信息
   - 连接需求到其实现的设计模块。
   - 这个信息用来评估需求变更对系统设计和实现带来的影响。

需求变更管理

变更管理应该处理全部的需求变更。



软件开发

3 软件需求分析

主要内容

① 需求分析的任务

② 需求内容

③ 需求文档

④ 举例：需求分析

4.1 软件需求分析的任务

1. 需求分析是借助于当前系统导出目标系统的逻辑模型，解决目标系统做什么。

2. 深入描述软件的功能和性能，确定软件设计的限制和软件同其它系统元素的接口细节，定义软件的其它有效性需求。

3. 软件需求规格说明还为评价软件质量提供依据。

需求分析的任务

软件需求定义

软件工程术语表 GB/T11457-2006和IEEE软件工程标准词汇表（1997年）中的需求（requirement）定义：

①用户解决问题或达到目标所**需求的条件或性能**（capability）。

②系统或系统部件要**满足**合同、标准、规范或其他正式规定文档或性能。

③一个反映上面①或②所描述的条件或性能的文档说明。

定义用户需求一般从下面三方面来展开：

- 构建用户角色
- 描述使用场景
- 定义用户问题

需求分类

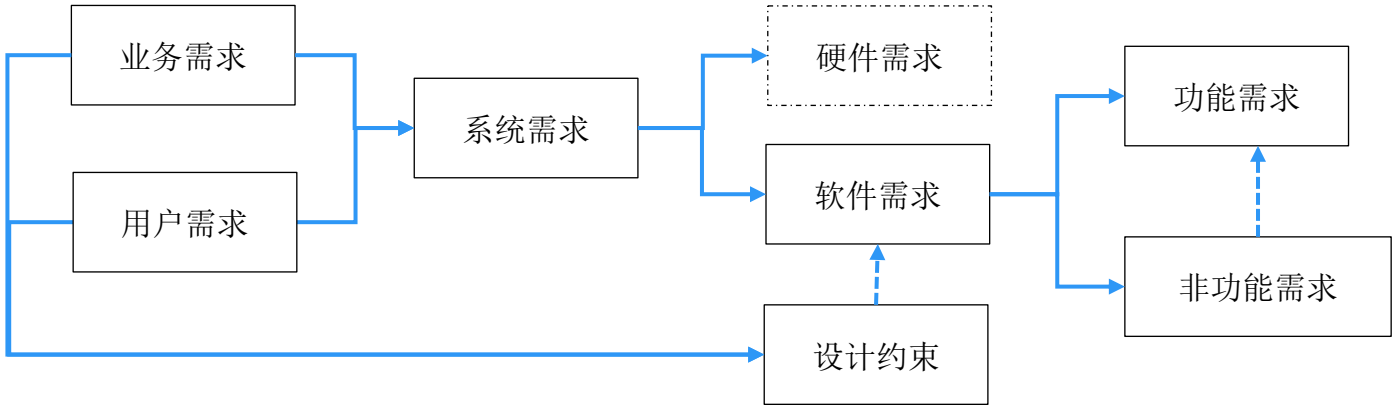
由于来源不同、层次不同、粒度不同，需求可以细分为不同类别，以外卖平台系统为例。

- **业务需求**
  - 业务需求又分为目标层业务需求和操作层业务需求。
  - 例如：业务目标“外卖平台系统应该支持企业成为全国最大的餐饮中介服务提供商，为百万级餐饮服务提供商及亿级消费者提供餐饮服务”即为目标层业务需求。业务规则“只有已支付订单才能进行配送”即为操作层业务需求。
- **用户需求**
  - 用户需求是指描述用户使用待开发系统需要完成什么任务、怎么完成的需求。
  - 例如：消费者的需求“希望能够在下单后30分钟内拿到餐品”和快递小哥的需求“希望在用餐高峰期和天气不好时延长送餐时限”就互相矛盾。
- **系统需求**
  - 系统需求是指描述包含有多个子系统的系统的顶级需求。
  - 例如：可以包含外卖平台商家子系统、骑手子系统和消费者子系统，这三个子系统均为软件子系统。
- **软件需求**
  - 软件需求是指用户对目标软件产品在功能、行为、界面、性能、设计约束、质量保障等方面的期望。
  - 例如：用户需求“希望在用餐高峰期和天气不好时延长送餐时限”对应的软件需求为“系统应该提供送餐时限自定义功能，并支持根据设定的条件和当前状态自动计算送餐时限”。

（续）

- **功能需求**
  - 功能需求是指目标软件产品必须实现的软件功能或软件系统应具有的外部行为。
  - 例如：“系统应该提供送餐时限自定义功能”、“系统应该支持根据设定的条件和当前状态自动计算送餐时限”、“用户账户中的余额不足时支付不能成功”都是功能需求。
- **非功能需求**
  - 非功能需求是指目标软件产品为满足软件需求而必须具备的、除功能需求以外的需求。
  - 例如：界面需求“系统应该支持不少于4种界面风格”、对外接口需求“系统应该提供外部支付系统接入接口”、数据需求“上传的菜品图片分辨率不低于512*384”、性能需求“系统支付功能应该能够支持百万用户并发访问”等都是非功能需求。
- **设计约束**
  - 设计约束是指软件开发人员在设计和实现目标软件产品时必须满足的要求和必须遵从的标准规范。
  - 例如：时间约束“系统必须在半年内上线”、资源约束“系统必须运行在云平台上”、文化约束“系统应该使用红色显示注意事项或警告标志”、法律约束“系统只能允许具有营业执照和餐饮服务许可证的商家上线提供服务”等都是设计约束。

需求之间的关系



通过调研获取业务需求和用户需求后，通过整理分析得到系统需求，再从中分离出由软件子系统实现的需求构成软件需求，而后将其进一步具体化为对目标软件产品的功能需求和非功能需求，对于目标软件产品宏观、整体的要求，则记为设计约束。

Categories of Software Requirements

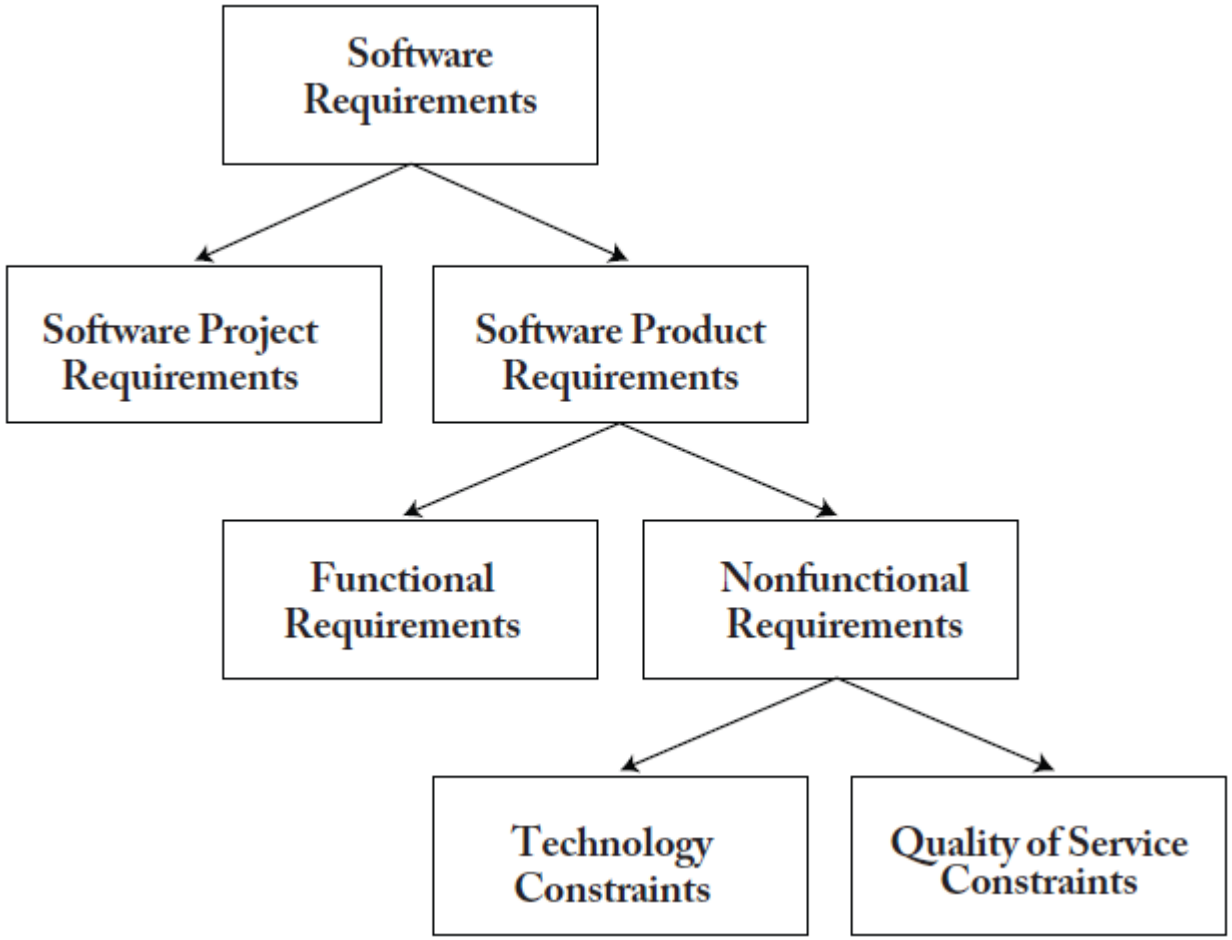


Figure 1.2. Categories of Software Requirements

SWEBOK® GUIDE V4.0

软件需求的3个不同层次

3.2 需求的内容

2
业务需求
目标层：主体希望通过系统开发达成的目标。
操作层：系统必须支持完成的业务、任务、工作流程或必须遵守的业务规则等。
1
系统需求
详细给出系统将提供的服务的约束，也称为功能描述
是甲已双方合同重要内容
3
用户需求
用自然语言加图表描述
描述系统提供的服务及约束
设计约束
指软件开发人员在设计和实现目标软件产品时必须满足的要求和必须遵从的标准规范。
设计约束包括企业或组织对目标软件产品设计、构造、运维上的特殊限制和源于法律法规、风俗文化的要求。

需求场景

- **需求场景**（Requirements Scenarios）是指在什么情况下会产生这种需求，是需求发生的环境和触发条件的一种总称。
- 需求场景主要包括四种类型：
  - 正常的用例场景
  - 备选的用例场景
  - 异常的用例场景
  - 假定推测的场景
- 确定用户需求的场景四要素：
  - 时间
  - 空间
  - 用户
  - 行为

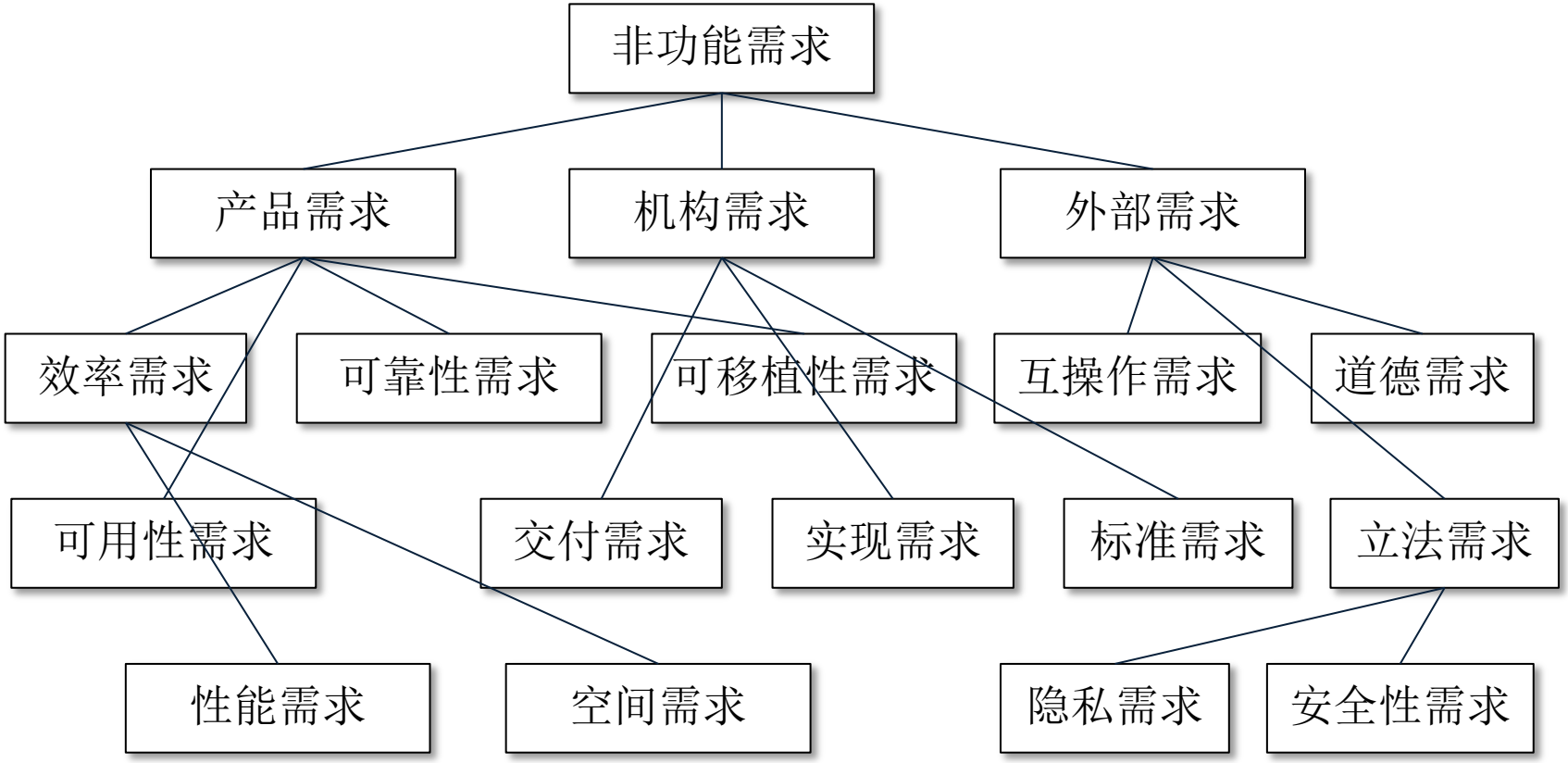
什么是功能性需求

- 需求按照功能性（行为的）和非功能性（其它所有的行为）来分类。
- **功能性需求**（functional requirement）规定开发人员必须在产品中实现的软件功能，用户利用这些功能来完成任务，满足业务需求。
  - 例如：用户登录、订单查询、电子支付、论坛网站的发帖留言等。
- 功能需求有时也被称作**行为需求**（behavioral requirement），通常用“应该”对其进行描述：“系统应该发送电子邮件来通知用户已接受其预定”。

什么是非功能性需求

- 非功能性需求（Non-functional requirement）是指依据一些条件判断系统运作情形或其特性，而不是针对系统特定行为的需求。
- 一般而言，功能需求会定义系统的行为，而非功能性需求会定义系统的特性。
- 非功能性需求一般被称为**系统的品质**，或限制、品质属性、品质目标、品质服务需求、非行为性的需求。
- 非功能性需求可以分为以下二类：
  - **运行品质**（Execution qualities），在系统运行时观察到的品质，例如安全性及易用性等。
  - **发展品质**（Evolution qualities），与软件系统结构及开发过程有关的品质，例如软件可测试性（testability）、可维护性、可扩展性、可伸缩性（scalability）等。

非功能需求的类型


非功能需求
产品需求
机构需求
外部需求
效率需求
可靠性需求
可移植性需求
互操作需求
道德需求
可用性需求
交付需求
实现需求
标准需求
立法需求
性能需求
空间需求
隐私需求
安全性需求

例：非功能性需求实例

产品需求

4.C.8 它应该能将所有JEE和用户之间的必需的通信用标准的Java字符表达式。

机构需求

9.3.2 系统开发过程和交付的文档将遵照YXZCo-SP-STAN-95中的相关定义。

外部需求

7.6.5 系统不应该对系统的操作人员公开客户除名字和索引代码之外的任何个人信息。

例：系统目标和可验证的需求

系统目标
系统应该很好用，即使对一个没有经验的用户，而且应该使得用户错误降到最少。
可检验的非功能需求
对一个没有经验的用户来说，经过两个小时的培训就应该能使用系统的所有功能。在这样的培训之后，一个有经验的用户每天的出错平均数不应超过两次。

例：指定非功能需求的度量

性质	度量方法
速度	每秒处理的事务 用户/事件响应时间 屏幕刷新时间
规模	K字节 RAM芯片数
易用性	培训时间 帮助画面数
可靠性	失败平均时间 无效的概率 失败发生率 有效性
健壮性	失败之后的重启次数 事件引起失败的百分比 失败中数据崩溃的可能性
可移植性	依赖于目标的语句百分比 目标系统数

例：“可用性”描述

- 可用性（Availability）的定义：
  - ISO9241/11的定义： The extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use. (一个产品可以被特定的用户在特定的境况中，有效、高效并且满意得达成特定目标的程度。)
  - GB/T3187-97的定义：在要求的外部资源得到保证的前提下，产品在规定的条件下和规定的时刻或时间区间内处于可执行规定功能状态的能力。它是产品可靠性、维修性和维修保障性的综合反映。
- 可用性就是一个系统处在可工作状态的时间的比例，用于衡量一个系统正确地对外提供服务（可工作）的能力。
- 通常采用SLA（Service Level Agreement）来衡量系统可用性，也就是经常听到的的几个9，其对应的系统不可用时间可以参照表：

系统可用性%	不可用时间/年	不可用时间/月
90%	36.5 天	72 小时
99%	3.65 天	7.20 小时
99.9%	8.76 小时	43.8 分
99.99%	52.56 分	4.38 分
99.999%	5.26 分	25.9 秒

3.3 需求文档

需求规约

- **需求规约**（requirements specification）是对系统需求的规范化描述。包括功能和行为描述、性能需求和设计约束、验收标准等。
- 需求规约是把已经确认的需求通过文档的形式固化下来，难点是能够清晰的描述需求规格。

需求描述所使用的符号

- **结构化自然语言**
  - 该方法依赖于定义标准格式或模板来表达需求
- **设计描述语言**
  - 该方法使用一种类似于程序设计语言，但是具有更多抽象特征，通过定义系统的操作模型来定义需求。
- 图形化符号
  - 图形语言辅之以文本注释来定义系统的功能需求。如：SADT、Use-case等。
- **数学描述**
  - 基于像有限状态机或集合这样的数学概念的符号。
  - 无二义性描述减少了客户和承包商之间关于功能的争论。然而，绝大多数客户不懂形式化描述，因而不愿接受这样的系统合同。

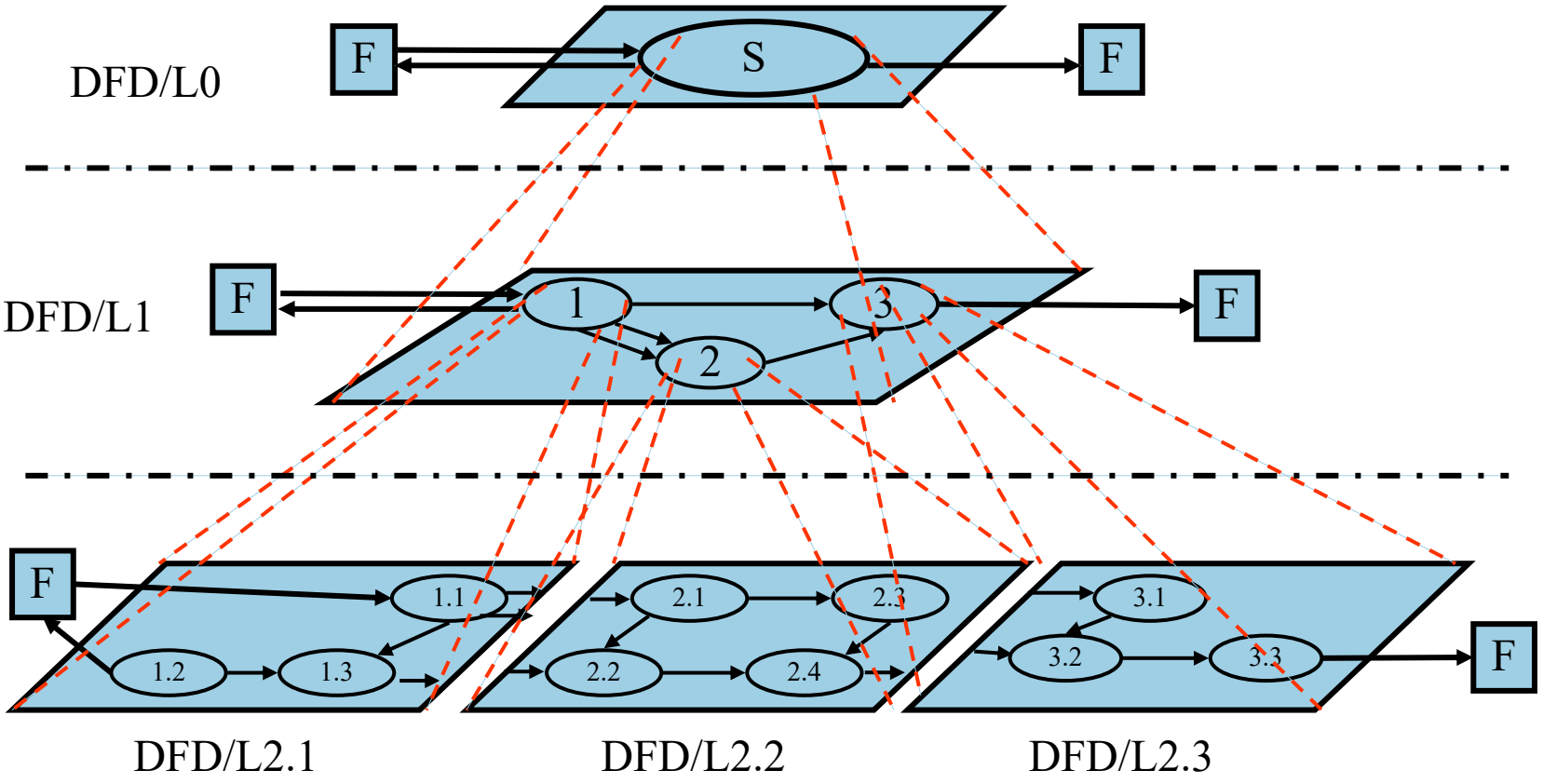
例：使用标准格式的系统需求描述

功能	
描述	
输入	
来源	
输出	
目的地	
要求	
前条件	
后条件	
副作用	

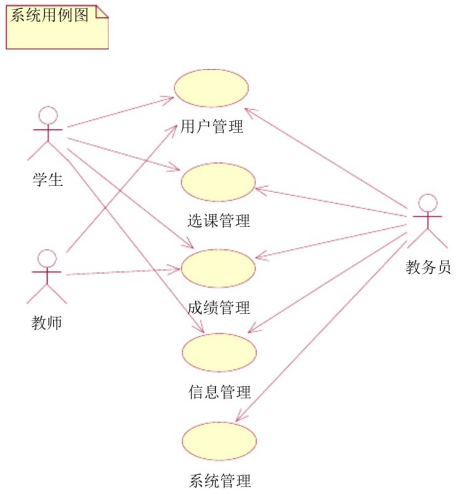
例：打印服务器接口的Java PDL描述

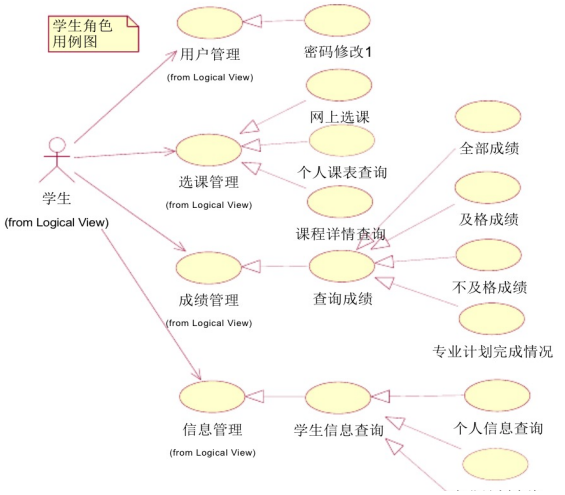
```
interface PrintServer {
// defines an abstract printer server
// requires: interface Printer, interface PrintDoc
// provides: intialize, print, displayPrintQueue, cancel PrintJob, switchPrint
    void intialize ( Printer P );
    void print ( Printer p, PrintDoc d );
    void displayPrintQuene (Printer p );
    void cancelPrintJob (Printer p, PrintDoc d );
    void switchPrinter ( Printer p1, Printer p2, PrintDoc d );
} //PrintServer
```

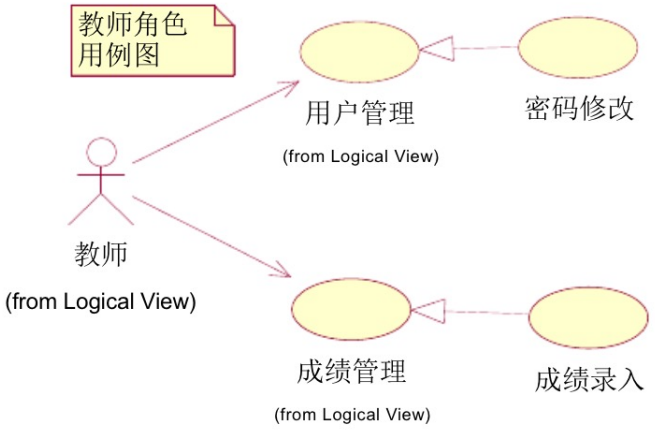
例：分层数据流图

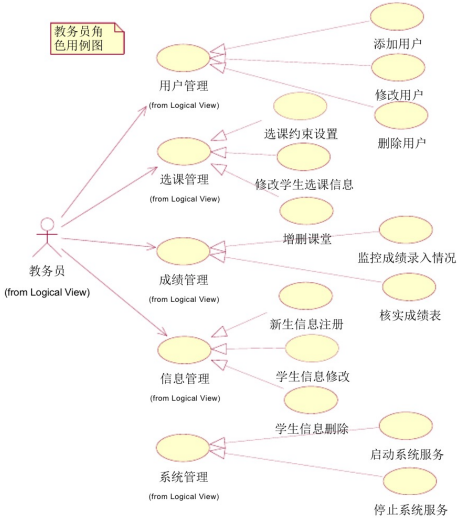

DFD/L0
F
S
F
DFD/L1
F
1
2
3
F
F
1.1
1.2
1.3
2.1
2.2
2.3
2.4
3.1
3.2
3.3
F
DFD/L2.1
DFD/L2.2
DFD/L2.3

例：UML描述的模型


系统用例图
学生
教师
用户管理
选课管理
成绩管理
信息管理
系统管理
教务员


学生角色
用例图
用户管理
(from Logical View)
密码修改1
网上选课
个人课表查询
选课管理
(from Logical View)
课程详情查询
学生
(from Logical View)
全部成绩
及格成绩
不及格成绩
专业计划完成情况
成绩管理
(from Logical View)
查询成绩
信息管理
(from Logical View)
学生信息查询
个人信息查询


教师角色
用例图
用户管理
(from Logical View)
密码修改
教师
(from Logical View)
成绩管理
(from Logical View)
成绩录入


教务员角
色用例图
用户管理
(from Logical View)
添加用户
修改用户
删除用户
选课约束设置
选课管理
(from Logical View)
修改学生选课信息
增删课堂
教务员
(from Logical View)
监控成绩录入情况
成绩管理
(from Logical View)
核实成绩表
新生信息注册
信息管理
(from Logical View)
学生信息修改
学生信息删除
启动系统服务
系统管理
(from Logical View)
停止系统服务

软件开发

4 结构化分析

主要内容

① 结构化分析思想

② 数据流图

③ 数据字典

结构化

- **结构**（structure）是指系统内各个组成要素之间的相互联系、相互作用的框架。
- **结构化**（Structured）是将复杂的事物按照一定的规则、逻辑和层次进行组织，使其变得清晰、有序且易于管理。
- 结构化是一套分层、分块、有序、规范的软件构造思想，核心是把复杂系统拆成层级清晰、边界明确、逻辑线性的单元。

结构化开发方法

- **结构化开发方法**（structured development method），又被称为**面向数据流的开发方法**，或**面向过程的开发方法**，
- 从早期的“结构化编程”到如今的“结构化数据”、“结构化设计”，这一理念对软件的意义主要体现在以下几个核心方面：
  ① 降低复杂性（化繁为简）
  ② 提高可维护性与可读性
  ③ 促进代码复用与模块化
  ④ 便于分工协作，支持团队开发
  ⑤ 提升测试与调试的效率
  ⑥ 数据层面的结构化
  ⑦ 提升可维护性（长期项目最关键价值）

4.1 结构化分析思想

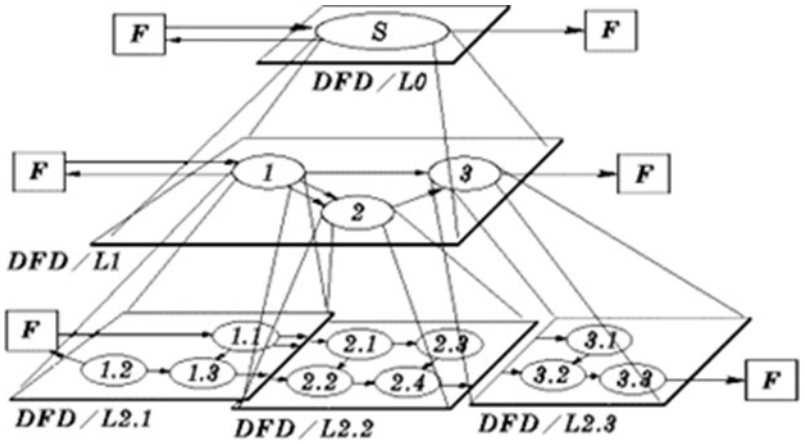
- **结构化分析方法**（SA，Structure Analysis）是面向数据流的需求分析方法，以数据在不同模块中移动的观点来看待一个系统，系统的功能可以用转换数据流的程序来表示，适合于数据处理类型软件的需求分析。
- SA基本思想是：
  - 抽象概念（**功能分解**和**抽象**）
  - 面向数据流（数据在系统内**流动**、**加工**、**存储**）
  - 自顶向下逐层分析（**结构化**、**模块化**、**自顶向下**）

结构化分析的步骤如下：

① 分析当前的情况，做出反映当前物理模型的DFD；
② 推导出等价的逻辑模型的DFD；
③ 设计新的逻辑系统，生成数据字典和基元描述；
④ 建立人机接口，提出可供选择的目标系统物理模型的DFD；
⑤ 确定各种方案的成本和风险等级，据此对各种方案进行分析；
⑥ 选择一种方案；
⑦ 建立完整的需求规约。

SA常用的工具

- 数据流图，DFD
- 数据字典，DD
- 结构化英语
- 判定树和判定表

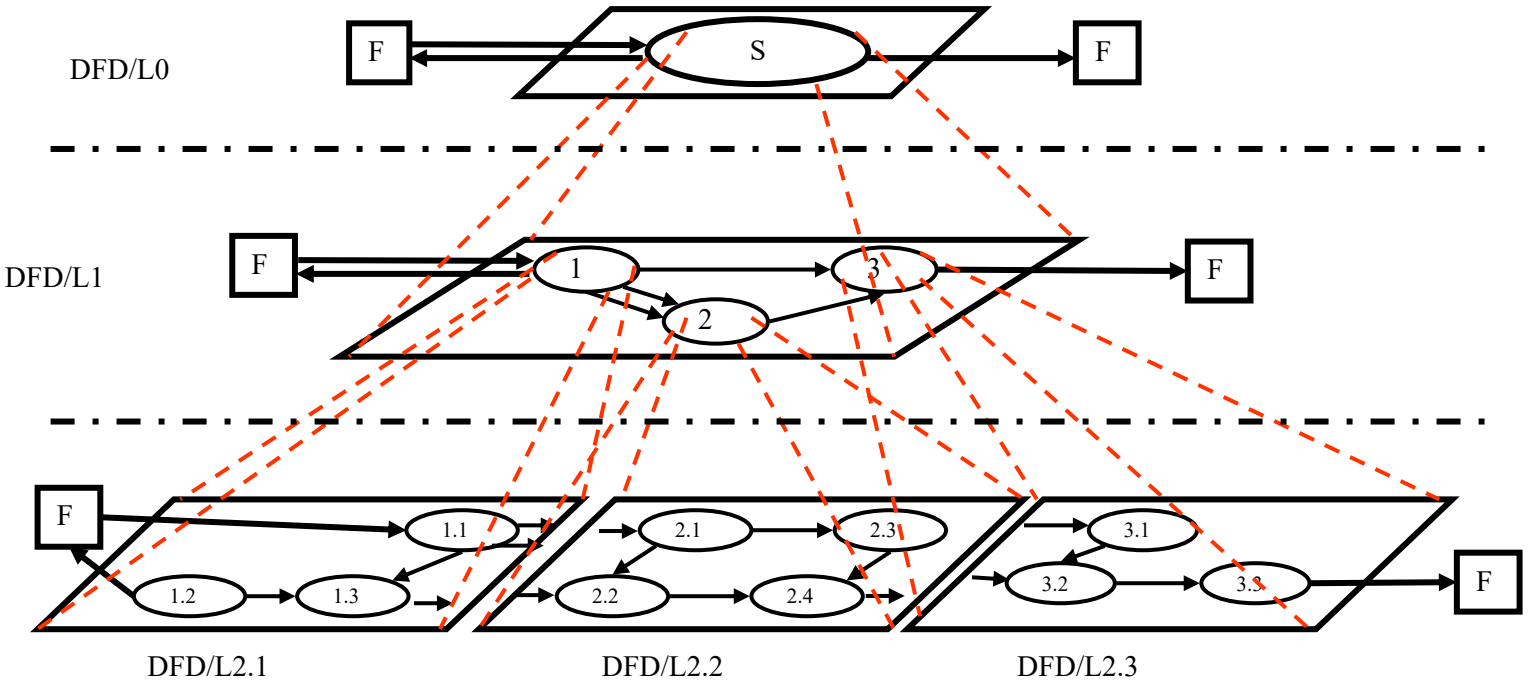


SA方法的不足

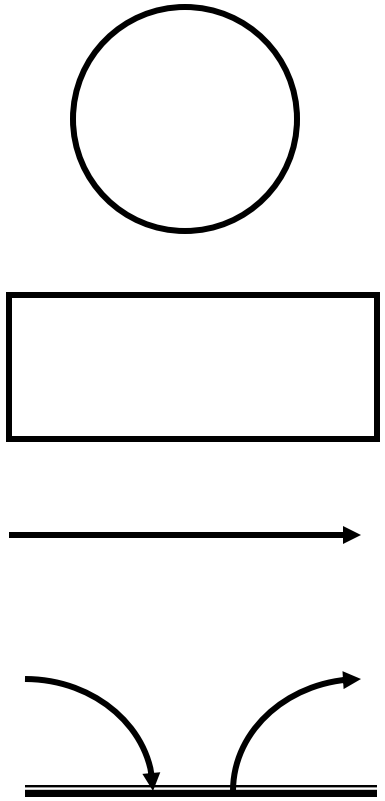
- 不提供对非功能性系统需求的有效理解和建模。
- 不提供对用户选择的合适方法的指导，也没有对方法适用的特殊环境的忠告。
- 往往产生大量的文档，系统的需求要素被隐藏在一大堆具体细节的描述中。
- 产生的模型太注重细节以致用户觉得难以理解，因而很难去验证这些模型的真实性。

4.2 数据流图

- **数据流图**（DFD，Data Flow Diagram）也称Bubble Chart或Data Flow Graph，是描述数据处理的工具。
- DFD从数据传递和加工的角度，以图形的方式刻图数据流从输入到输出的移动变换过程。
- DFD只反映系统必须完成的逻辑功能，是一种功能模型。

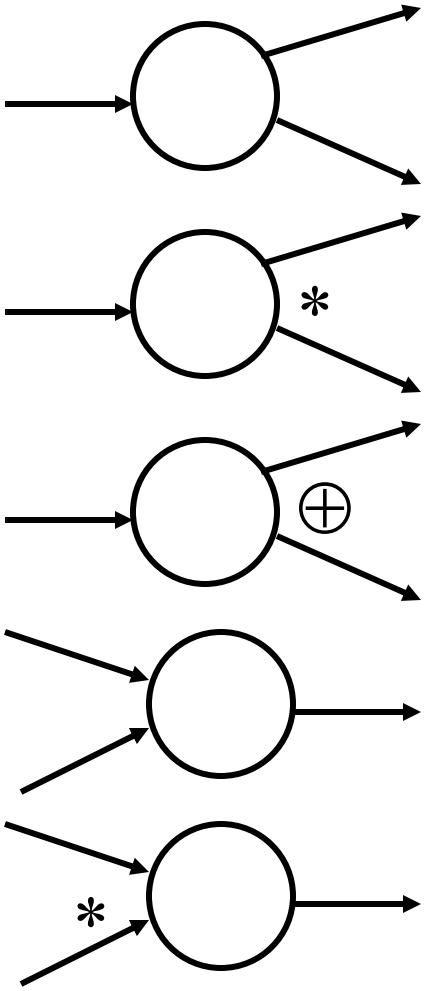


DFD的主要图形元素：



- 加工
  - 输入数据在此进行变换产生输出数据
  - 其中要注明加工的名字
- 源点和汇点
  - 其中要注明源点和汇点的名字
- 数据流
  - 被加工的数据与流向
  - 要注明数据流名字，可用名词、名词性短语。
- 数据存储文件
  - 要注明文件名字，用名词或名词短语

表明多个数据流与加工之间关系的符号



有A则有B或C，或两者都有

有A则有B与C，两者都有

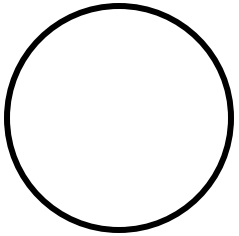
有A则有B或C，但不会同时有B与C

当A或B有一个存在，就有C

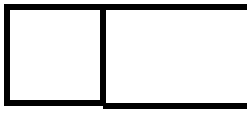
当A与B都存在，就有C

符号对照

数据源点和汇点



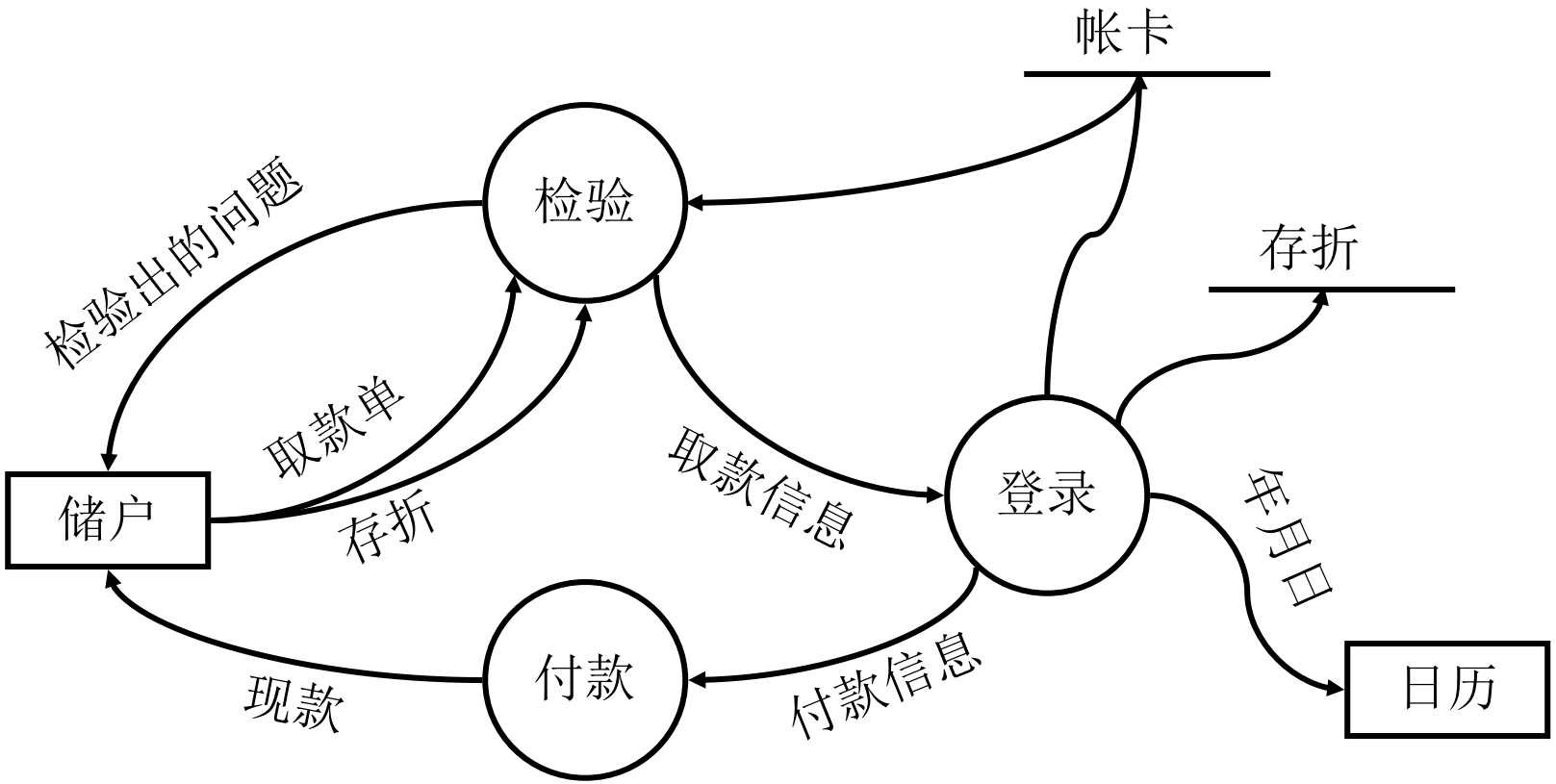
加工



数据文件

数据流

办理取款手续的数据流图



数据流图的作用：

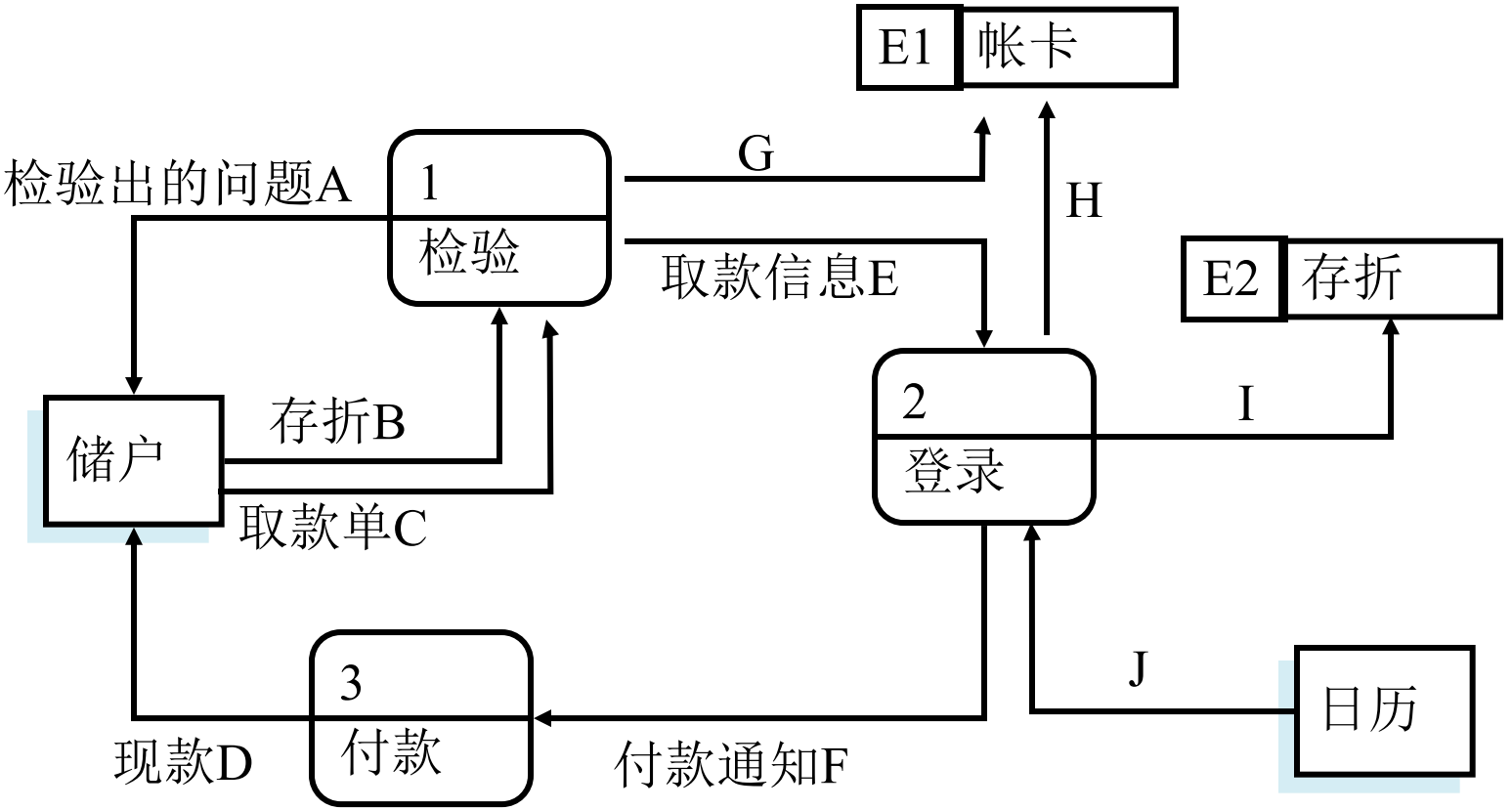
1）便于用户表达功能需求和数据需求及其联系；

2）便于两类人员共同理解现行系统和规划系统的框架；

3）清晰表达数据流的情况；

4）有利于系统建模。

银行取款过程的数据流图



等价矩阵

	储户	日历	E1帐卡	E2帐卡	检验	登录	付款
储户	—					B，C	
日历		—				J	
E1帐卡			—		G		
E2存折				—			
检验	A		H	I	—	E	
登录						—	F
付款	D						—

画单张数据流图时，必须注意以下原则：

1. 一个加工的输出数据流不应与输入数据流同名，即使它们的组成成分相同。
2. 保持数据守恒。也就是说，一个加工所有输出数据流中的数据必须能从该加工的输入数据流中直接获得，或者说是通过该加工能产生的数据。
3. 每个加工必须既有输入数据流，又有输出数据流。
4. 所有的数据流必须以一个外部实体开始，并以一个外部实体结束。
5. 外部实体之间不应该存在数据流。

DFD的画法

一般遵循“由外向里”的原则，即先确定系统的边界或范围，再考虑系统的内部，先画加工的输入和输出，再画加工的内部。即：

(1)识别系统的输入和输出。

(2)从输入端至输出端画数据流和加工，并同时加上文件。

(3)加工的分解“ 由外向里”进行分解。

(4)数据流的命名，名字要确切，能反映整体。

(5)各种符号布置要合理，分布均匀，尽量避免交叉线。

(6)先考虑稳定态，后考虑瞬间态。如系统启动后在正常工作状态，稍后再考虑系统的启动和终止状态。

4.3 数据字典

- **数据字典**（DD，Data Dictionary）就是用来描述数据流和加工的具体含义。
- DFD只描述了系统的“分解”，它没有表达出各个数据和加工的具体含义。
- 有了DFD和DD，才算是较完整的地描述了一个系统。
- 对于DFD中出现的所有命名的图形元素在DD中作出一个词条加以定义，使得每一个图形元素的名字有一个确切的解释。

数据流词条描述

- **数据流名：**
  - 说明：简要介绍作用，即它产生的原因和结果。
  - 数据流来源：来自何方。
  - 数据流去向：去向何处。
  - 数据流构成：数据结构。
  - 每个数据量流通量：数据量，流通量。
- **数据元素名：**
  - 类型：数字（离散值，连续值），文字（编码类型）
  - 长度
  - 取值范围
  - 相关的数据元素及数据结构
- **数据文件名：**
  - 简述：存放的是什么数据。
  - 输入数据
  - 输出数据
  - 数据文件组成：数据结构
  - 存储方式：顺序，直接，关键码
  - 存取频率

加工逻辑词条描述

- **加工名：**
  - 加工编号：反映该加工的层次。
  - 简要描述：加工逻辑及功能简述。
  - 输入数据流
  - 输出数据流
  - 加工逻辑：简述加工程序，加工顺序。
- **源点及汇点词条描述**
  - 名称：外部实体名称。
  - 简要描述：什么外部实体。
  - 有关数据流
  - 数目

在数据字典的定义式中出现的符号

符号	含义	解释
=	被定义为	
+	与	x=a+b，表示x由a和b组成
[…,…]	或	x=[a,b]，x=[a|b]，表示x由a或由b组成
[…|…]	或	
{…}	重复	x={a}，表示x由0个或多个a组成
m{…}n	重复	x=3{a}8,表示x中至少出现3次a,至多出现8次a
(…)	可选	x=(a)，表示a可在x中出现，也可不出现
"…"	基本数据元素	x="a"，表示x可取值为a的数据元素
..	连接符	x=1..9，表示x可取1到9之中的任一值

例：存折格式

户名　　所号　　帐号

开户日　　性质　　印密

日期 处月日	摘要	支出	存入	余额	操作	复核

例：存折在DD中的定义

- 存折=户名+所号+帐号+开户日+性质+(印密)+1{存取行}50
- 户名=2{字母}24
- 所号="001"…"99999999"
- 开户日=年+月+日
- 性质="1"…"6"
- 印密="0"
- 存取行=日期+(摘要)+支出+存入+余额+操作+复核
- 日期=年+月+日
- 年="00".."99"
- 月="01".."12"
- 日="01".."31"
- 摘要=1{字母}4
- 支出=金额
- 金额="0000000.01".."9999999.99"
- 操作="00001".."99999"

4.4 加工逻辑说明

写基本加工逻辑的说明时，应满足如下要求：

- 对DFD的每一个基本加工，必须有一个加工逻辑说明；
- 加工逻辑说明必须描述基本加工如何把输入数据流变换为输出数据流的加工规则；
- 加工逻辑说明必须描述实现加工和策略而不是实现加工的细节。

结构化英语（Structure English）

- 是在自然语言基础上加入一些限制而得到的语言，是使用有限的词汇和有限的语句来描述加工逻辑。
- 由命令动词、数据词典中定义的名字、有限的自定义词和控制结构关系词等组成。

例：商店业务处理系统中“检查发货单”

```
IF the invoice exceeds $500 THEN
        IF the account has any invoice more than 60 days overdue THEN
                the confirmation pending resolution of the debt
        ELSE (account is in good standing)
                issue confirmation and invoice
        ENDIF
ELSE (invoice $500 or less)
        IF the account has any invoice more than 60 days overdue THEN
                issue confirmation, invoice and write message on credit action report
        ELSE (account is in good standing)
                issue confirmation and invoice
        ENDIF
ENDIF
```

判定表（Decision Table）

- 判定表能够把在什么条件下系统应完成哪些操作，表达得十分清楚、准确、一目了然。这是用语言说明难以准确、清楚表达的。判定表的组成如下：

I条件类别	II条件组合
III操作	IV操作执行

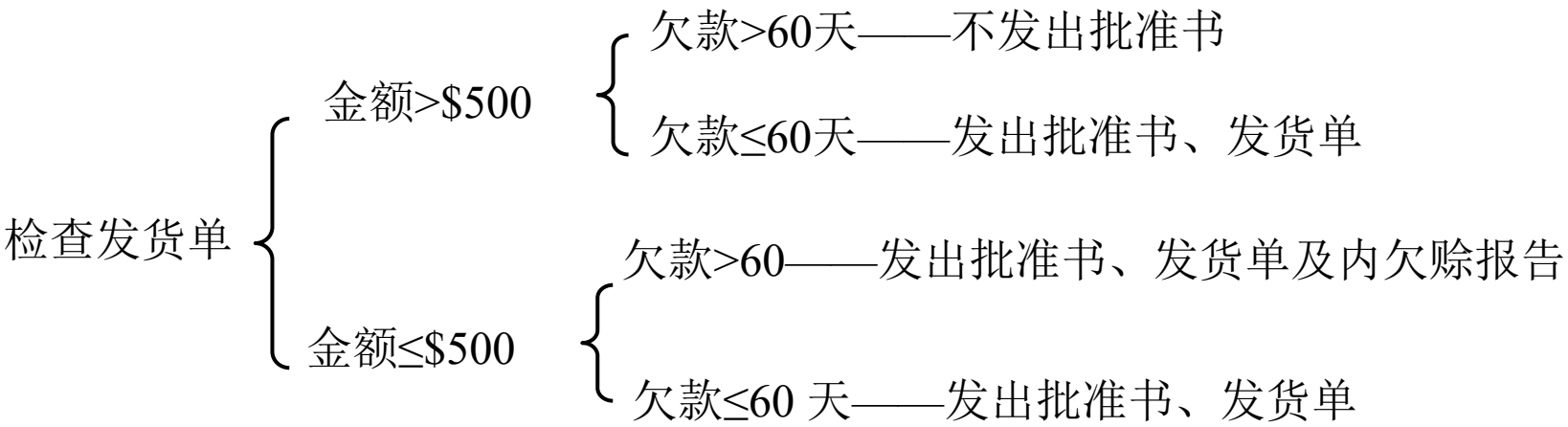
- 判定表分4个区：
  - I条件类别：列出所有条件类别
  - II条件组合：列出所有条件组合
  - III操作：列出所有操作
  - IV操作执行：标出在相应的组合条件下，某个操作是否执行或执行情况。

例：“检查发货单”的判定表

		1	2	3	4
条件	发货单金额	＞$500	＞$500	≤$500	≤$500
	赊欠情况	＞60天	≤60天	＞60天	≤60天
操作	不发出批准书	√			
	发出批准书		√	√	√
	发出发货单		√	√	√
	发出赊欠报告			√	

判定树（Decision Tree）

- 用图形描述加工逻辑，其特点是结构清晰，易读易懂。
- 例：“检查发货单”的判定树



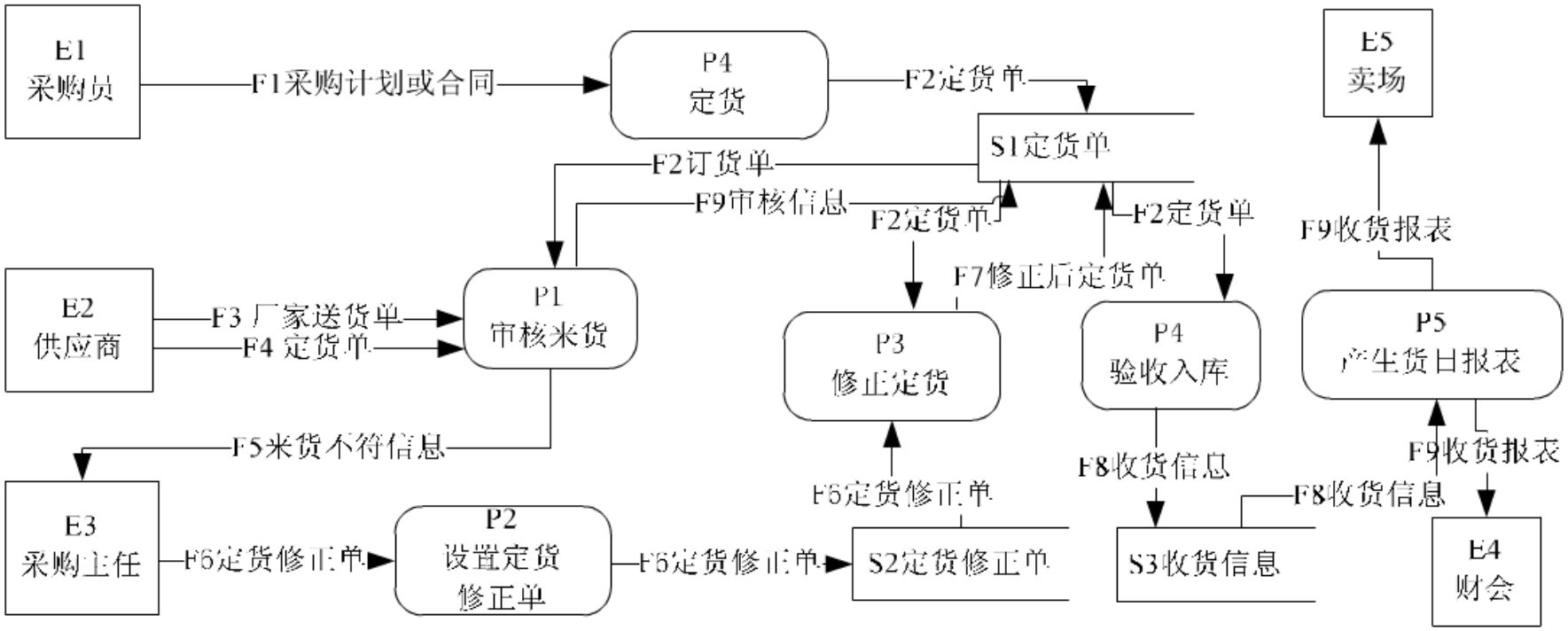
练习题：连锁超市进货操作流程为：厂家持“定货单”及“厂家送货单”送货，库管科检查货物。

- 如果来货相符，库管科做收货确认，（包括：在“定货单”之“点核数量”栏手工填写收货数量；在电脑中找出“定货单”，在“点核数量”栏输入收货数量；在“收货日报表”上记录该收货；在“定货单”、“厂家送货单”上加盖库管科公章），然后库管科打印商品条码，收送货组与卖场办理移交手续，卖场收货人在“定货单”上签名盖章，日末，库管可将单据汇总，连同“收货日报表”分别送卖场、财会，并自留一份，第二天电脑科打印“验收进货单”，分送卖场、采购、财会。
- 如果来货不符，通知负责该次定货之采购主任，采购主任填写“定货修正单”，报采购部主管签名同意，“定货修正单”交库管科，库管科从电脑中提出原“定货单”，按“定货修正单”内容进行修改，将修改后之新“定货单”打印（一式三份），和原“定货单”连在一起，最后进行上述来货相符后的工作。
- 要求：

  （1）画出该系统的数据流图。

  （2）给出该系统主要处理、数据存储或外部实体的数据字典。

（1）画出该系统的数据流图



（2）给出该系统主要处理、数据存储或外部实体的数据字典

- 处理逻辑编号：P1
  - 处理逻辑名称：审核来货
  - 简述：查询统计专利申请、受理、授权、转让情况
  - 输入的数据流：F3厂家送货单（来自E2供应商）、F4订货单（来自E2供应商）、F2订货单（来自S1订货单数据存储）
  - 输出的数据流：货品相符的F9审核信息（去向S1订货单数据存储）、F5来货不相符信息（去向E3采购主任）
  - 处理描述：根据供货商“订货单”及“厂家送货单”送货情况，库管科检查来货。如果来货相符，库管科做收货确认，如果来货不符，通知该次订货的采购主任。处理频率：不限
- 外部实体编号：E1
  - 外部实体名称：采购员
  - 简述：根据采购计划，指定采购合同，与供应商签订合同，根据合同，与向供应商订货，指定订货单
  - 输入的数据流：无
  - 输出的数据流：采购计划、采购合同

（续）

- 数据存储编号：S1
  - 数据存储名称：订货单
  - 简述：反映当前定货的情况
  - 数据存储组成：单号+订货日期+到货日期+供应商+货品序列+货品+单价+数量+计量单位+采购员
  - 关键字：单号，货品序号
  - 流入的数据流：F2订货单（P4订货处理过程）、F7修正后订货单（P3修正订货）
  - 流出的数据流：F2订货单到P3修正定货和P4验收入库

课堂练习——教学管理系统建模

一个简化的教学管理系统的描述如下：

●与学生相关的活动：

◇学生向系统提出注册申请

◇系统向学生反馈学籍通知

◇系统向部分优秀学生发出奖励通知

●与教务处相关的活动：

◇教务处将学籍审批结果输入系统

◇系统向教务处显示统计报告

◇教务处向系统提供奖励凭证

●与系教学办相关的活动：

◇系统显示学籍报告管理情形

◇系教学办将学籍处理意见和奖励建议输入系统

●与教师相关的活动：

◇系统将教学安排通知教师

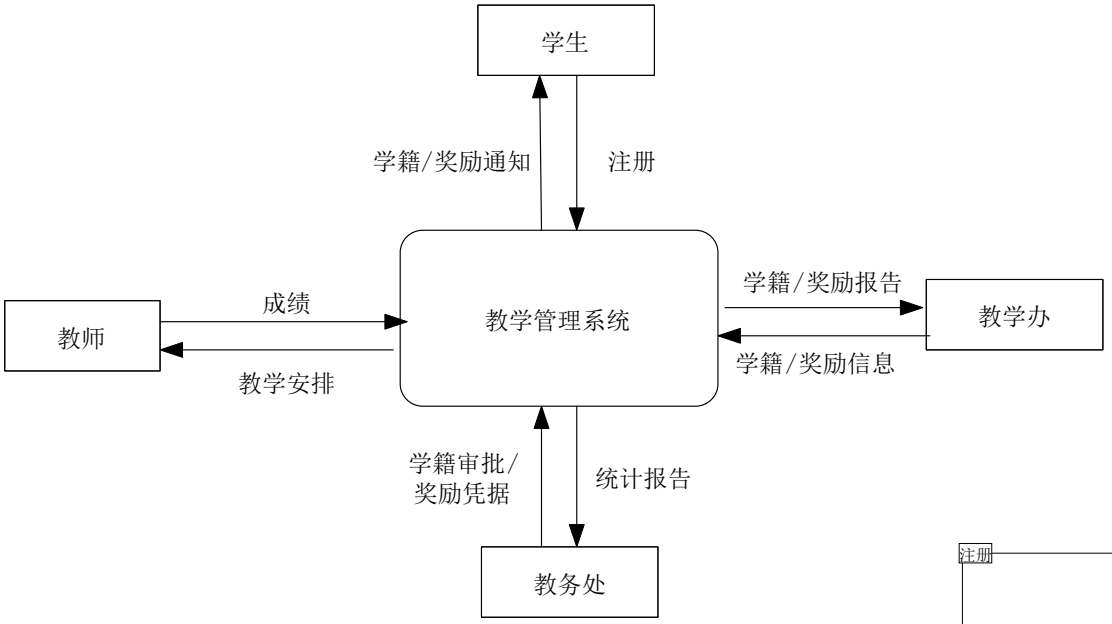
◇教师将学生成绩输入系统

1.根据以上描述，画出该系统的顶层数据流图；

2.写出相关的数据文件；

3.如果系统的主要功能是注册管理、学籍管理、成绩管理、奖励管理，请画出系统的第一层数据流图。

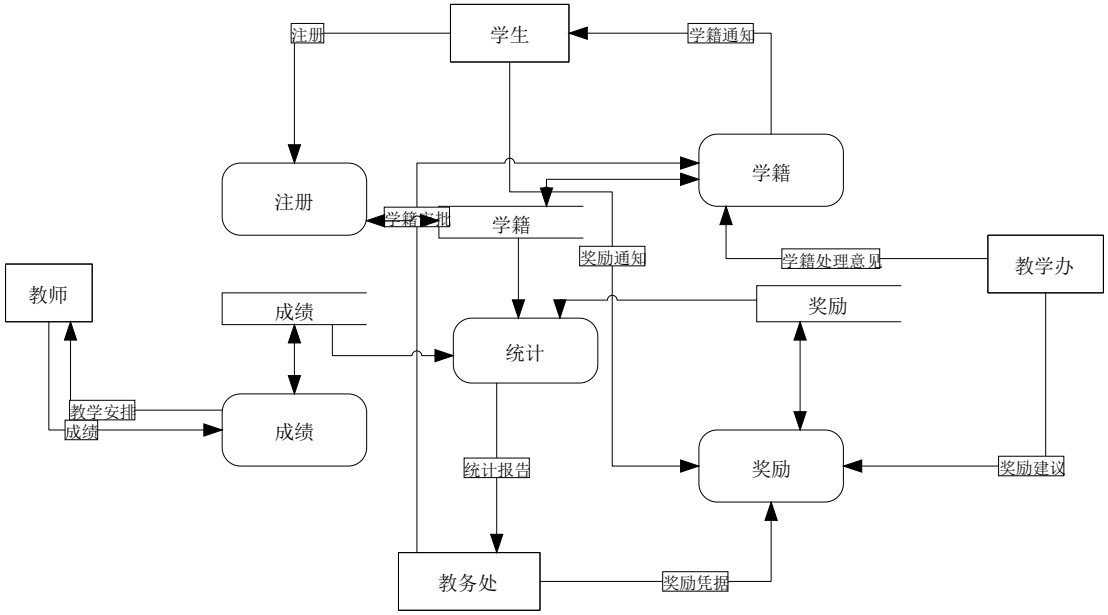
1. 顶层数据流图



2. 数据文件

学籍、奖励、成绩

3. 主要功能（注册、学籍、成绩、奖励）的第一层数据流图



软件开发

5 软件设计

主要内容

① 软件设计任务

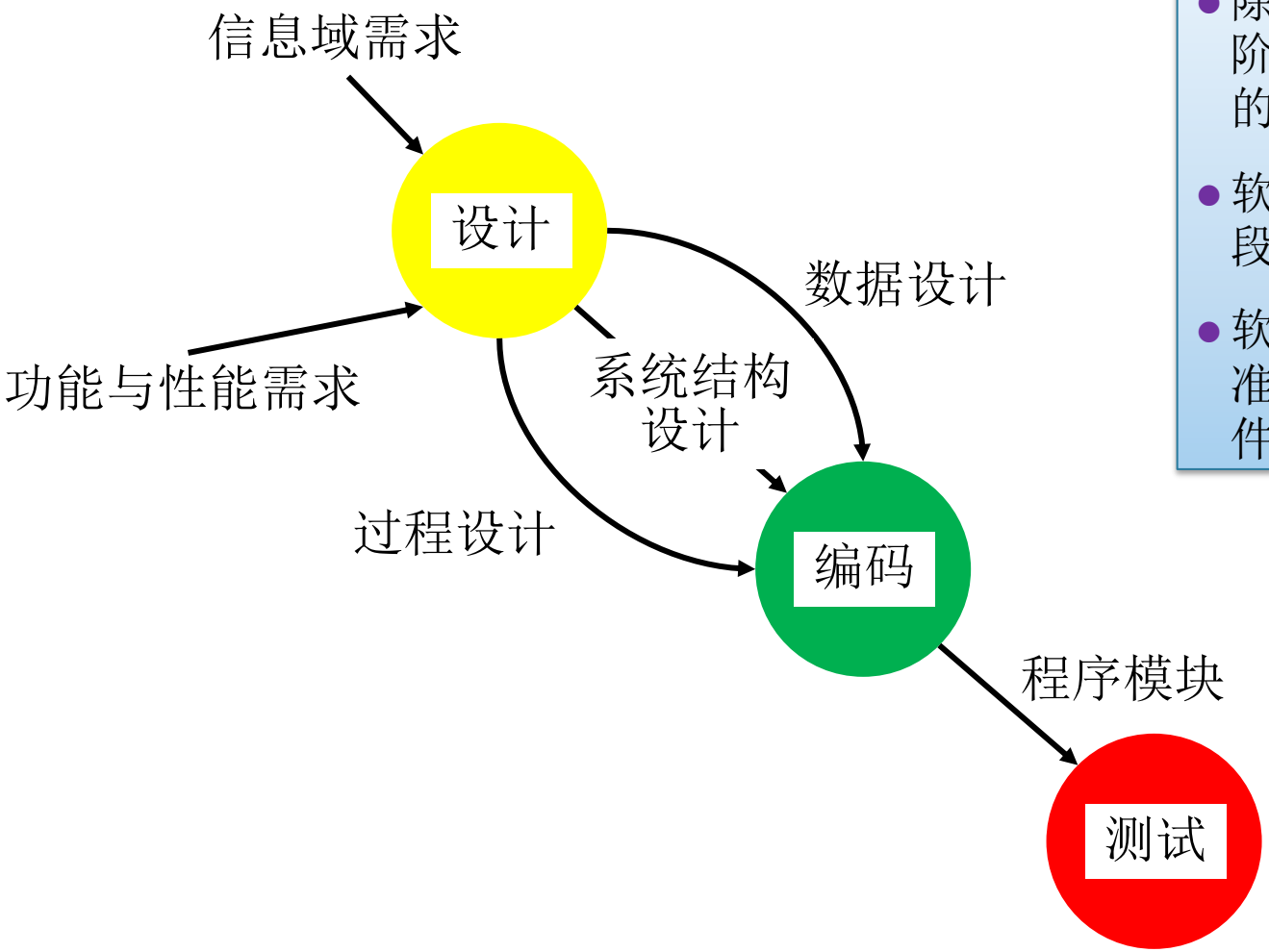
② 软件设计要素

③ 体系结构设计

④ 结构化设计方法

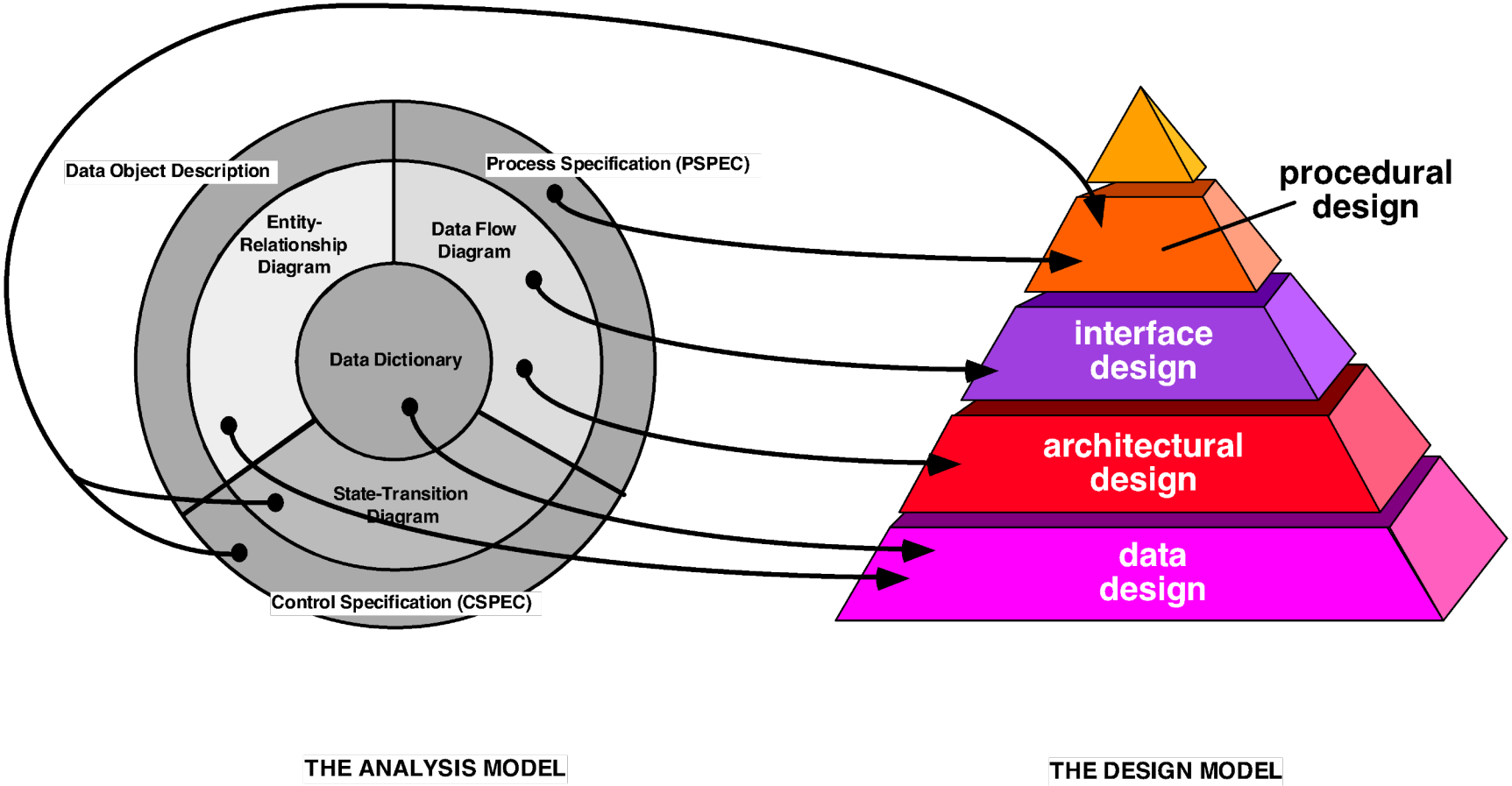
- 程序员为了更快做完优化外卖送餐时间算法的需求而不得不996，没有时间吃饭只能点外卖；
- 外卖员为了完成30分钟送达的需求，只能求程序员来写出如何继续缩短自己送餐时间的算法。

5.1 软件设计的重要性



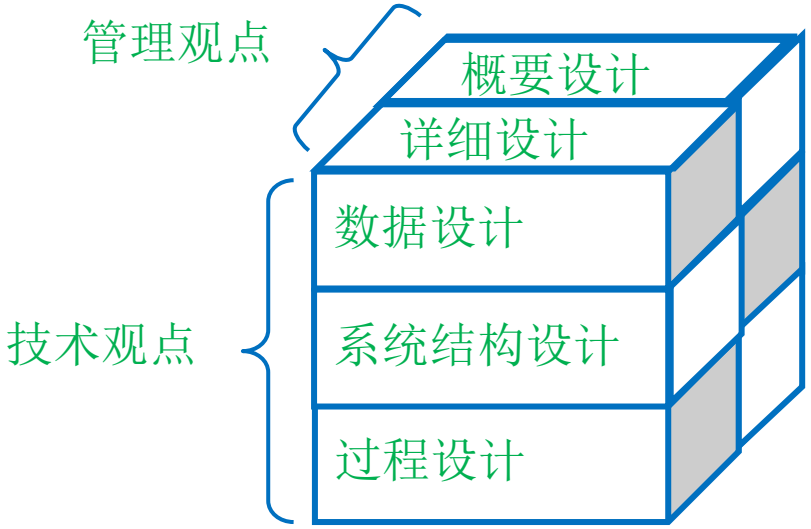
- 除维护阶段以外，开发阶段占软件工程总成本的75%以上。
- 软件设计是软件开发阶段最重要的步骤。
- 软件设计是将用户要求准确地转化为最终的软件产品的唯一途径。

Analysis to Design



设计分类：

- **数据设计**
  - 侧重于数据结构的定义
- **系统结构设计**
  - 定义软件系统各主要成份之间的关系。
- **过程设计**
  - 把结构成份转换成软件的过程性描述。



5.2 软件设计任务

- 软件设计的目标是要取得最佳解决方案。
- 最佳是指所候选方案中，就节省开发费用，降低资源消耗，缩短开发时间的条件，选择能够赢得较高生产率、较高的可靠性和可维护性的方案。

概要设计的任务：

① 制定规范
② 软件系统结构的总体设计
③ 处理方式设计
④ 数据结构设计
⑤ 数据的保护性设计
⑥ 可靠性设计
⑦ 编写概要设计文档

详细设计的任务：

① 确定软件各个组成部分内算法以及部分的内部数据组织。
② 选定某种过程表达形式来描述各种算法。
③ 进行详细设计评审。

5.3 软件设计的要素

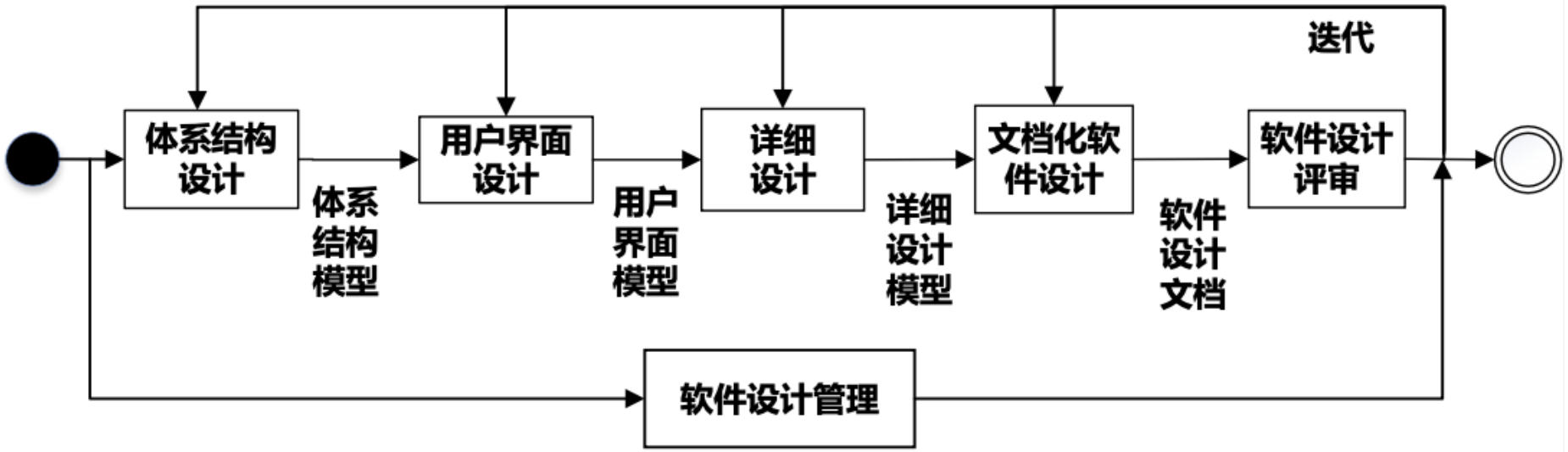
软件结构

- 体系结构
- 程序结构
- 数据结构
- 软件过程

设计思想

- 模块化
- 抽象化
- 信息隐蔽
- 自顶向下，逐步细化

软件设计的过程



- Architectural design stage
- High-level design stage
- Detailed design stage

软件设计的原则

① 抽象和逐步求精
② 模块化与高内聚、低耦合
③ 信息隐蔽
④ 多视点及关注分离
⑤ 软件重用
⑥ 迭代设计
⑦ 可追踪性
⑧ 权衡抉择

软件设计工程师应具有的知识、能力和素质：

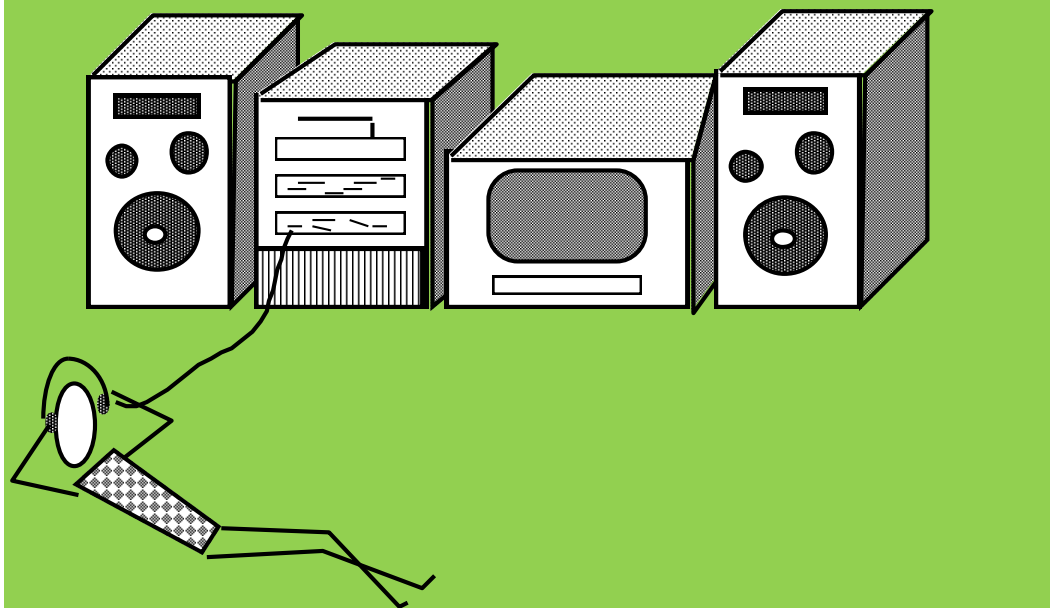
① 创新
② 抽象和建模
③ 质量保证
④ 组织、沟通和协调
⑤ 权衡抉择

Software design principles

- Abstraction 抽象
- Separation of concerns (SoC) 关注点分离（SoC）
- Modularization (or refinement or decomposition) 模块化（或细化或分解）
- Encapsulation (or information hiding) 封装（或信息隐藏）
- Separation of interface and implementation 接口与实现分离
- Coupling 耦合
- Cohesion (or localization) 内聚性（或本地化）
- Completeness (or sufficiency) 完整性（或充分性）
- Verifiability 可验证性
- *Ethically Aligned Design* 符合伦理的设计

Modular Design

easier to build, easier to change, easier to fix ...



模块是模块化设计和制造的功能单元，具有三大特征：

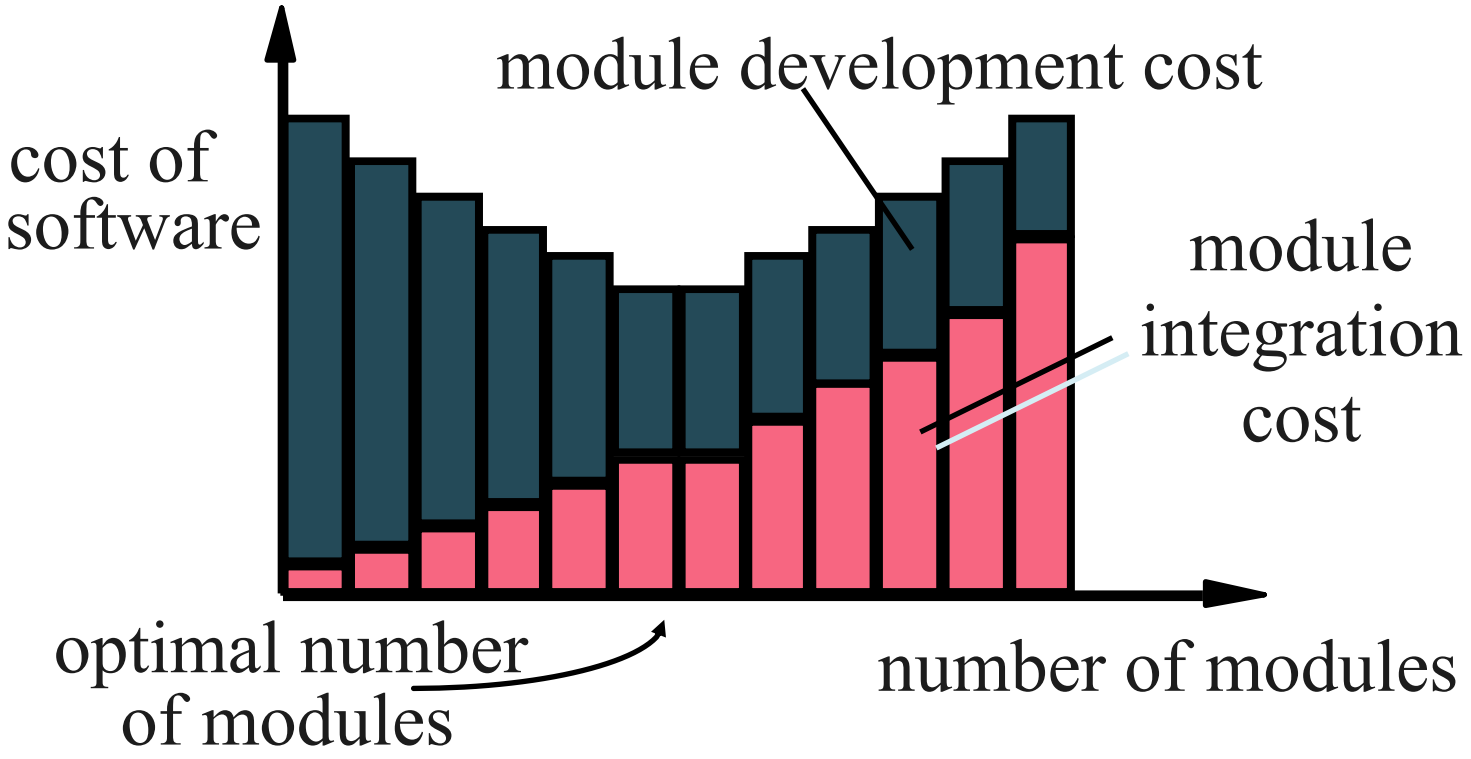
① **相对独立性**，可以对模块单独进行设计、制造、调试、修改和存储，这便于由不同的专业化企业分别进行生产；

② **互换性**，模块接口部位的结构、尺寸和参数标准化，容易实现模块间的互换，从而使模块满足更大数量的不同产品的需要；

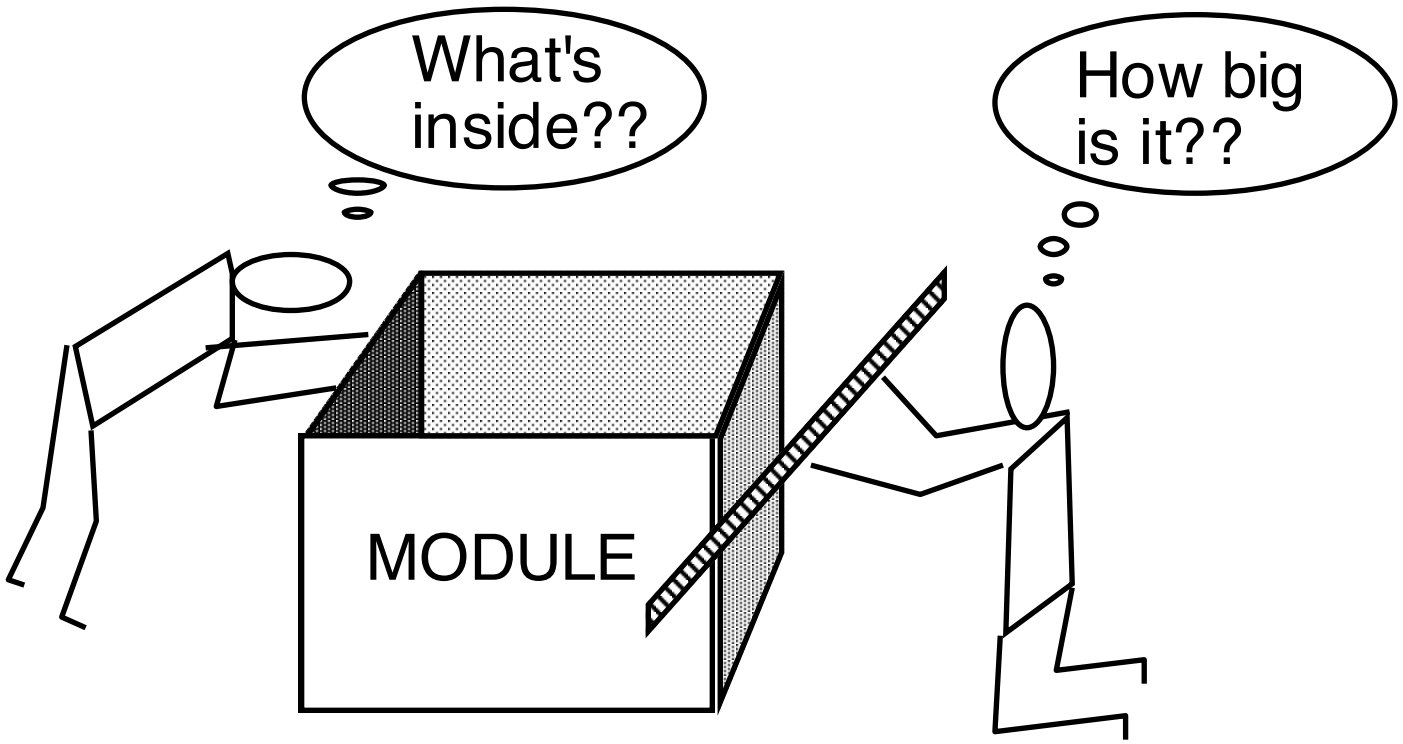
③ **通用性**，有利于实现横系列、纵系列产品间的模块的通用，实现跨系列产品间的模块的通用。

Modularity: Trade-offs

What is the "right" number of modules for a specific software design?



Sizing Modules: Two Views

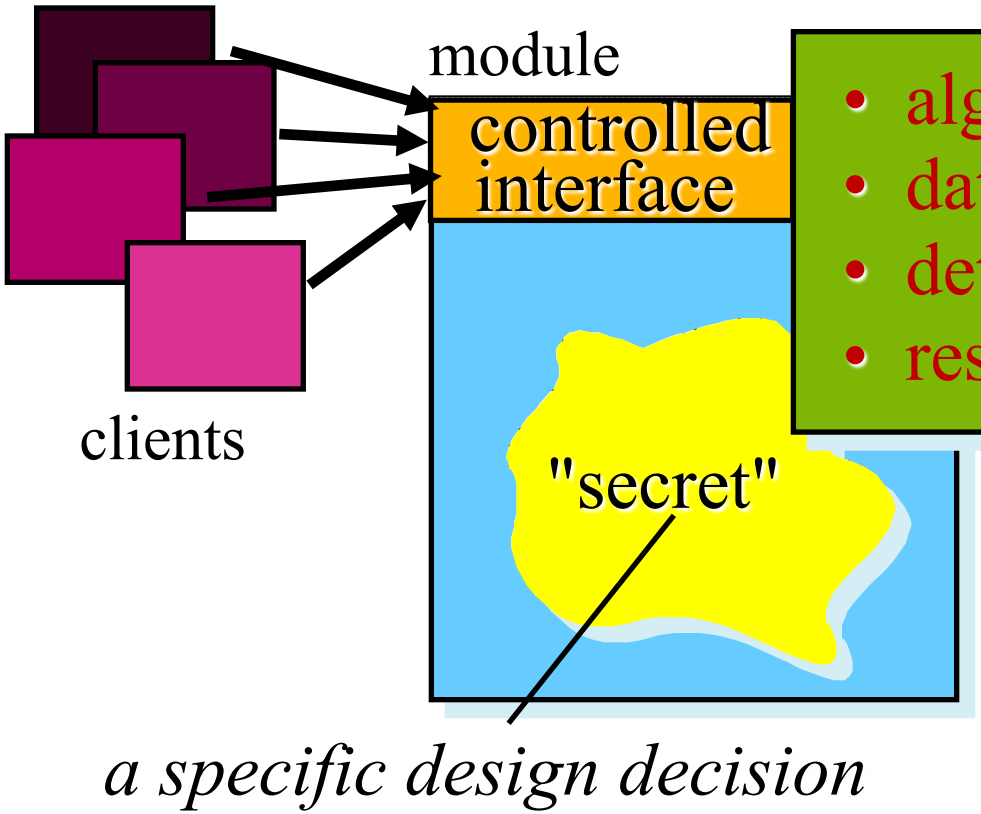


Functional Independence

COHESION - the degree to which a module performs one and only one function.

COUPLING - the degree to which a module is "connected" to other modules in the system.

Information Hiding



- algorithm
- data structure
- details of external interface
- resource allocation policy

信息隐藏是指，程序的设计思路以及内部逻辑应当包含在模块内部，对其他模块不可见。

模块独立性（Module Independence）

- 模块独立性是软件系统中每个模块只涉及软件要求的具体子功能，而与和软件系统中其他的模块接口是简单的。例如，如果一个模块只具有单一的功能，并且与其他的模块没有太多的联系，则称此模块具有模块独立性。
- 模块独立性是模块抽象和信息隐藏的直接结果。
- 度量模块独立性的2个定性的标准：
  - 耦合：指模块间相互关联的程度。
  - 内聚：指一个模块内部各个元素之间关系的紧密程度。
- 模块独立性的基本原则：
  - 为得到好的模块结构，理想的情况是块间联系尽可能少，块内联系尽可能大。
  - 块间联系和块内联系是同一事物的两个方面。

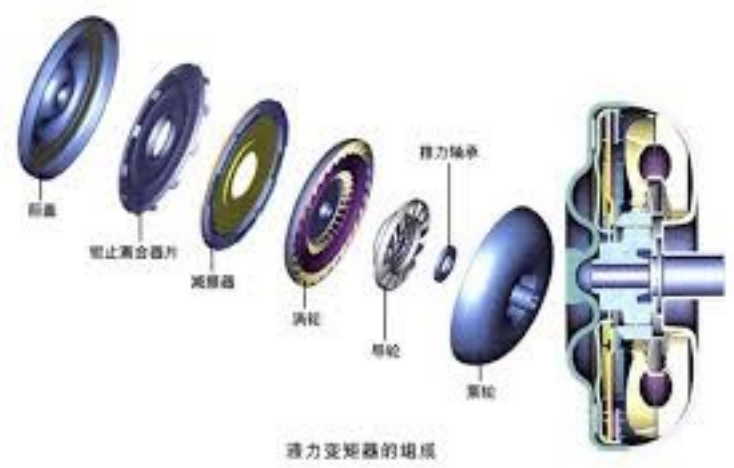
耦合性

耦合性（Coupling）是程序结构中各个**模块之间**相互关联的度量。它取决于各个模块之间接口的复杂程序、调用模块的方式以及哪些信息通过接口。

低 耦合性 高

- 非直接耦合
- 数据耦合
- 标记耦合
- 控制耦合
- 外部耦合
- 公共耦合
- 内容耦合

强 模块独立性 弱



模块之间的连接方式

内聚性

- **内聚**（Cohesion）标志一个**模块内各个元素**彼此结合的紧密程度，它是信息隐蔽和局部化概念的自然扩展。简单地说，理想内聚的模块只做一件事情。
- 设计时应该力求做到高内聚，通常中等程度的内聚也是可以采用的，而且效果和高内聚相差不多；但是，低内聚很坏，不要使用。

高 内聚性 低

- 功能内聚
- 信息内聚
- 通信内聚
- 过程内聚
- 时间内聚
- 逻辑内聚
- 巧合内聚

强 模块独立性 弱

功能单一

功能分散

模块内聚性的7个类型

5.4 体系结构设计

讨论问题：

1.一般来说，软件设计工作中最先开展什么设计，为什么？（概要设计、详细设计；数据设计、过程设计、结构设计）

2.软件体系结构设计要哪些工作？

- 大型系统总是要分解成一些子系统，这些子系统提供一些相关的服务。
- 初始设计过程的任务是识别出这些**子系统控制和通信的框架**，这个过程叫体系结构设计（Software Architecture Design），其输出是软件体系结构的描述。
- 体系结构影响性能、健壮性、分布能力和可维护性等。
- 一个应用程序的结构和风格依赖于系统的非功能性需求。
- 对于绝大多数大型软件的体系结构而言，不可能由一个模型描述清楚。系统的不同部分可能使用不同的体系结构模型来设计。

什么是架构？

ISO/IEC 42010：20072 中对架构的定义如下：

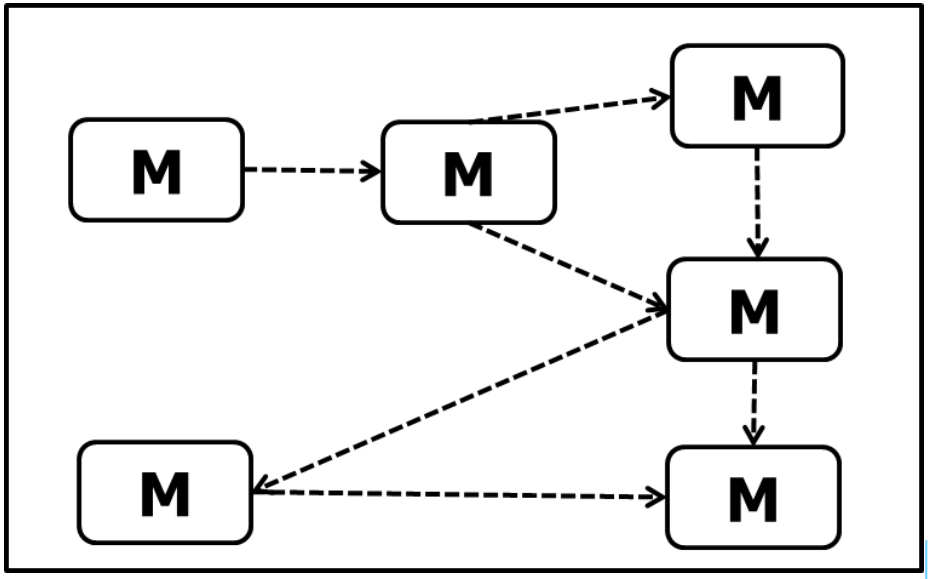
The fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution.

定义了架构的三要素：

- 职责明确的模块或者组件
- 组件间明确的关联关系
- 约束和指导原则

软件体系结构的设计元素

- 构件(Component)
  - 构成（又称组件）体系结构的基本功能部件
- 连接件(Connector)
  - 构件之间的连接和交互关系



- 约束(Constraint)
  - 构件中的元素应满足的条件以及构件经由连接件组装成更大模块时应满足的条件

体系结构设计方法

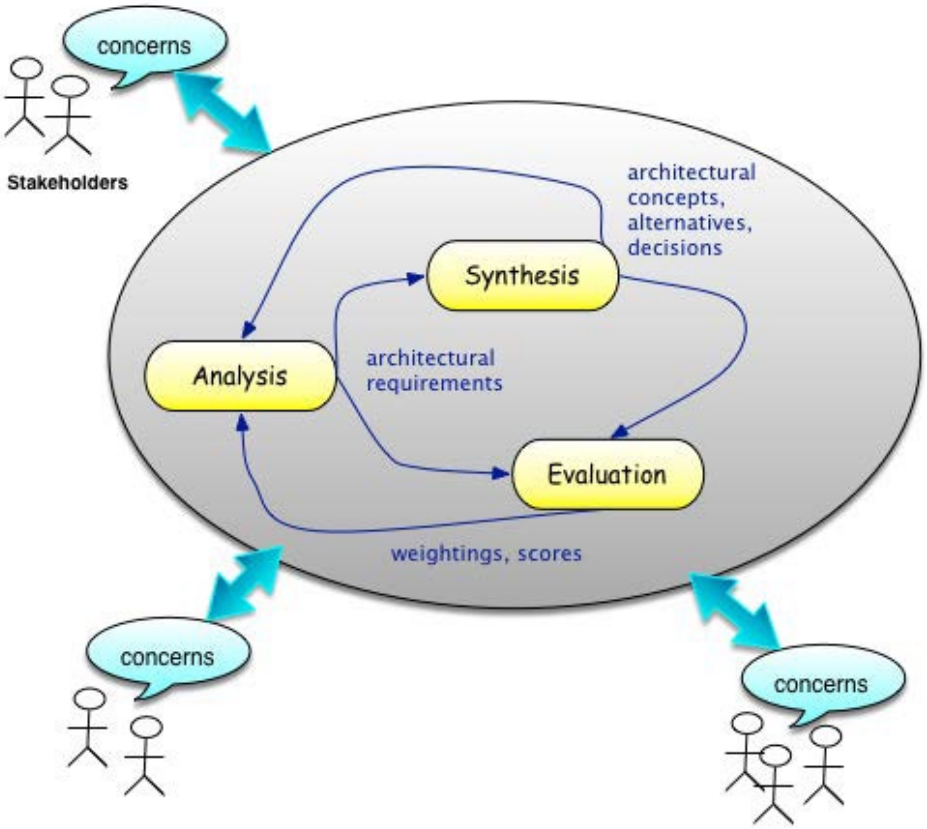
设计过程

- 系统结构化
- 控制模块
- 模块分解

设计的内容

- 静态结构模型
- 动态结构模型
- 接口模型
- 关系模型

体系结构设计模型



A general model of architectural design

Key ideas：

① Architecture is about what is fundamental to a software system; not every element, interconnection, or interface is considered fundamental.

② Architecture considers a system in its environment. Much like building architecture, software architecture is outward-looking; it considers a system's context beyond its boundaries including the people, organizations, software, hardware and other devices with which the system must interact.

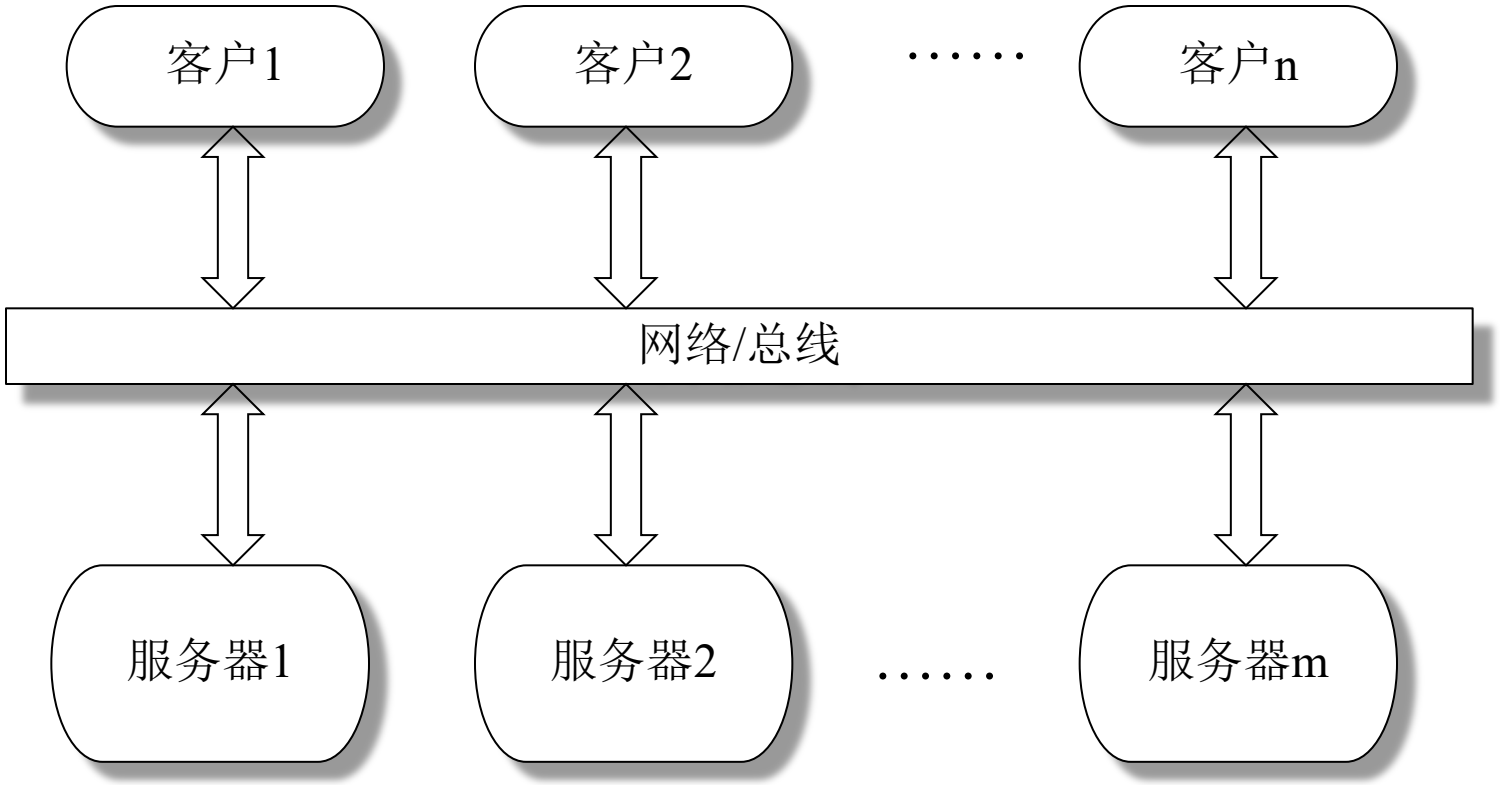
软件架构演化

① **体架构**（Monolithic）— 所有功能一个代码库，早期银行/CRM 典型
② **分层/垂直拆分**（Layered & Vertical） — 表现层/业务层/数据层，或按功能拆模块
③ **面向服务**（SOA - Service Oriented Architecture）— ESB总线集成，粗粒度服务
④ **微服务**（Microservices）— 细粒度、去中心化、容器化
⑤ **服务网格**（Service Mesh）— Sidecar 托管通信治理，Istio/Linkerd
⑥ **云原生/无服务器**（Cloud Native & Serverless）— K8s 编排、函数计算、弹性极致
⑦ **AI原生**（AI-Native）- 以AI为核心驱动业务（骨架与核心），从底层把模型、推理、Agent、语义作为要素构建系统

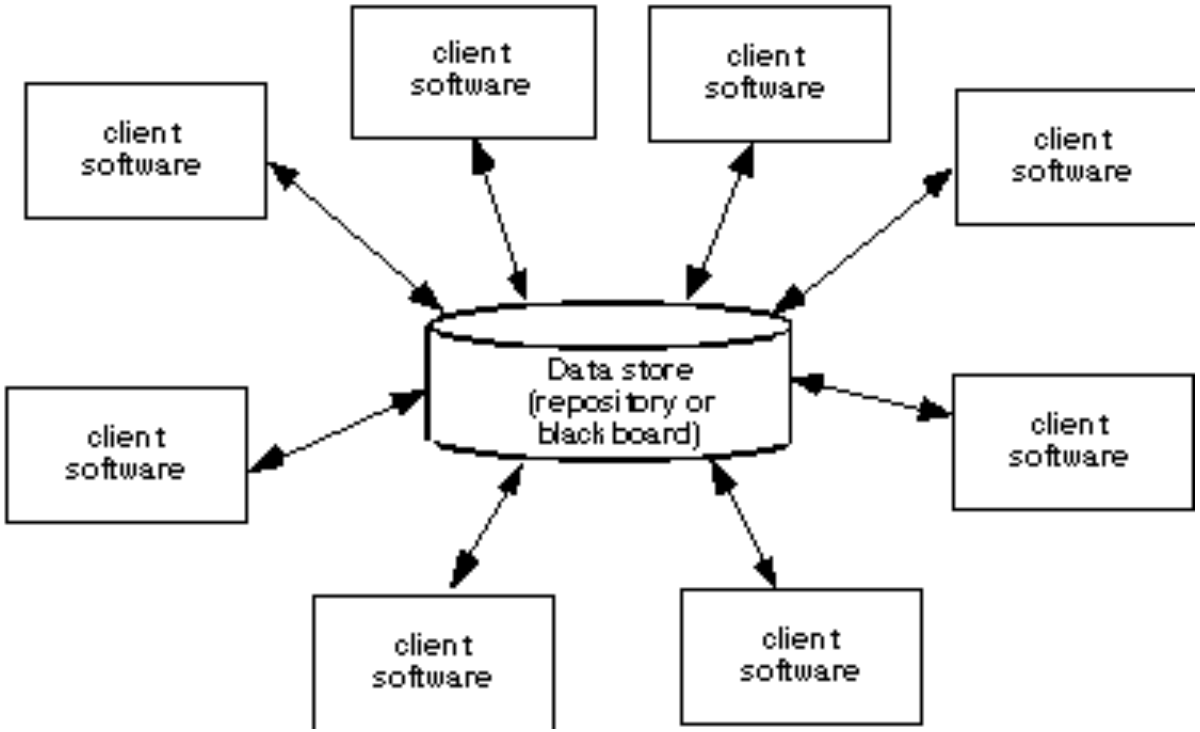
体系结构标准模型

- 客户/服务器（client/server architecture）
- 数据中心结构（data-centered architecture）
- 数据流结构（data flow architecture）
- 调用返回结构（call and return architecture）
- 事件驱动架构（Event-driven Architecture）
- 微服务架构（μ-services Architecture）
- 层次结构（layered architecture）
- 模型-视图-控制器架构（MVC）

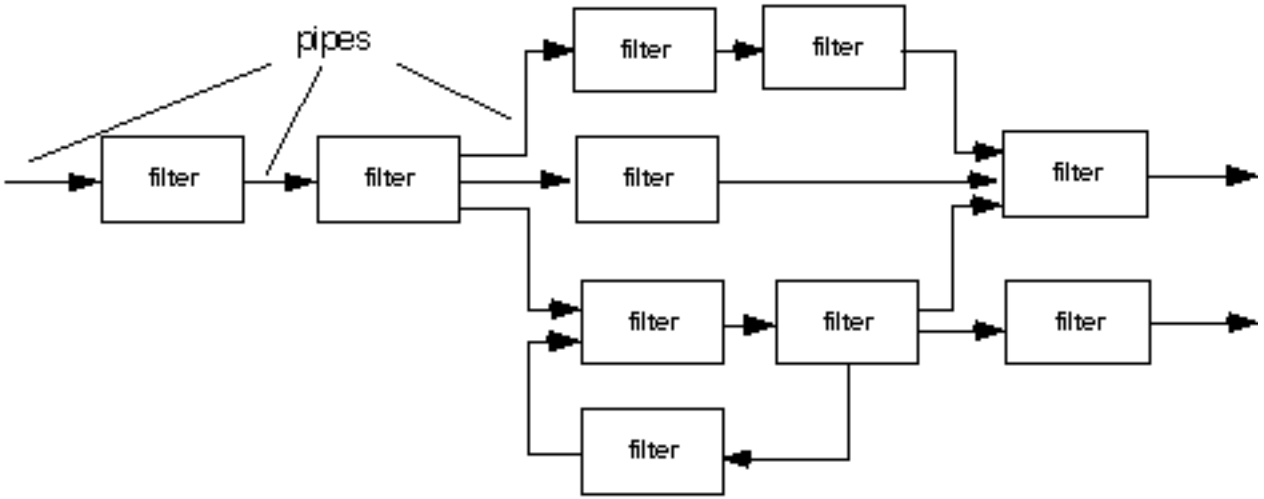
Client/Server Architecture



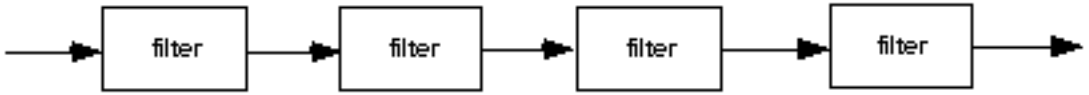
Data-Centered Architecture



Data Flow Architecture

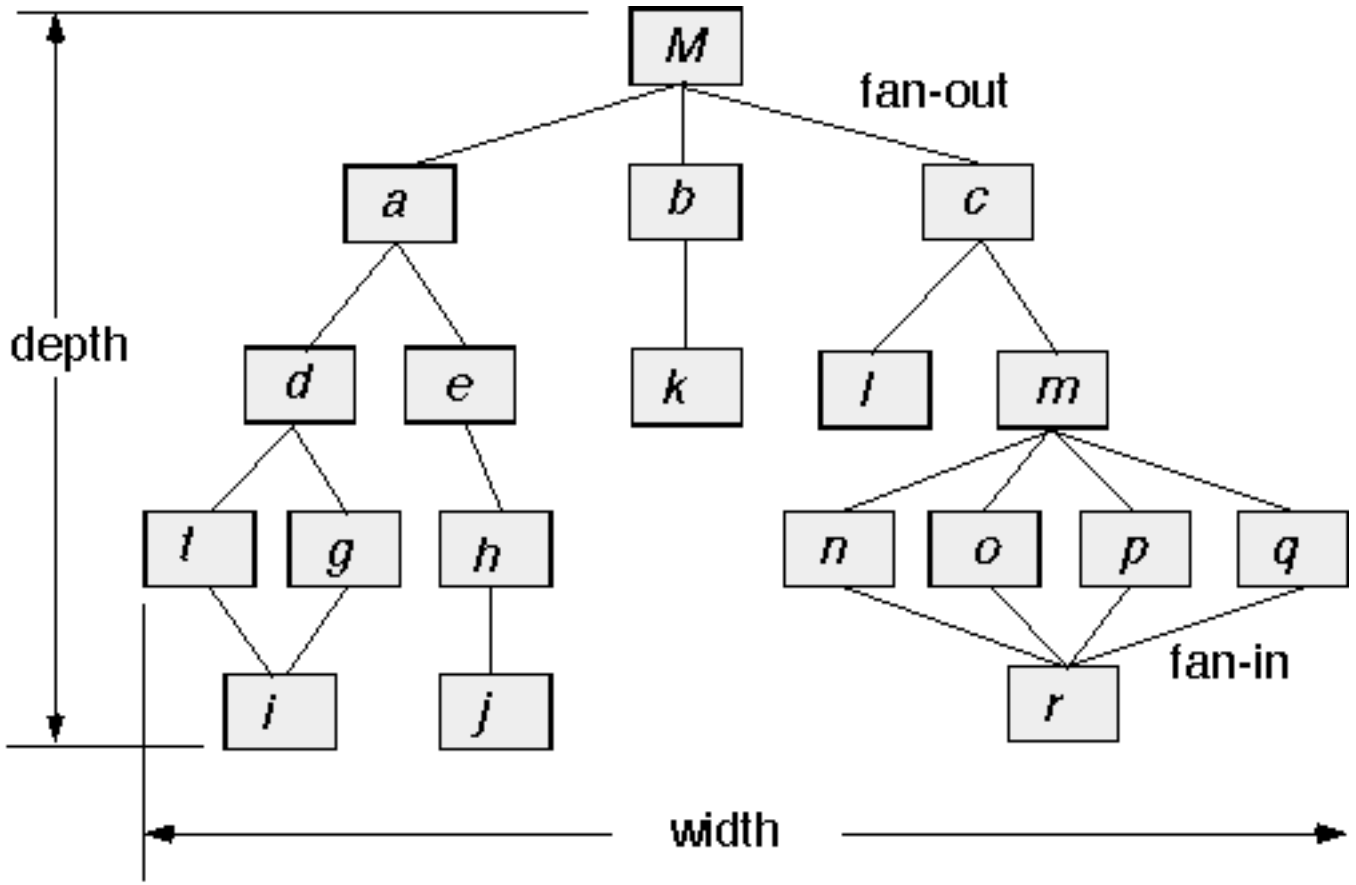


(a) pipes and filters

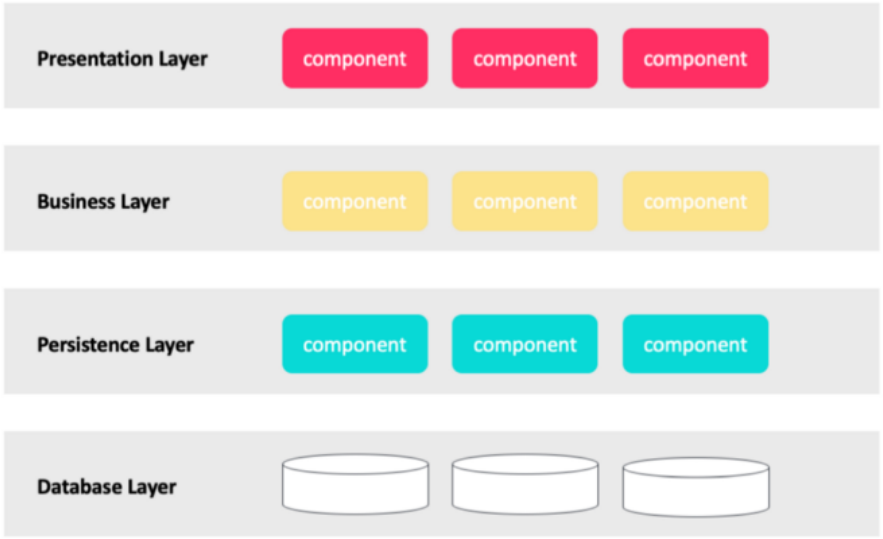


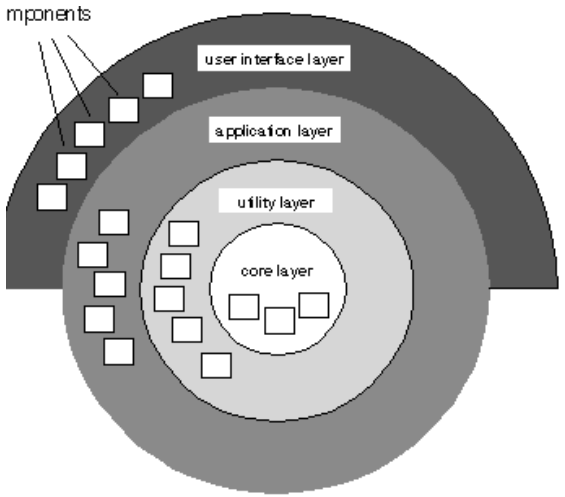
(b) batch sequential

Call and Return Architecture

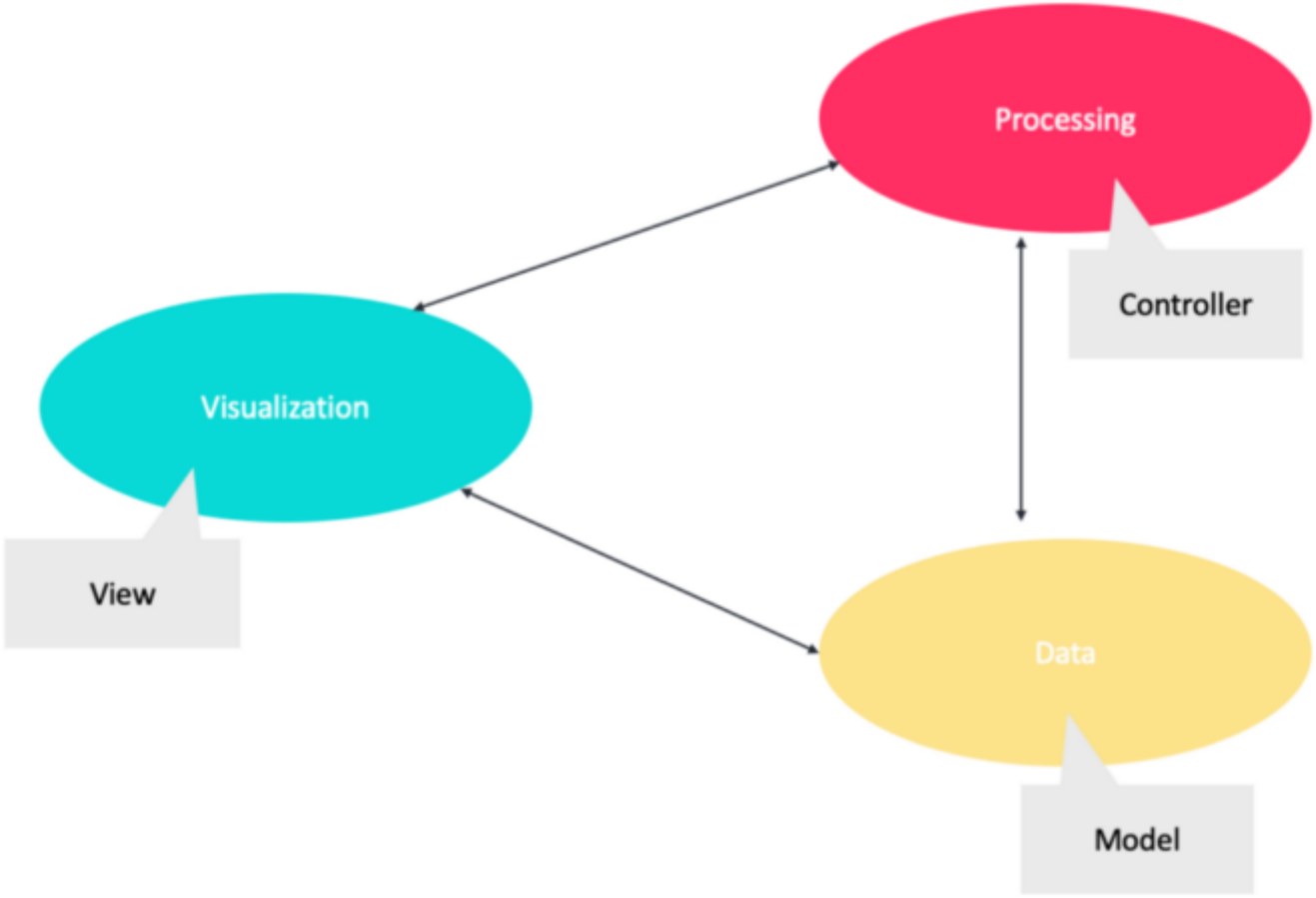


Layered Architecture

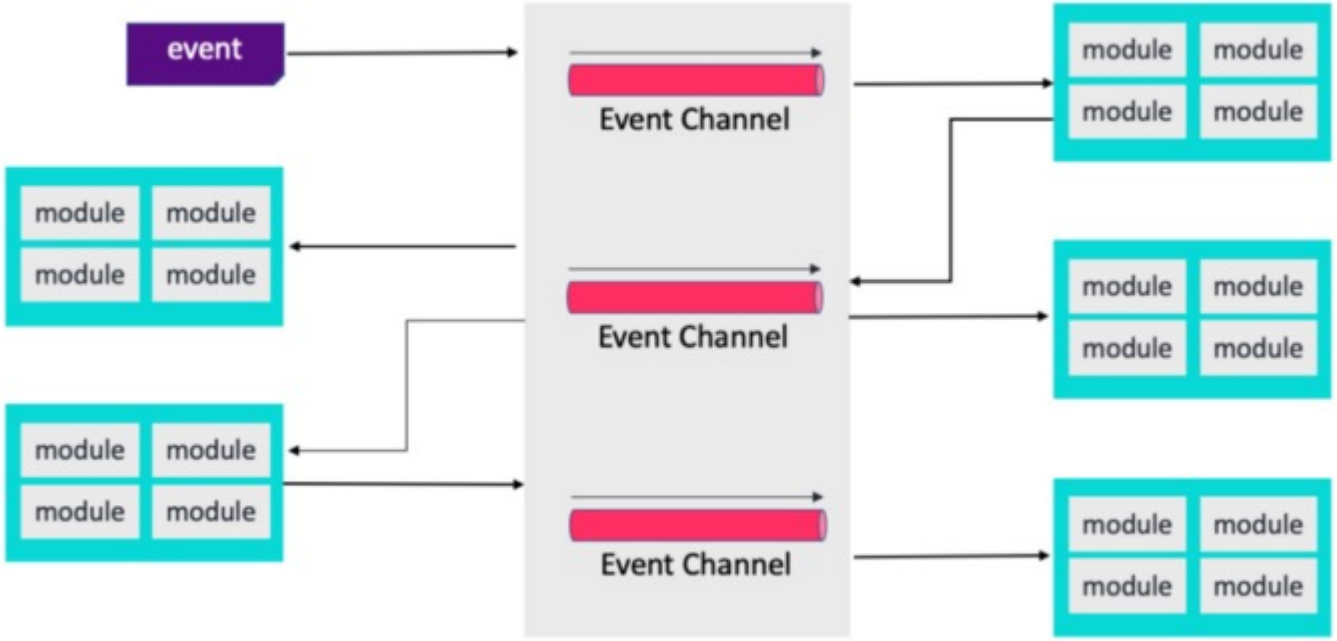




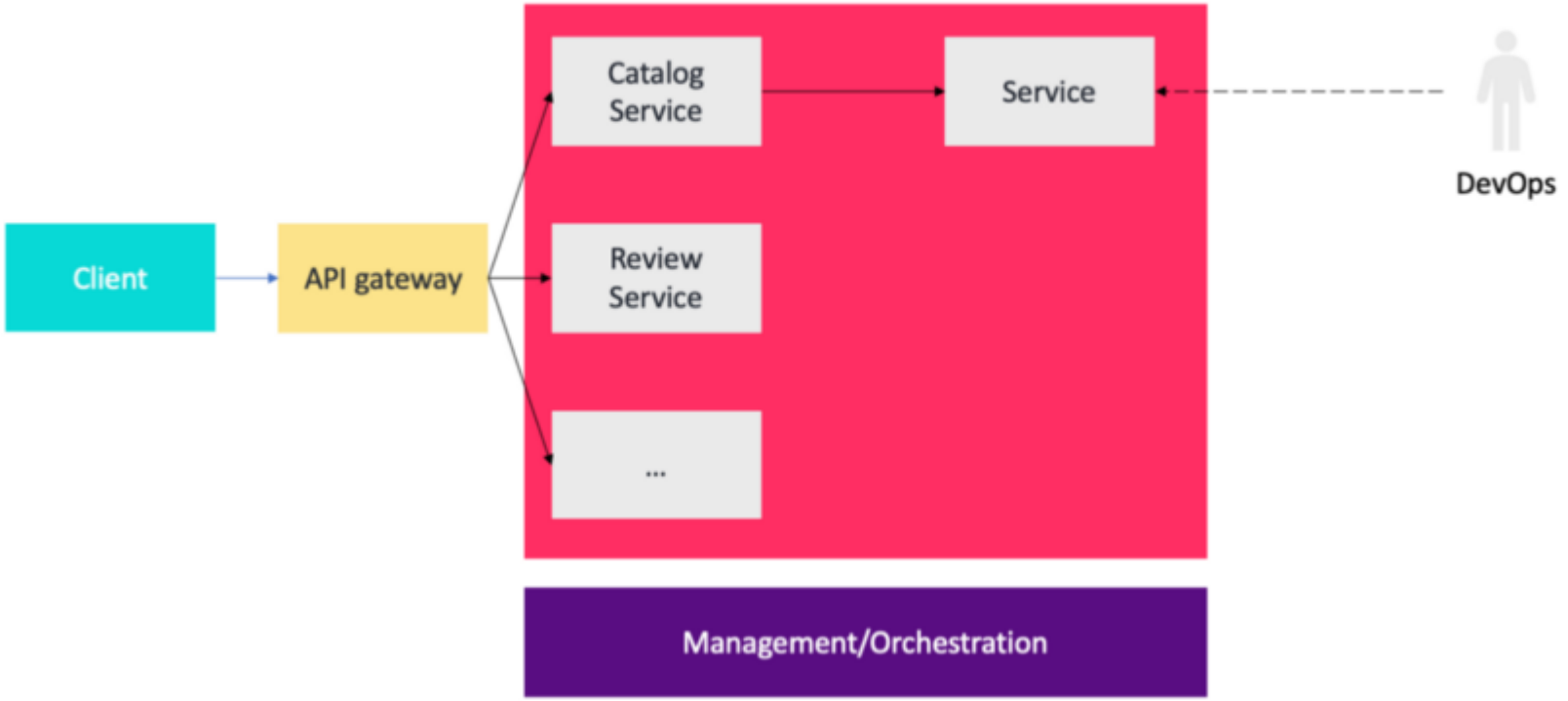
MVC（模型 - 视图 - 控制器架构）



Event-driven Architecture



μ-services Architecture

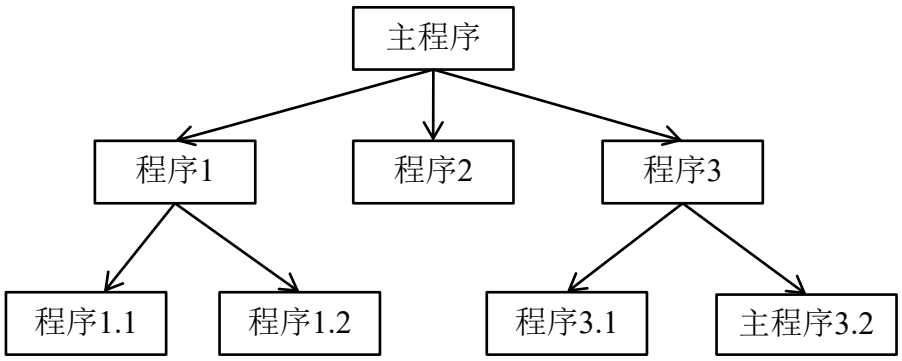


5.5 控制模型

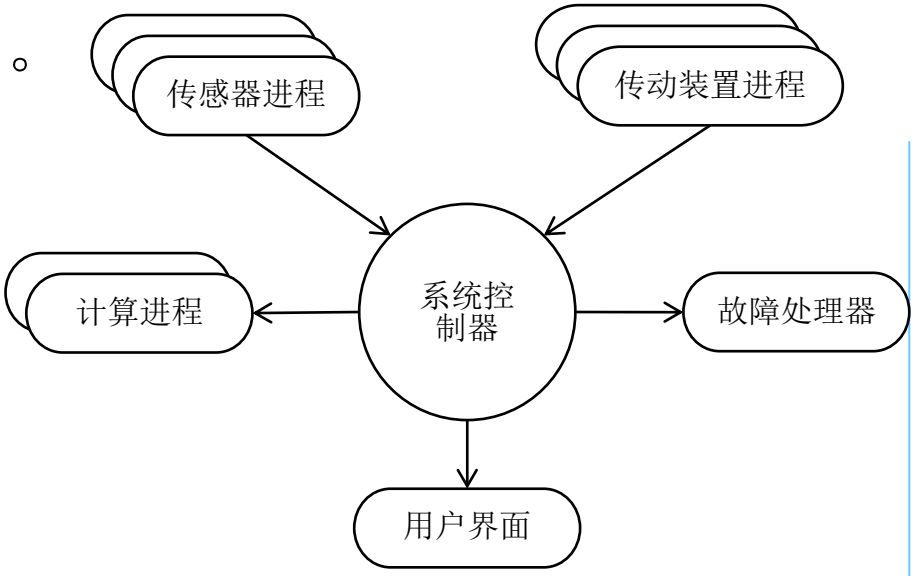
- 在体系结构层次上的控制模型关心的是子系统之间的控制流。
- 控制建模有2个一般方法：
  ① **集中式控制：** 一个子系统专门负责控制，控制其它子系统的启动和停止；它也可能将控制交给另一个子系统，但是在控制完成后，控制权仍要归还给它。
  ② **事件驱动系统：** 每个子系统都接受来自外部子系统的事件并对此做出响应。事件可能来自其它子系统或来自系统的环境。

集中式控制

- 根据子系统的顺序是否并发执行而分为2类：
  - ① 调用-返回模型：自上而下的子过程模型
  - ② 管理者模型 ：适用于并发系统的模型。
- 优点：容易分析控制流，从而找出对特别的输入系统将会做什么响应。
- 缺点：对异常处理显得笨拙。



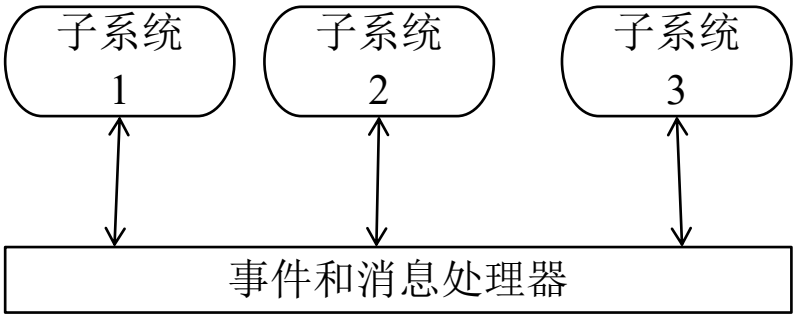
控制的调用-返回模型



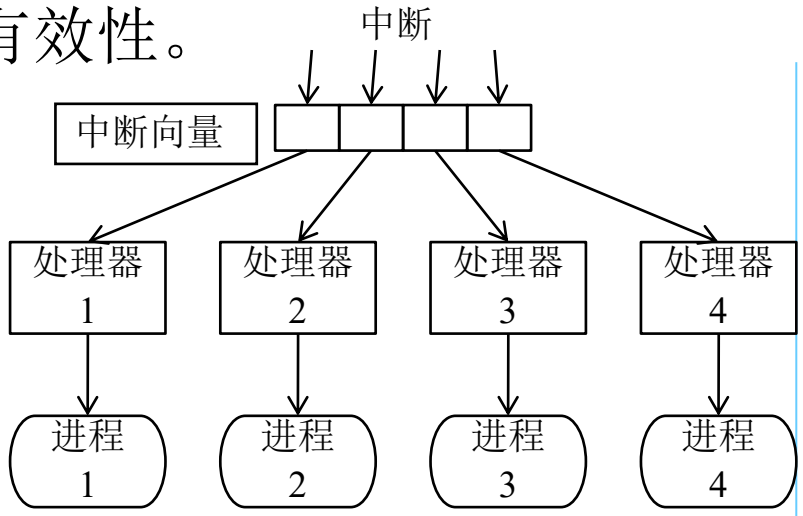
管理者模型

事件驱动系统

- 讨论2种事件驱动系统：
  - ① 广播模型：事件广播到所有子系统，任何能处理该事件的子系统都会响应。
  - ② 中断驱动模型：专用于实时系统中，由中断处理器对来自外部的中断进行检测，然后在其它组件中处理这些中断。
- 优点：能够迅速响应事件。
- 缺点：编程复杂，且不易验证有效性。



基于选择广播的控制模型



中断驱动控制模型

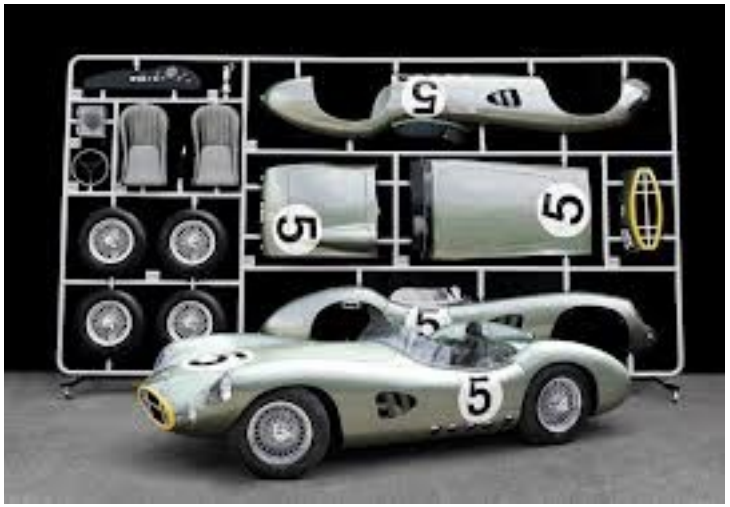
软件开发

6 结构化设计方法

6 结构化设计方法的主要内容

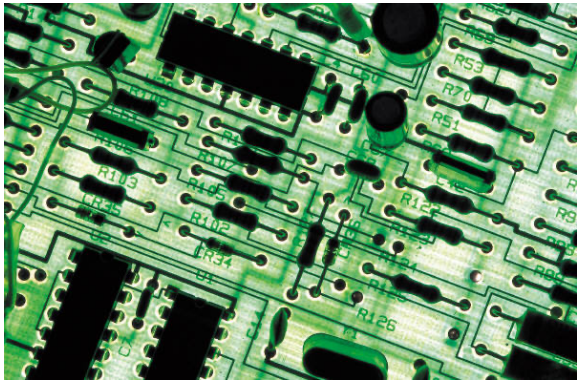
- 自顶向下，逐步细化
- 软件结构
- 程序结构
- 数据结构
- 软件过程
- 模块化
- 抽象化
- 信息隐蔽

结构化设计方法（SD, Structure Design）是基于模块化、自顶向下逐层细化、结构化程序设计等程序设计技术基础上发展起来的。



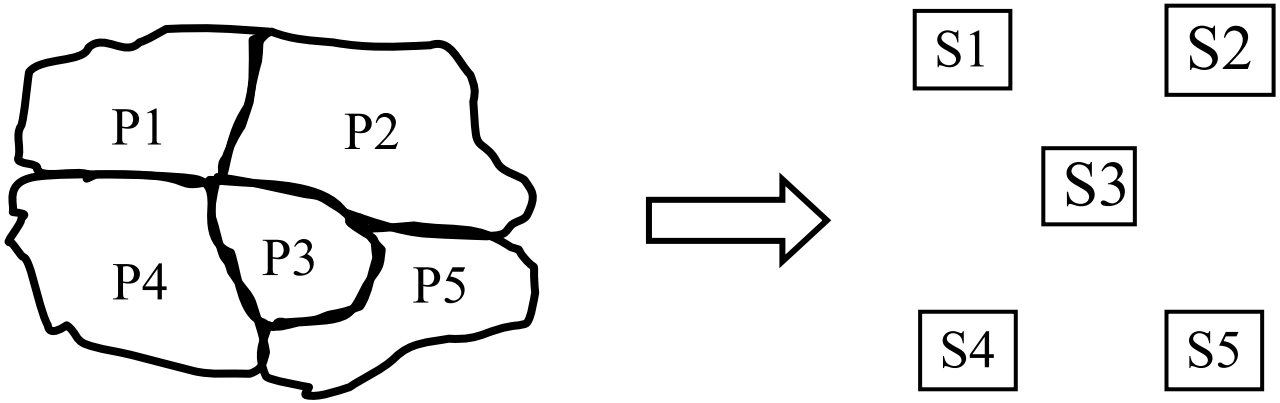
6.1 自顶向下，逐步细化

- 将软件的结构按自顶向下方式，对各个层次的过程细节和数据细节逐层细化，直到用程序设计语言的语句能够实现为止，从而最后确立整个的体系结构。
- 自顶向下是一个反复推敲的过程。
- 由抽象到具体。



6.2 软件结构

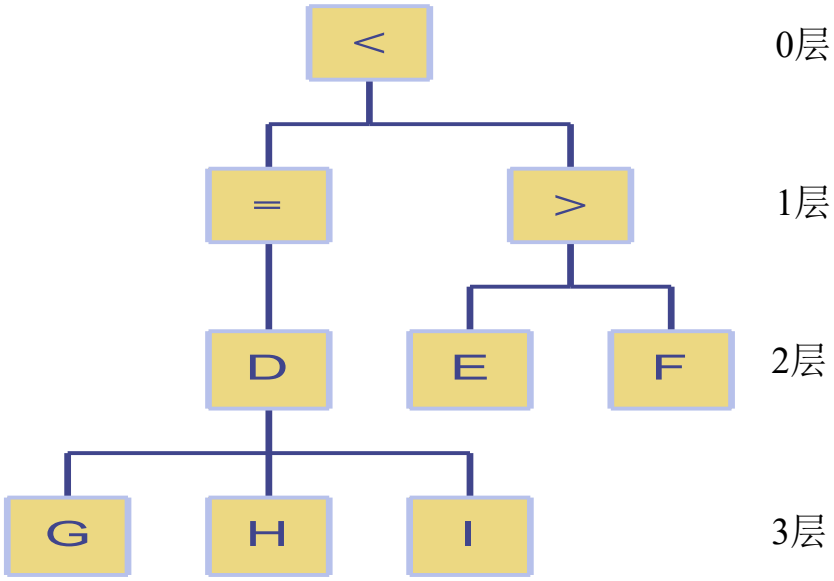
- 软件的结构包括2部分：
  - 程序模块的层次结构
  - 数据的结构
- 对于同一组软件需求，用不同的软件设计方法，可能得到不同的软件结构。



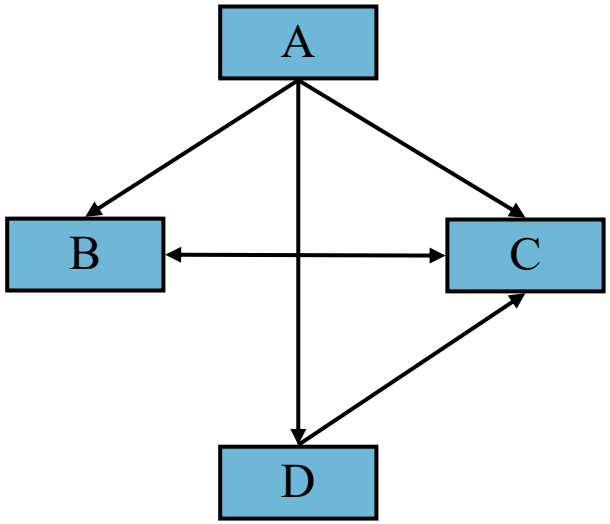
软件结构的形成

6.3 程序结构

- 程序结构表明了程序各个成份（模块）的组织情况，它通常是树状结构或网状结构。



树状结构

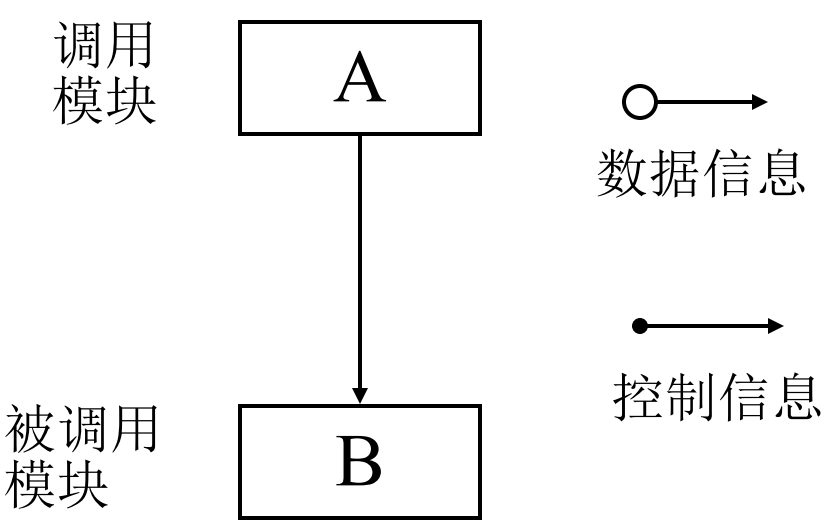


网状结构

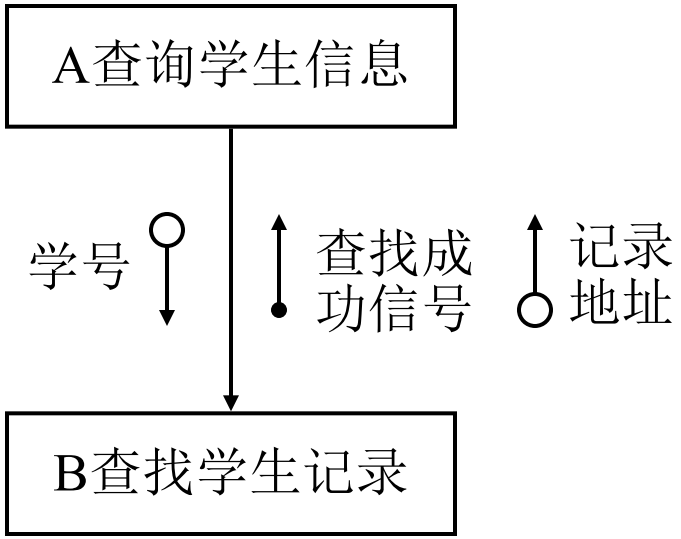
结构图

- 程序结构图：
  - 清楚地反映出程序中模块间的层次调用关系和联系
  - 定义了各个模块的名字、功能和接口
  - 反映了设计思想
- 结构图的形态特征
  - **结构图的深度：** 在多层次的结构图中，其模块结构的层次称为结构图的深度。
  - **结构图的宽度：** 结构图中同一层模块的最大模块数称为结构图的宽度。
  - **模块的扇入：** 调用一个给定模块的模块个数。
  - **模块的扇出：** 扇出表示一个模块直接调用的其它模块数目。

模块间的调用关系和接口表示

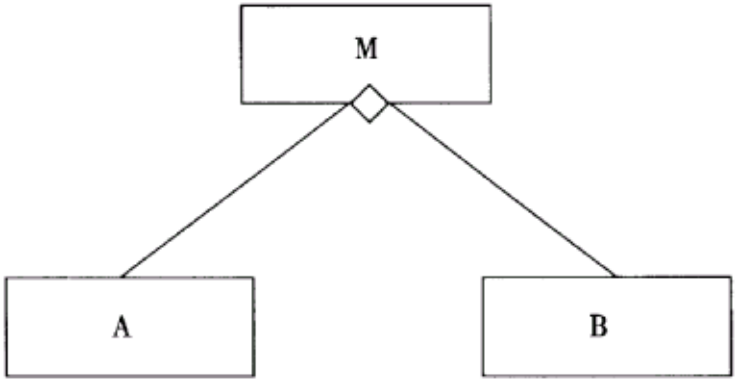


模块间调用关系

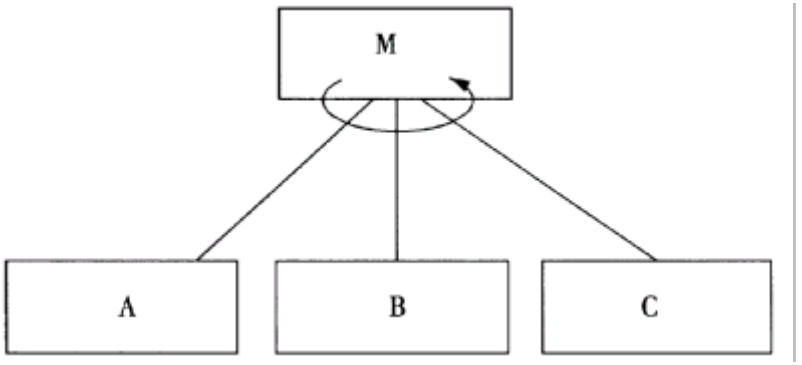


模块间接口的表示

条件调用和循环调用表示



条件调用



循环调用

6.4 数据结构

- **数据结构**是数据的各个元素之间的逻辑关系的一种表示。
- 数据结构影响过程设计，它与过程结构同等重要。
- 数据结构可以在不同的抽象层次上表示。
- 数据结构设计的内容：
  - 数据组织
  - 存取方式
  - 相关程序
  - 处理方法

6.5 软件过程

- 软件过程设计：描述模块的处理细节，提供精确的处理说明，包括事件的顺序、正确的判定点、重复的操作、数据的组织和结构等。
- **模块最重要特征：抽象和信息隐藏。**
- 通过抽象，可以确定组成软件的过程（信息）实体；通过信息隐蔽，可以定义和实施对模块的过程细节和局部数据结构的限制。

模块化设计

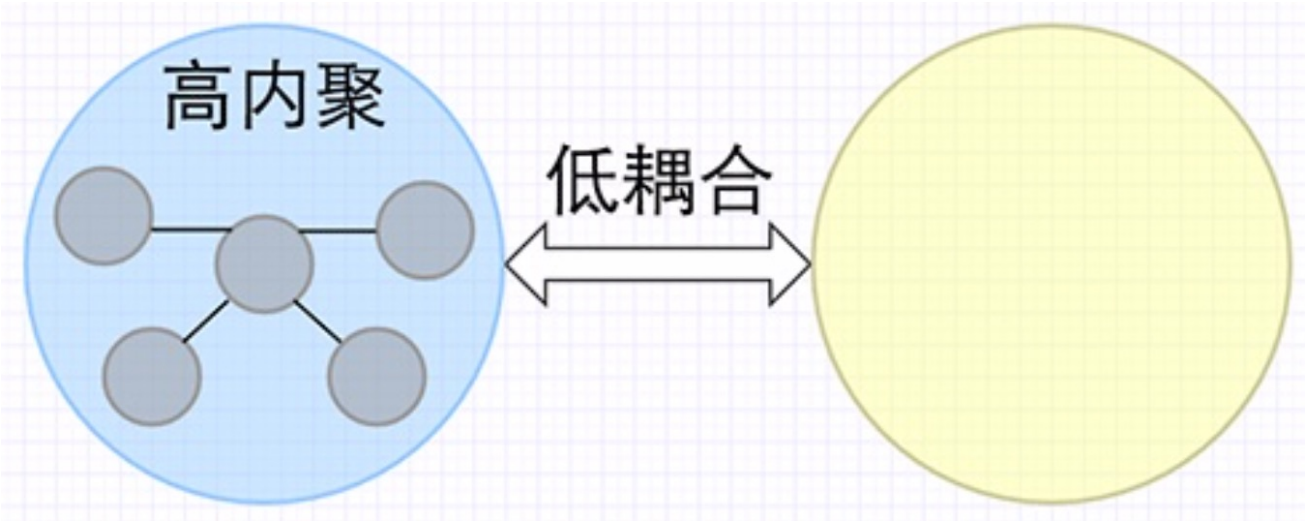
- 将子系统分解为模块时有2种模型：
  ① 面向对象
  ② 数据流模型
- 模块类似于高级语言中的：
  - procedure　　过程
  - function　　函数
  - subroutine　　子程序
  - section　　程序段
  - block　　程序块

模块

- 用一个名字就可调用的一段程序称为**模块**(Module)。
- 模块具有3个基本属性：
  - **功能**：要完成的工作
  - **逻辑**：描述模块内部的执行过程
  - **状态**：使用模块时的环境和条件。
- 模块特性：
  - 外部特性：指模块的名、参数表，以及对程序和整个系统造成的影响。
  - 内部特性：程序的功能代码和模块内部使用的函数。

设计原则

尽量使用数据耦合，少用控制耦合，限制公共环境耦合的范围，完全不用内容耦合。



6.6 抽象化

- 抽象就是将事务的相似方面集中和概括起来，暂时忽略它们之间的差异。或者说，抽象是抽象事务的本质特性而暂时不考虑它们的细节。
- 在软件设计时，可以有不同的抽象层次：
  - 在最高的抽象层次上，可使用自然语言描述问题。
  - 较低的抽象层次上，采用过程化方法描述问题解决方法。
  - 最低层的抽象层次上，使用能够直接实现的方式来描述解决问题的方法。
- 抽象是控制复杂性的基本策略之一。

抽象与细化

- 由高级抽象到低级抽象转换的过程中，要进行一系列过程抽象和数据抽象，这就是细化。
- 细化实际上是一个详细描述的过程，是设计者在原始说明的基础上进行详细说明，随着不断的细化给出更多的细节。
- 抽象与细化是紧密相关的。

过程的抽象（1/3）

- 抽象层次I：用问题所处环境的术语来描述这个软件。

- 例：

  一个CAD软件包括一个计算机绘图界面，向绘图员显示图形，以及一个数字化仪界面，用以代替绘图板和丁字尺。所有直线、折线、矩形、圆及曲线的描画、所有的几何计算、所有的剖面图和辅助图都可以用这个CAD软件实现。

过程的抽象（2/3）

- 抽象层次II：任务需求描述。

- 例：

CAD SOFTWARE TASK:
    user interaction task;
    2-D drawing creation task;
    graphics display task;
    drawing file management task;
END.

过程的抽象（3/3）

- 抽象层次III：程序过程表示。

- 例：以2-D绘图生成任务为例。

```
PROCEDURE: 2-D drawing creation
    REPEAT UNTIL (drawing creation task terminates)
        DO WHILE (digitizer interaction occurs)
            digitizer interface task;
        DETERMINE drawing request CASE;
            line: line drawing task;
        rectangle: rectangle drawing task;
        circle: circle drawing task;
        ……
    END;
    DO WHILE (keyboard interaction occurs)
            keyboard interaction task;
        PROCESS analysis/computation CASE;
            view: auxiliary view task;
            section: cross sectioning task;
            ……
        END;
        ……
    END REPETITION;
END PROCEDURE.
```

数据抽象

- 数据抽象在不同层次上描述对象的细节，可以定义与该数据对象相关的操作来规定数据对象。
- 例：

```
TYPE drawing IS STRUCTURE DEFIND
    number IS STRING LENGTH(12);
    geometry DEFIND……
    notes IS STRING LENGTH(256);
    BOM DEFIND
END drawing TYPE;
```

软件开发

7 基于数据流图的结构化设计

主要内容

① 典型系统结构图
② 变换分析设计方法
③ 事务分析
④ 软件模块结构的改进
⑤ 设计的后处理
⑥ 数据设计及文件设计

SD方法的要点：

① 研究、分析和审查数据流图。

② 根据数据流图决定问题的类型。

③ 由数据流程图导出初始的系统结构图。

④ 利用一些试探性原则来改进系统的初始结构图，直到得到符合要求的结构图为止。

⑤ 修改和补充数据词典。

⑥ 制定测试计划。

7.1 典型的系统结构图

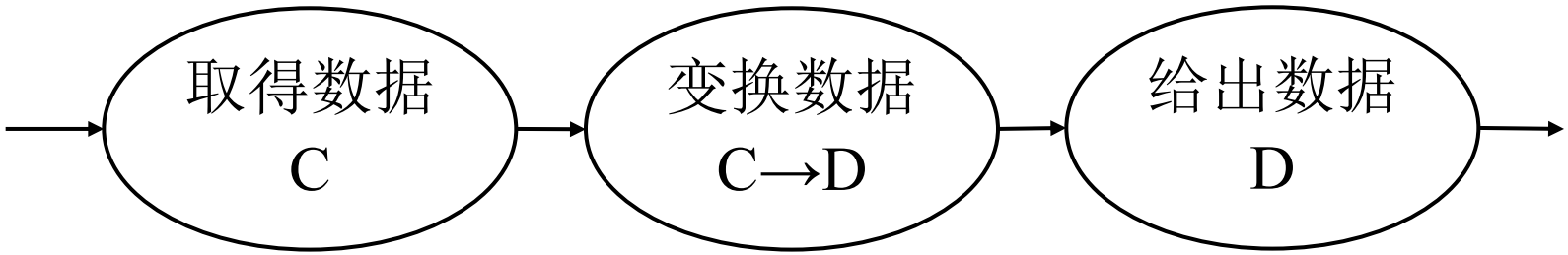
- **原子模块：**在系统结构图中，不能再分解的底层模块。
- **完全因子分解的系统：**如果一个软件系统的全部实际加工都由原子模块来完成，而其它所有非原子模块仅仅执行控制或协调功能。
- 如果系统结构图是完全因子分解的，就是最好的系统。

在系统结构图中有4种类型的模块：

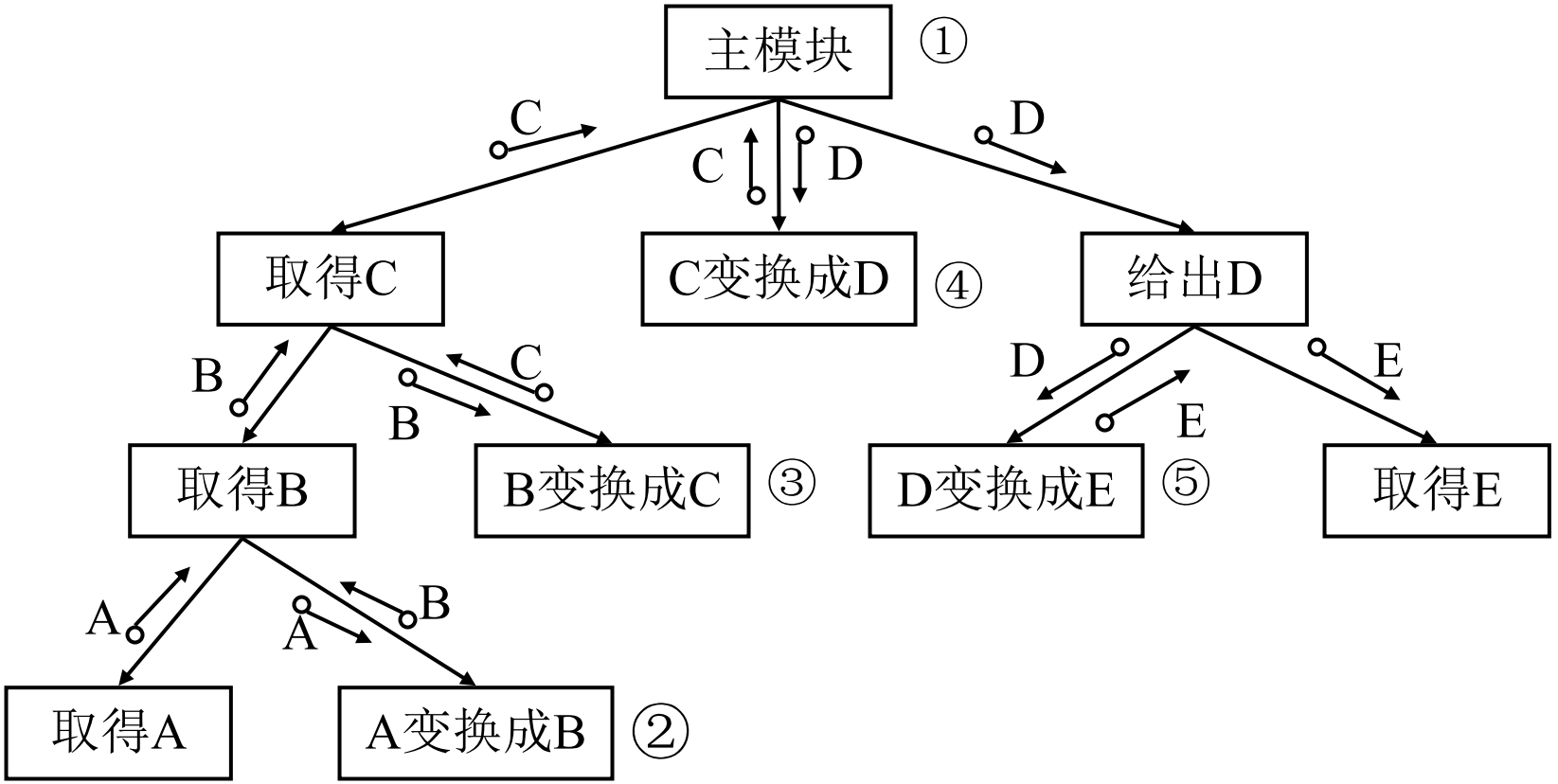
- 传入模块
  - 从下属模块取得数据，经过某些处理，再将其传送给上级模块。
- 传出模块
  - 从上级模块获得数据，进行某些处理，再将其传送给下属模块。
- 变更模块
  - 它从上级模块取得数据，进行特定的处理，转换成其它形式，再传送回上级模块。
- 协调模块
  - 对所有下属模块进行协助调和管理的模块。

变换型系统结构图

- **变换型数据流图**呈一种线性状态，信息沿输入通路进入系统，同时由外部形式变换成内部形式，进入系统的信息通过变换中心，经加工处理以后再沿输出通路变换成外部形式离开软件系统。
- 工作过程：
  1. 取得数据
  2. 变换数据
  3. 给出数据



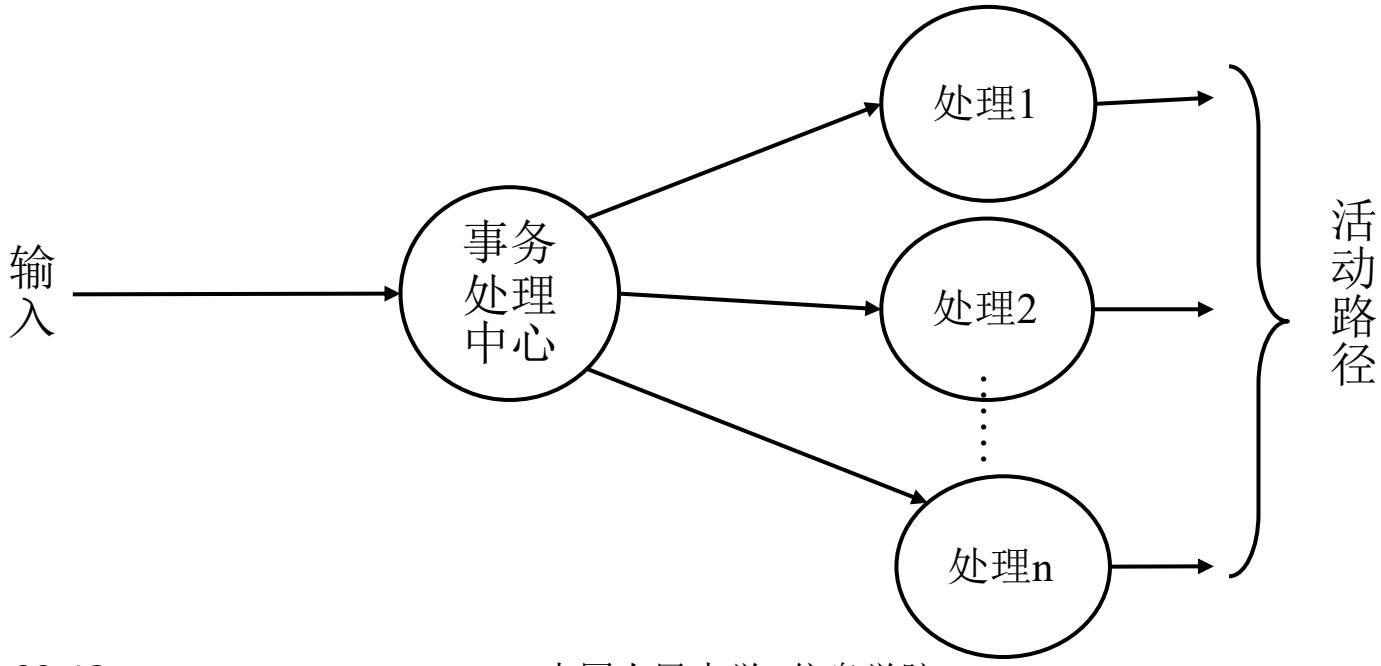
变换型的系统结构图



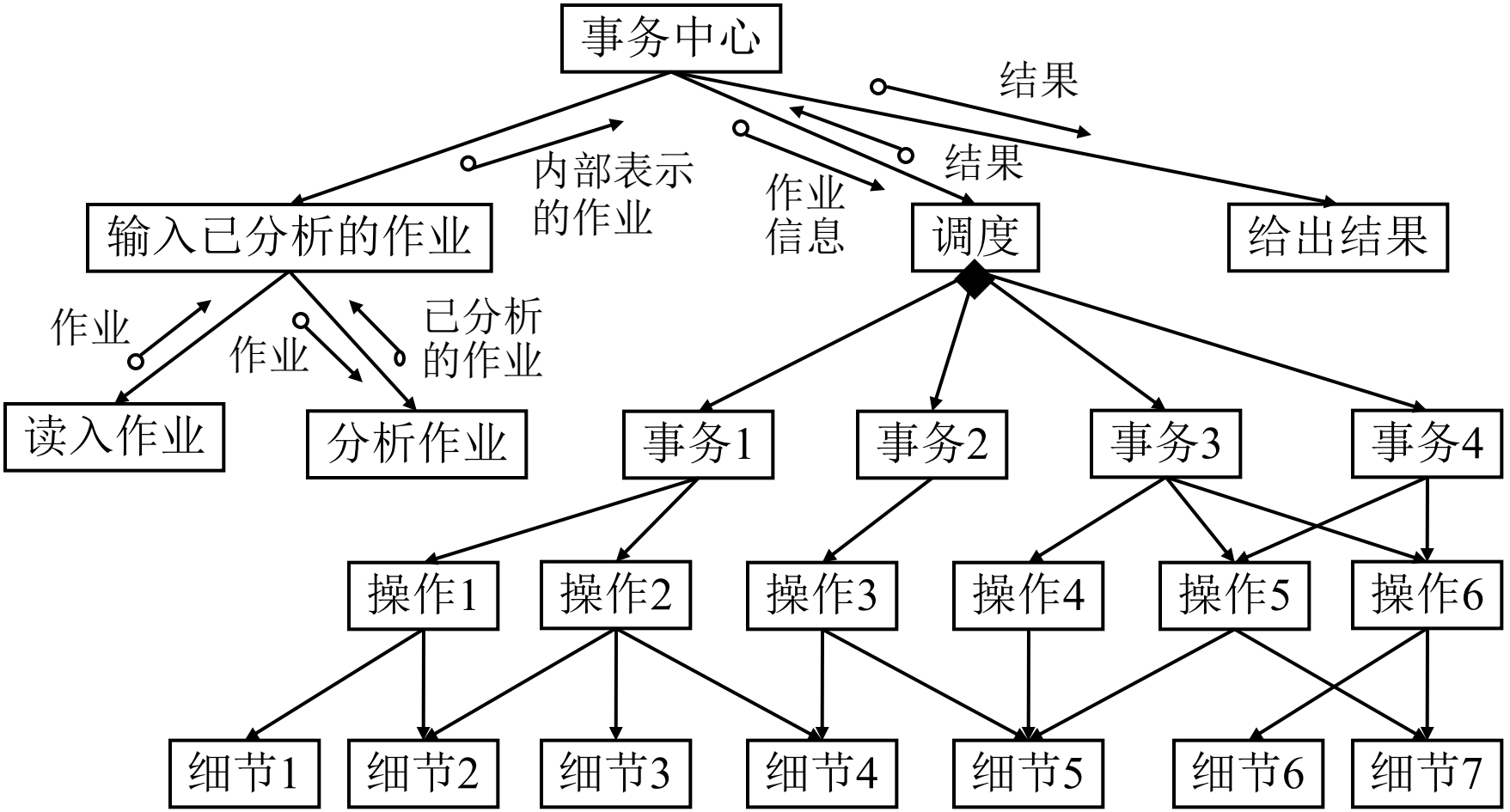
变换过程：$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

事务型系统结构图

- 某个加工将它的输入分离成一串发散的数据流，形成许多活动路径，并根据输入的值选择其中一条路径。具有这样特征的数据流图称为**事务型数据流图**。
- 通常把完成选择分派任务的部分叫做**事务处理中心**。



事务型系统结构图


事务中心
结果
内部表示
的作业
作业
信息
结果
输入已分析的作业
调度
给出结果
作业
作业
已分析
的作业
读入作业
分析作业
事务1
事务2
事务3
事务4
操作1
操作2
操作3
操作4
操作5
操作6
细节1
细节2
细节3
细节4
细节5
细节6
细节7

7.2 变换分析设计方法

- 变换分析方法步骤：
  1. 重画数据流图
  2. 区分有效（逻辑）输入、有效（逻辑）输出和中心变换部分
  3. 进行一级分解，设计上层模块
  4. 进行二级分解，设计中、下层模块

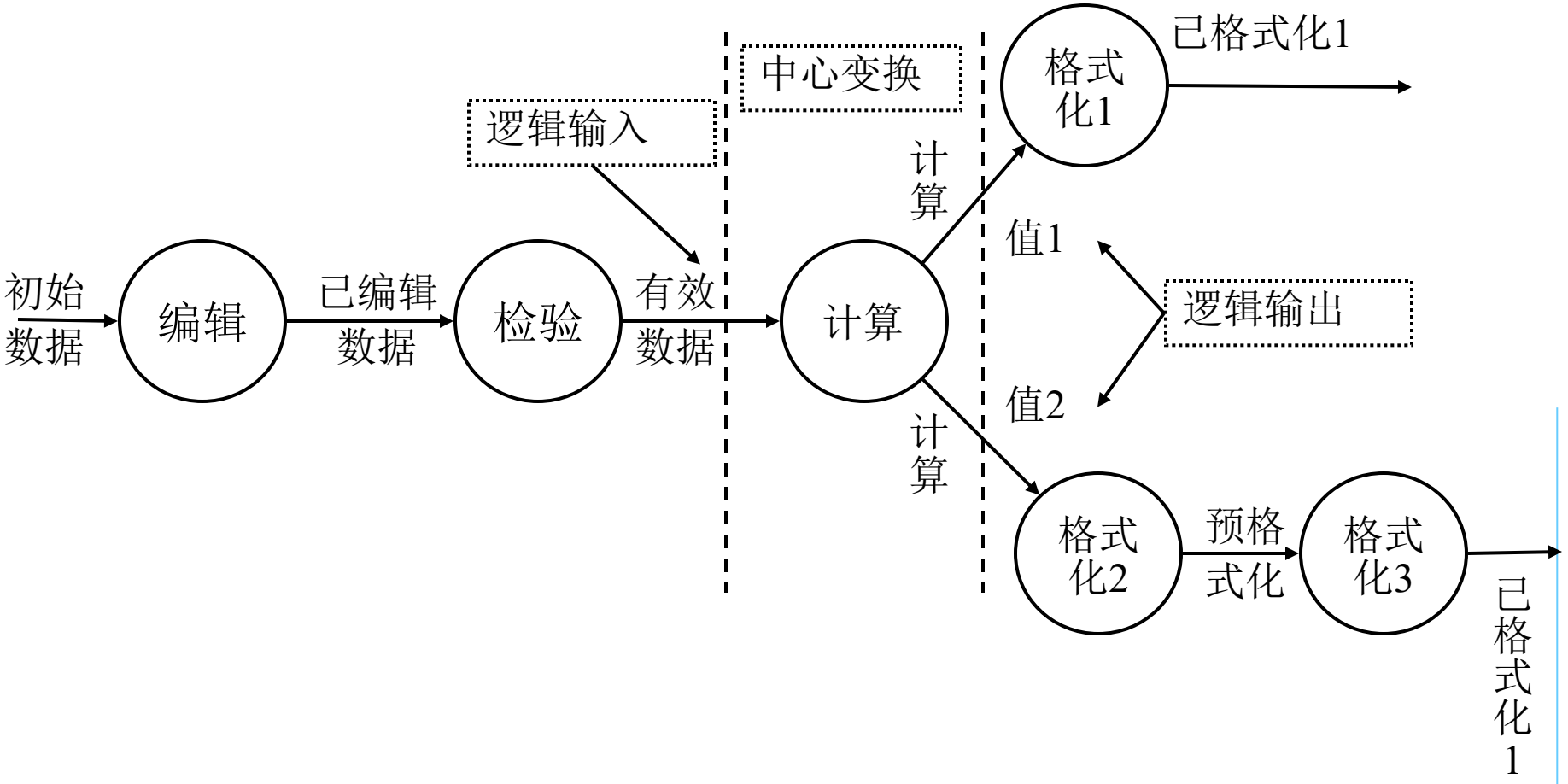
（1）重画数据流图

- 要点：
  - 以需求分析阶段得到的数据流图为基础重画数据流图时，可以从物理输入到物理输出，或者相反，还可以从项层加工框开始，逐层向下。
  - 在图上不要出现控制逻辑（如判定、循环）。用箭头表示的是数据流，不是控制流。
  - 不要去管理系统的开始和终止。
  - 当数据流进入和离开一个加工时，要仔细地标记它们，不要重名。
  - 如有必要，可以使用逻辑运算符*和⊕。

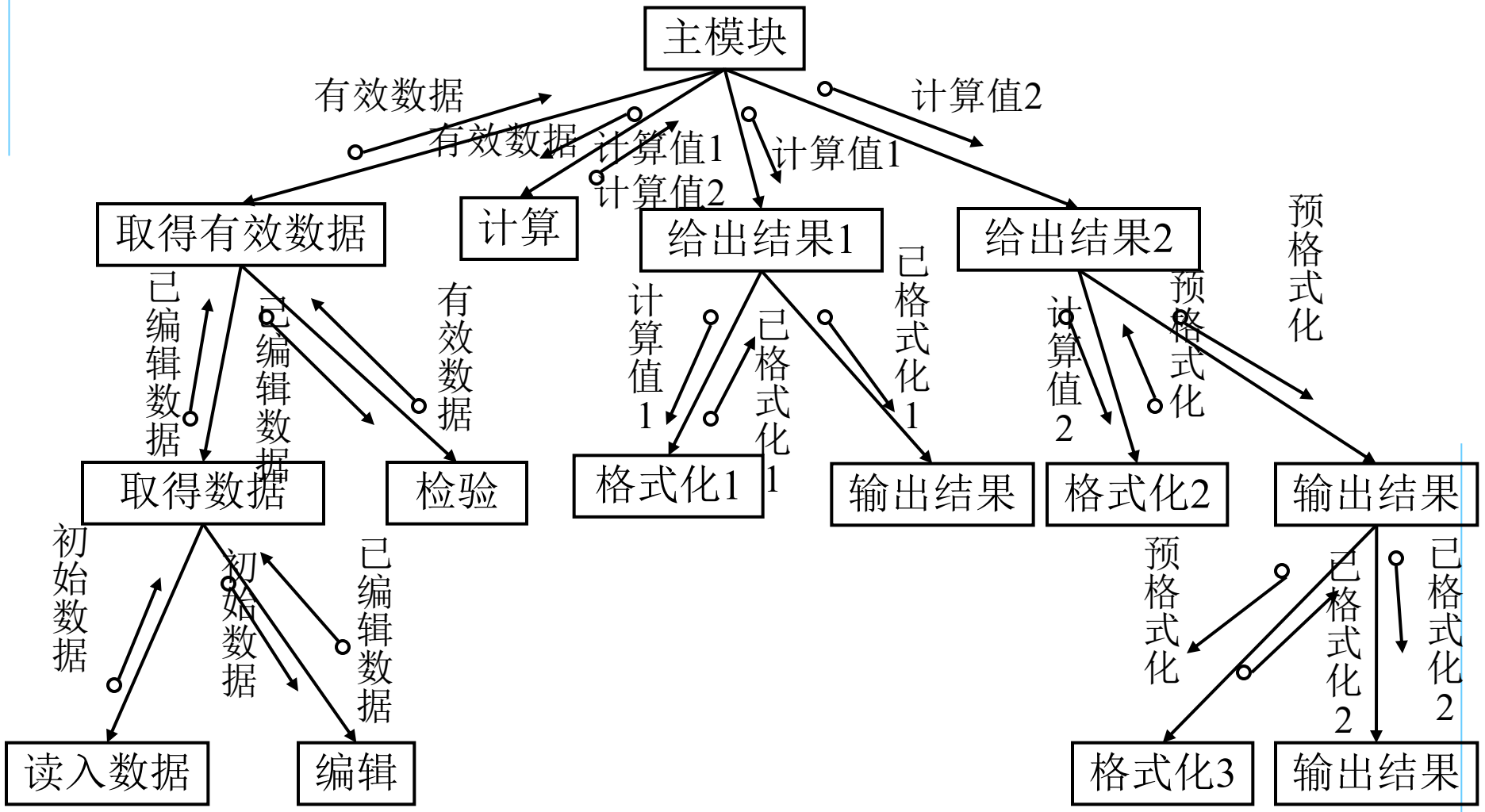
（2）在数据流图上区分系统逻辑输入、逻辑输出和中心变换部分

- 逻辑输入就是离物理输入端最远的，但仍被看作是系统输入的数据流。
- 逻辑输出就是离物理输出端最远的，但仍被看用是系统输出的数据流。
- 从物理输入端到逻辑输入，构成系统的输入部分；从物理输出端到逻辑输出，构成输出部分；夹在输入部分和输出部分之间的就是中心变换部分。
- 有些系统仅仅有输入和输出部分没有中心变换部分。

变换型问题的数据流图


初始
数据
编辑
已编辑
数据
检验
有效
数据
逻辑输入
中心变换
计算
计算
计算
值1
值2
逻辑输出
格式
化1
已格式化1
格式
化2
预格
式化
格式
化3
已
格
式
化
1

变换型问题的数据流图导出结构图



（续）

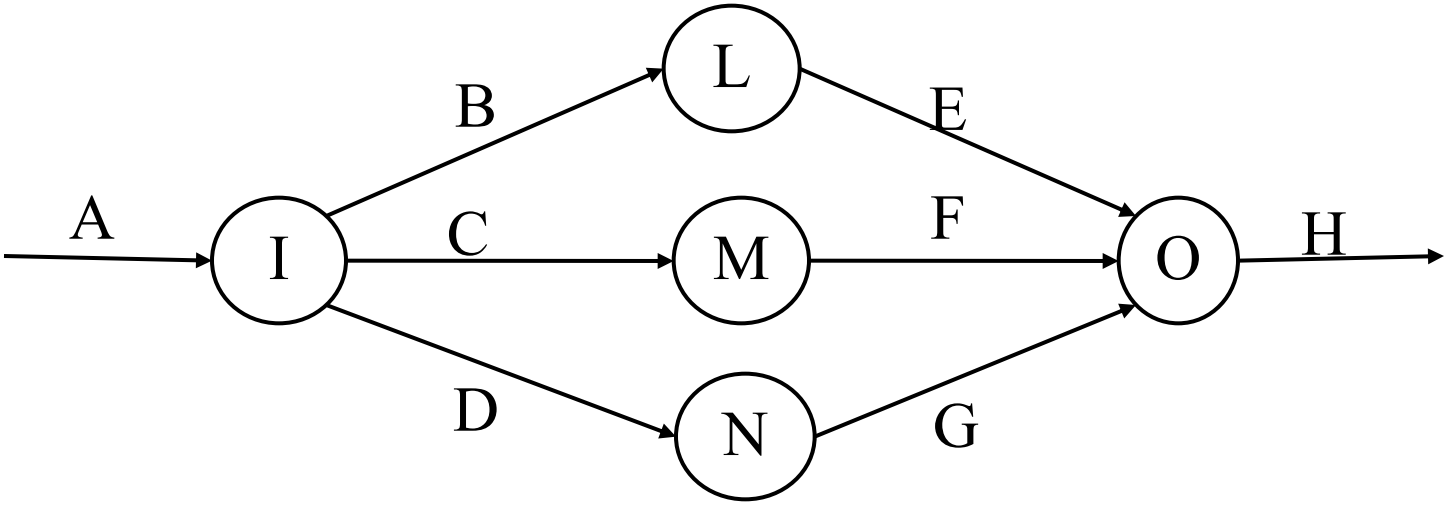
（3）进行一级分解，设计系统模块结构的顶层和第一层

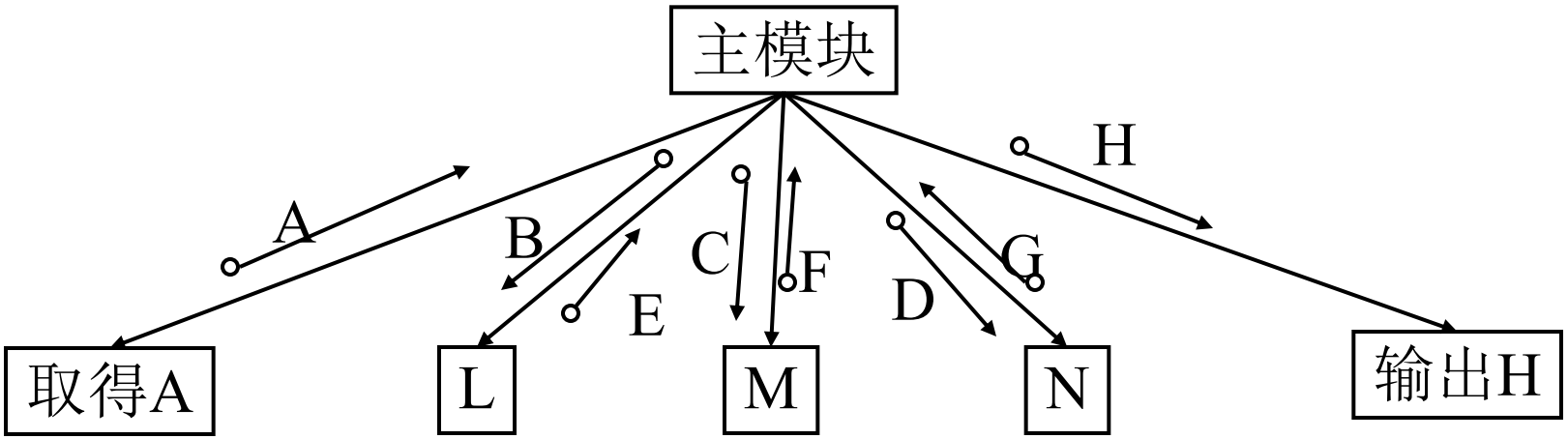
（4）进行二层分解，设计输入、中心变换、输出部分的中心、下层模块

7.3 事务分析

- 事务分析从数据流图开始，自顶向下，逐步分解，建立到结构图。
- 事务分析方法的步骤：
  ① 识别事务源。
  ② 规定适当的事务型结构。
  ③ 识别各种事务和它们定义的操作。
  ④ 注意利用公用模块。
  ⑤ 对每一事务，或对联系密切的一组事务，建立一个事务处理模块。
  ⑥ 对事务处理模块规定它们全部的下层操作模块。
  ⑦ 对操作模块规定它们的全部细节。

事务型问题导出的系统结构图





设计准则

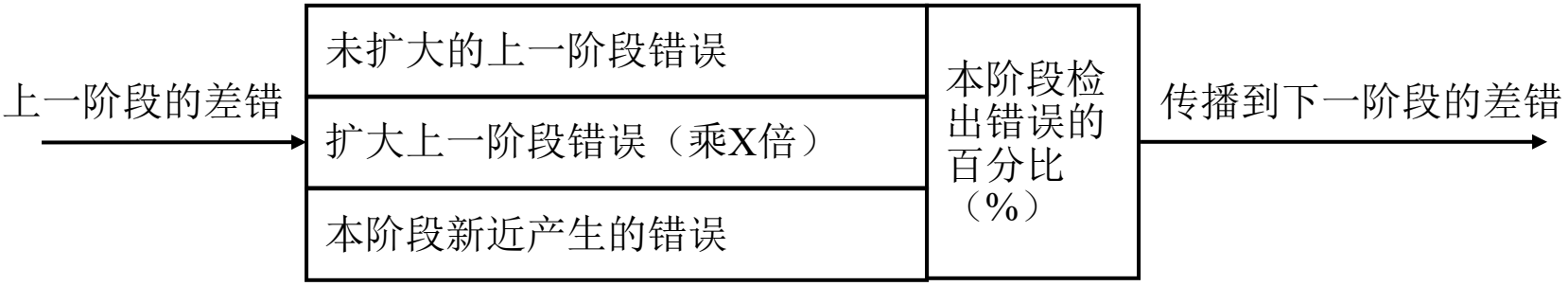
1) 评价软件的初始结构，通过模块的分解和合并，以求减少块间联系，增大联系。
2) 模块调用个数最好不要超5个。
3) 一个模块的作用范围应处在这个模块的控制范围之内
4) 降低模块接口的复杂性
5) 力求设计单入口和单出口的模块，避免“病态连接”。
6) 模块的大小要适中。

7.4 软件模块结构的改进

1) 模块功能的完善。
2) 消除重复功能，改善软件结构。
3) 模块的作用范围应在控制范围之间。
4) 尽可能减少高扇出结构，随着深度增大扇入。
5) 模块大小要适中。
6) 设计功能可预测的模块，但避免过分受限制的模块。

7.5 设计的后处理

1) 处理说明
2) 接口说明
3) 数据结构说明
4) 设计评审
5) 设计优化



错误的扩大模型

7.6 数据设计及文件设计

- 数据结构对程序结构和过程复杂性的影响使得数据设计对软件质量产生不容忽视的影响。
- 数据设计原则：
  1) 用于功能和行为的系统分析原则也适用于数据。
  2) 应该标识所有的数据结构以及其上的操作。
  3) 应当建立数据字典，并用于数据设计和程序设计。
  4) 低层的设计决策应该推迟到设计过程的后期。
  5) 只有那些需要直接使用数据结构内部数据的模块才能看到该数据结构的表示。
  6) 应该开发一个由有用的数据结构和应用于其上的操作组成的库。
  7) 软件设计和程序设计语言应该支持抽象数据类型的定义和实现。

文件设计

- 文件设计是指数据存储文件的设计。
- 主要工作：

  根据使用要求、处理方式、存储的信息量、数据的活动性，以及所能提供的设备条件等，来确定文件类型，选择文件媒体，决定文件组织方式，设计文件格式，并估算文件的容量。

- 文件设计讨论的问题：

  1) 文件设计过程（逻辑设计和物理设计）
  2) 文件存储媒体的选择
  3) 确定文件的组织方式
  4) 确定记录的布局
  5) 预估存取时间
  6) 文件容量估算

软件开发

8 过程设计

主要内容

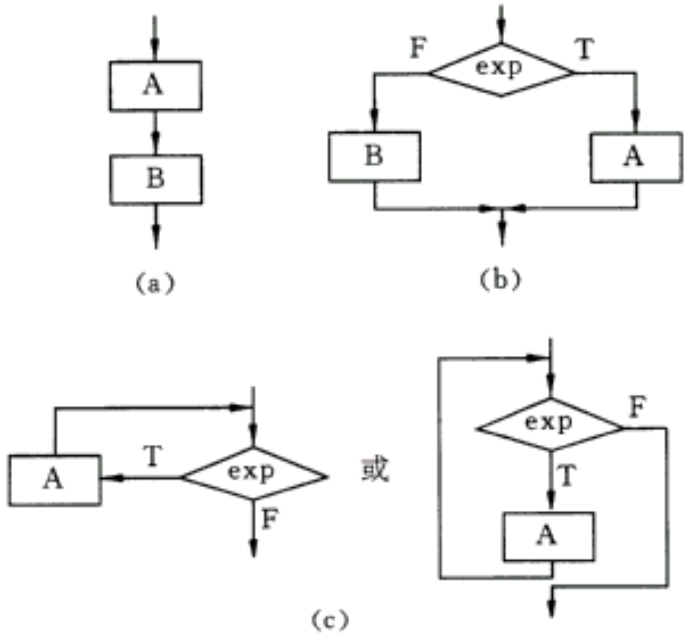
- 过程设计描述工具

过程设计

- 过程设计也叫详细设计或程序设计。
- 在过程设计阶段要决定各个模块的实现算法，并精确地表达这些算法。
- 详细设计工具分3类：
  - 图形工具
  - 表格工具
  - 语言工具

8.1 程序流程图

程序流程图又称为程序框图，它是历史最悠久使用最广泛的描述过程设计的方法，然而它也是用得最混乱的一种方法。



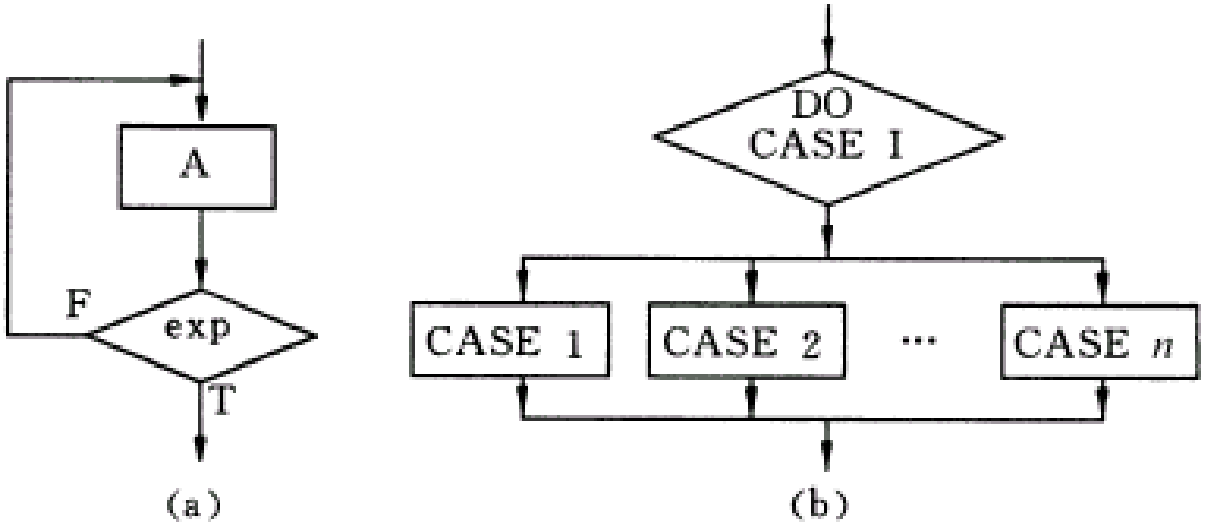
三种基本的控制结构：

(a)顺序结构，先执行A再执行B

(b)IF-THEN-ELSE型选择(分支)结构

(c)DO-WHILE型循环结构

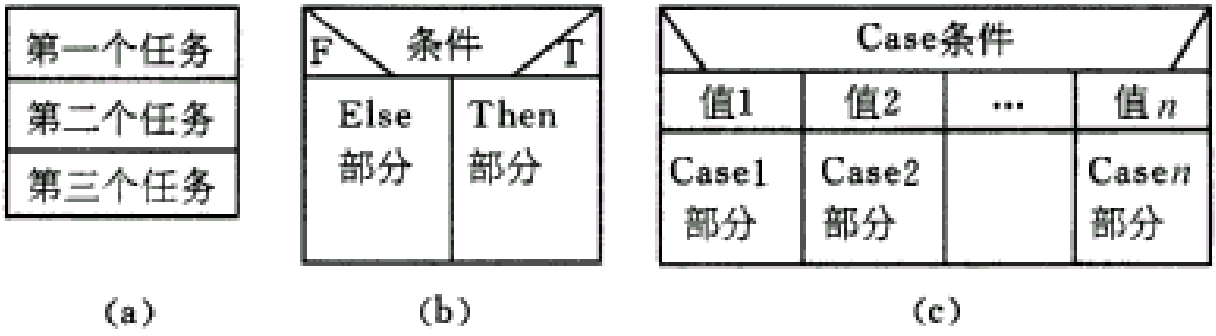
其他常用的控制结构

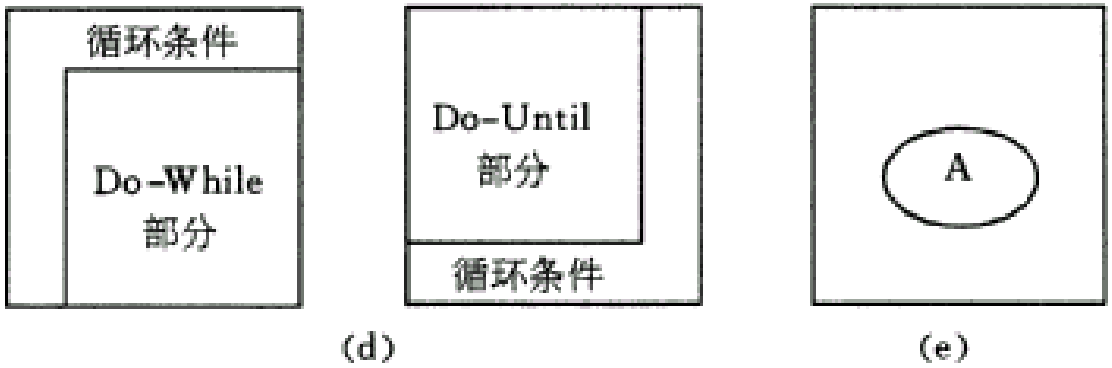


(a) DO-UNTIL型循环结构；(b)多分支结构

8.2 N-S图

- 由于流程图不是一种有效的结构化设计工具，Nassi和Shneiderman提出了盒图，又称为N-S图。
- N-S是一种结构化程序设计的图形描述工具。

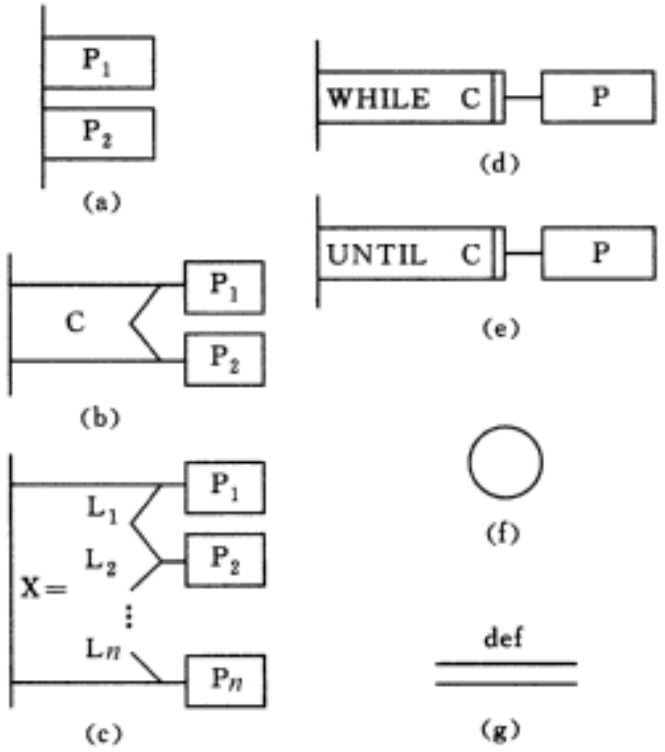




(a) 顺序；
(b) IF-THEN-ELSE型分支；
(c) CASE型多分支；
(d) 循环；
(e) 调用子程序A

8.3 PAD

- PAD(Problem Analysis Diagram)，自1973年由日本日立公司发明以后，已得到一定程度的推广。
- 它用二维树形结构的图来表示程序的控制流，将这种图翻译成程序代码比较容易。



(a) 顺序(先执行P1后执行P2)；

(b) 选择(IF C THEN P_1 ELSE P_2)；

(c) CASE型多分支；

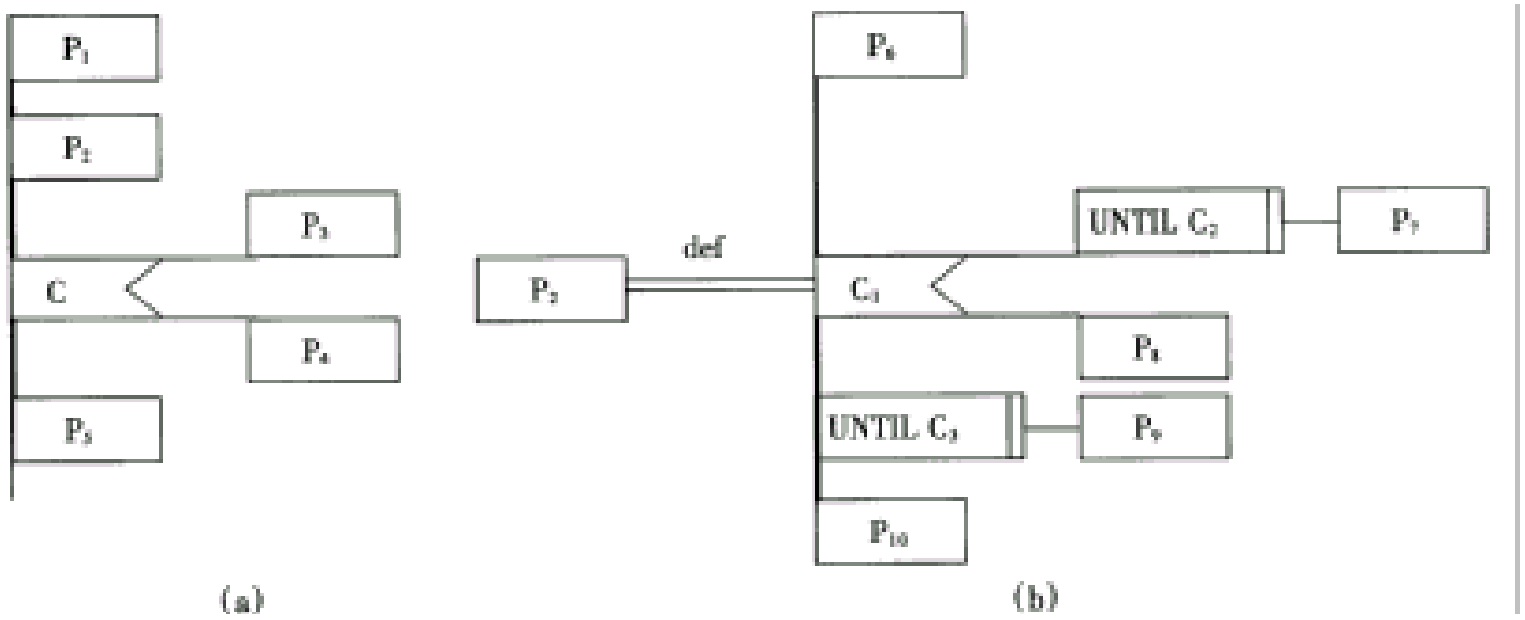
(d) WHILE型循环(WHILE C DO P)；

(e) UNTIL型循环(REPEAT P UNTIL C)；

(f) 语句标号；

(g) 定义

PAD例子



(a) 初始的PAD图；（b）使用def符号细化处理框P2

8.4 判定表

- 当算法中包含多重嵌套的条件选择时，用程序流程图、N-S图、PAD图或PDL都不易清楚地描述。
- 然而判定表却能够清晰地表示复杂的条件组合与应做的动作之间的对应关系。
- 判定表由4部分组成：
  1. 左上部列出所有条件
  2. 左下部是所有可能做的动作
  3. 右上部是表示各种条件组合的一个矩阵
  4. 右下部是和每种条件组合相对应的动作。判定表右半部的每一列实质上是一条规则，规定了与特定的条件组合相对应的动作。

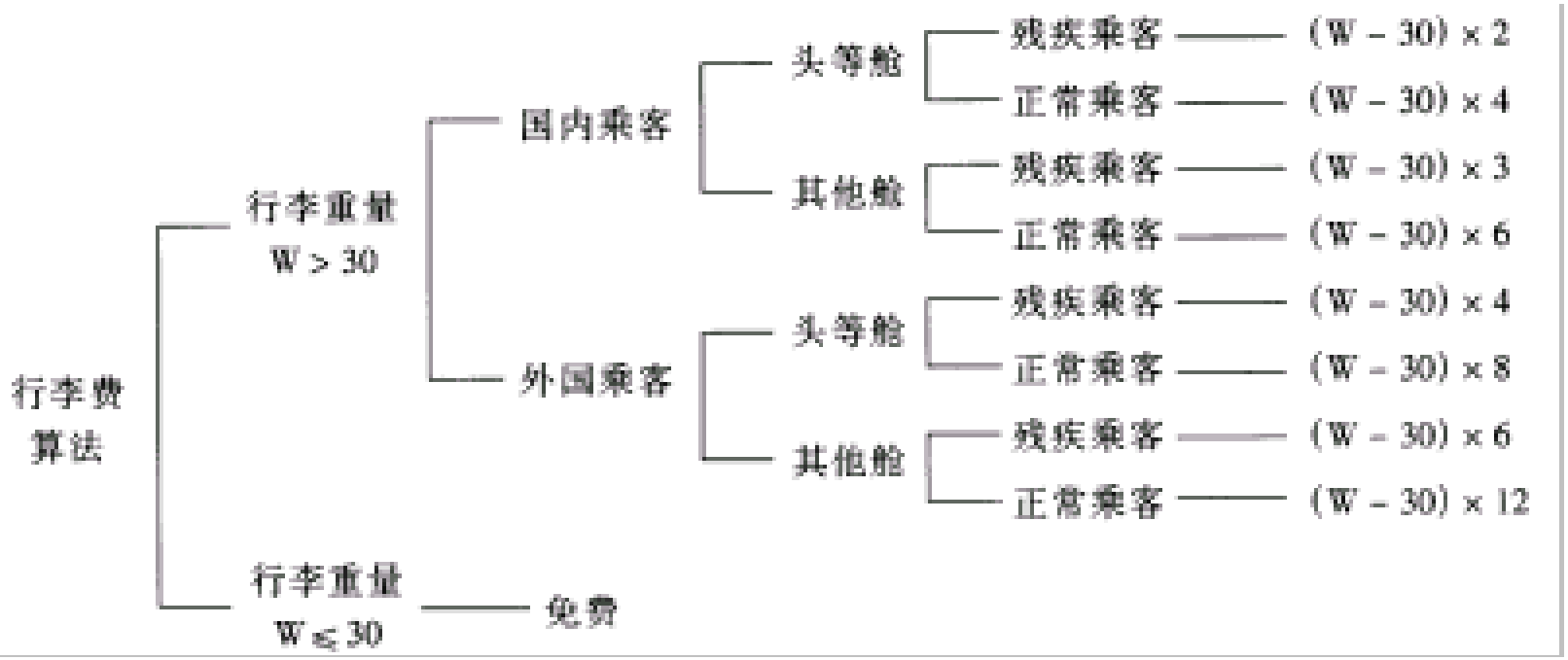
例：用判定表表示计算行李费的算法

	,	−	.	／	0	1	2	3	4
身瑚埠痔		O	O	O	O	A	A	A	A
橡□□		O	A	O	A	O	A	O	A
□□埠痔		A	A	O	O	A	A	O	O
□竈□□ R‰. +	O	A	A	A	A	A	A	A	A
	ㄙ								
#R(. +$ −				ㄙ					
#R(. +$.					ㄙ				
#R(. +$ ／		ㄙ						ㄙ	ㄙ
#R(. +$ 1			ㄙ						
#R(. +$ 3						ㄙ			
#R(. +$, −							ㄙ		

判定树

- 判定表虽然能清晰地表示复杂的条件组合与应做的动作之间的对应关系，但其含义并不直观。此外，当数据元素的值多于两个时，判定表的简洁程度也将下降。
- 判定树是判定表的变种，也能清晰地表示复杂的条件组合与应做的动作之间的对应关系。

用判定树表示计算行李费的算法



8.5 PDL（Program Design Language）

- PDL也称为伪码，这是一个统称，现在有多种不同的过程设计语言在使用。它是用正文形式表示数据和处理过程的设计工具。
- PDL具有严格的关键字外部语法，用于定义控制结构和数据结构；另一方面，PDL表示实际操作和条件的内部语法通常又是灵活自由的，以便可以适应各种工程项目的需要。

例：PDL描述查找拼错单词的算法。

```
PROCEDURE spellcheck IS
    BEGIN
        split document into signal words
        look up words in dictionary
        display words which are in dictionary
        create a new dictionary
    END spellcheck
```

过程设计表示法比较

表示法 / 准则	流程图	N-S图	PDL	PAD	判定表（树）
易用性	优	优	优	良	中
逻辑表达能力	中	中	良	良	优
机器可读性	中	中	中	中	差
易转换程序代码	良	良	优	良	良
结构化	差	优	良	优	差
易修改性	差	差	良	差	良
数据表示能力	差	差	中	中	中
易验证性	差	差	中	中	中
使用频率	高	低	高	低	低

软件开发

9 程序设计

主要内容

① 程序质量

② 程序设计风格

③ 程序效率

④ 程序设计方法学

⑤ 程序的复杂性

9.1 程序质量

- 程序的外部质量（external quality）
  - 用户视点
  - 对外可展示，用户可直接感触到、所关心的
  - 程序运行表现出：正确性、友好性、高效性、易用性、可靠性等。
- 程序的内部质量（internal quality）
  - 程序员视点
  - 体现在程序的内在方面，程序员可以感触到的、所关心的
  - 可理解性、维护性、可重用性等

对源程序的质量要求

- 好的编程风格
- 好的界面
- 易排错
- 易测试
- 高性能
- 移植性好

9.2 程序设计风格

- 具有良好风格的代码更容易阅读和理解，几乎可以保证其中的错误更少。
- 程序不仅需要给计算机读，也要给人读。
- 好的风格使代码更容易读，无论是对程序员本人，还是其他读程序的人。
- 风格是一种习惯，通过规范形式给出。

❑ 代码风格规范-语法

- ✓ 语法和结构层次
- ✓ 明确如何来规范程序的书写
- ✓ 表现为是否易于阅读和理解

❑ 代码设计规范-语义

- ✓ 语义和内涵层次，外在的
- ✓ 明确如何来组织和封装程序语句
- ✓ 表现为良好的结构和易于重用

9.3 程序效率

- 此处的性能主要是指软件的：
  - 运行速度
  - 存储空间
- 由于硬件的飞速发展，它对软件的速度和存储空间限制或要求越来越少。一个快速的程序如果给出的是错误结果，那么它一点也没有节省时间。

关于优化的讨论：

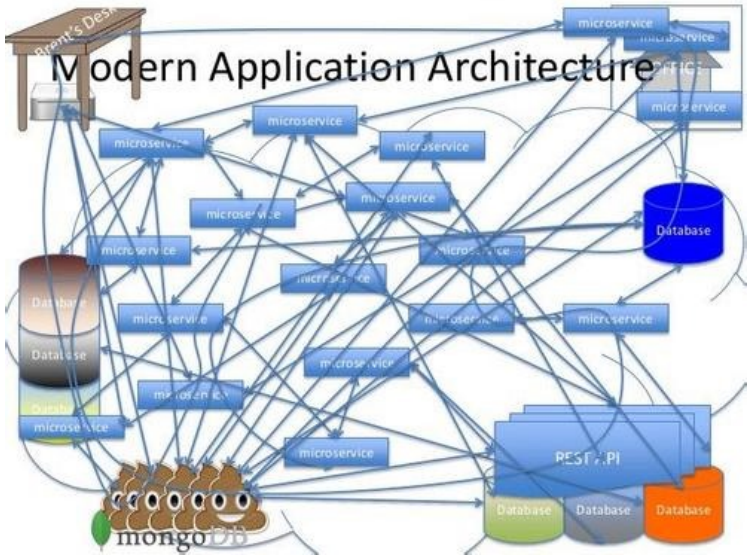
- 优化的第一要义是不做！
- 什么时候应该试图去加速一个程序？

改进性能的策略：

- 最好的策略是使用最简单、最清晰的算法和数据结构。
- 性能是其次要考虑的事情。
- 打开编译选择项，生成尽可能快的代码。
- 考虑对程序本身做什么改变才能有最大作用。
- 一次做一个改变并重新进行评价。
- 在改进性能过程中，始终保留最简单的版本，用它作为测试版本的对照物。

9.4 程序的复杂性

- 程序复杂度（区别于计算复杂度）是对软件内部诸多影响内部关联关系的属性的描述。
- 软件的高复杂度导致增加无意（与目的无关）的关联关系的风险，最终在做变更的时候更容易引入缺陷。在更极端的情况下，软件复杂到几乎无法修改。
- 有模块、类和程序三类复杂度：
  - 模块复杂度包含了关于模块的复杂度信息；
  - 类复杂度是针对那些使用McCabe面向对象特性的程序，它包含了关于类的复杂度信息；
  - 程序复杂度包含了关于程序的复杂度信息。



软件复杂性度量模型应遵循的基本原则：

(1) 软件复杂性与程序大小的关系不是线性的；

(2) 控制结构复杂的程序较复杂；

(3) 数据结构复杂的程序较复杂；

(4) 转向语句使用不当的程序较复杂；

(5) 循环结构比选择结构复杂，选择结构又比顺序结构复杂；

(6) 语句、数据、子程序和模块在程序中的次序对软件复杂性都有影响；

(7) 全程变量、非局部变量较多时程序较复杂；

(8) 参数按地址传递比按值传递更复杂；

(9) 函数副作用比显式参数传递更难以琢磨；

(10) 具有不同作用的变量共用一个名字时较难理解；

(11) 模块间或过程间联系密切的程序较复杂；

(12) 嵌套深度越深程序越复杂。

9.5 程序设计方法

- 语句设计
- 模块化设计
- 高内聚度、低耦合度原则

代码设计规范-语句设计

- 程序设计方面问题，程序内在质量的体现
- 单入口单出口，少用goto语句
- 加强对异常处理
  - 分析和验证输入参数正确性，如名字不为空，年龄大于0少于150
  - 执行结果处理，设置必要断言来分析

如 p = AllocateNewSpace( );

```
Assert(p !=  NULL);
if (p !=  NULL)  {
        ……
}
```

将更多的时间和精力放在处理异常代码上

  - 处理异常语句 Try { … } catch (Exception e) { … }

代码设计规范-模块化设计

- 模块是逻辑上相对独立、具有良定义接口的编程单位
  - 模块可表现为函数、过程、方法、类、程序包等
- 高内聚度：模块内各要素紧密相关，仅实现单一功能
  - 如果模块内多个要素关系不密切，需分解产生多个模块
- 低耦合度：模块间的关系应设计的非常松散
  - 如果多个模块间的关系非常密切，可将这些模块合并为一个模块

9.6 代码重用

- 何为代码重用
  - 在编写代码过程中，充分利用已有和现成的代码，并将其集成到程序之中，从而来实现程序功能
- 代码重用有何好处
  - 由于被重用的代码经过多次反复的使用，代码质量得到充分检验，因而代码重用不仅可极大提高编程效率，而且还可有效提高程序质量

重用函数、类和软构件

- C函数库
- MFC类库(Microsoft Foundation Classes)
- Java软件开发包
- 机器人操作系统（ROS）的节点软构件

重用开源代码

- 到Github中找到粗粒度的代码开源
  - 几万、甚至几十万的程序代码
- 重用开源代码来实现粗粒度的功能
  - 完成诸如数据库管理、图像识别、语音分析等功能
- 可在Stack Overflow上寻找到有价值的代码片段

9.7 程序中潜在的质量问题

- 质量问题
  - 编写不合理 – 没有遵循编码规范
  - 设计不合理 – 没有遵循设计规范
  - 代码有错误 – 代码编写的不正确
- 原因
  - 受软件开发人员经验和水平的限制
  - 人可能会犯错误，人为引入错误

代码质量分析的常用方法

- 人工审查方法
- 自动化分析方法
- 程序测试方法

软件开发

10 代码生成

代码生成原理

- 代码生成（ Code Generation）旨在将以某种规范形式表达的人类意图自动转化为可执行的计算机程序，是提升软件生产效率的根本途径。
- 本质：降低“人脑逻辑”到“机器代码”之间的转换成本。
- 工作过程：
  - 输入：需求、模型、抽象描述（AST、DSL、自然语言、流程图），输出可编译 / 可直接运行代码。
  - 输出：各类编程语言源码（Java/Python/Go/SQL 等）

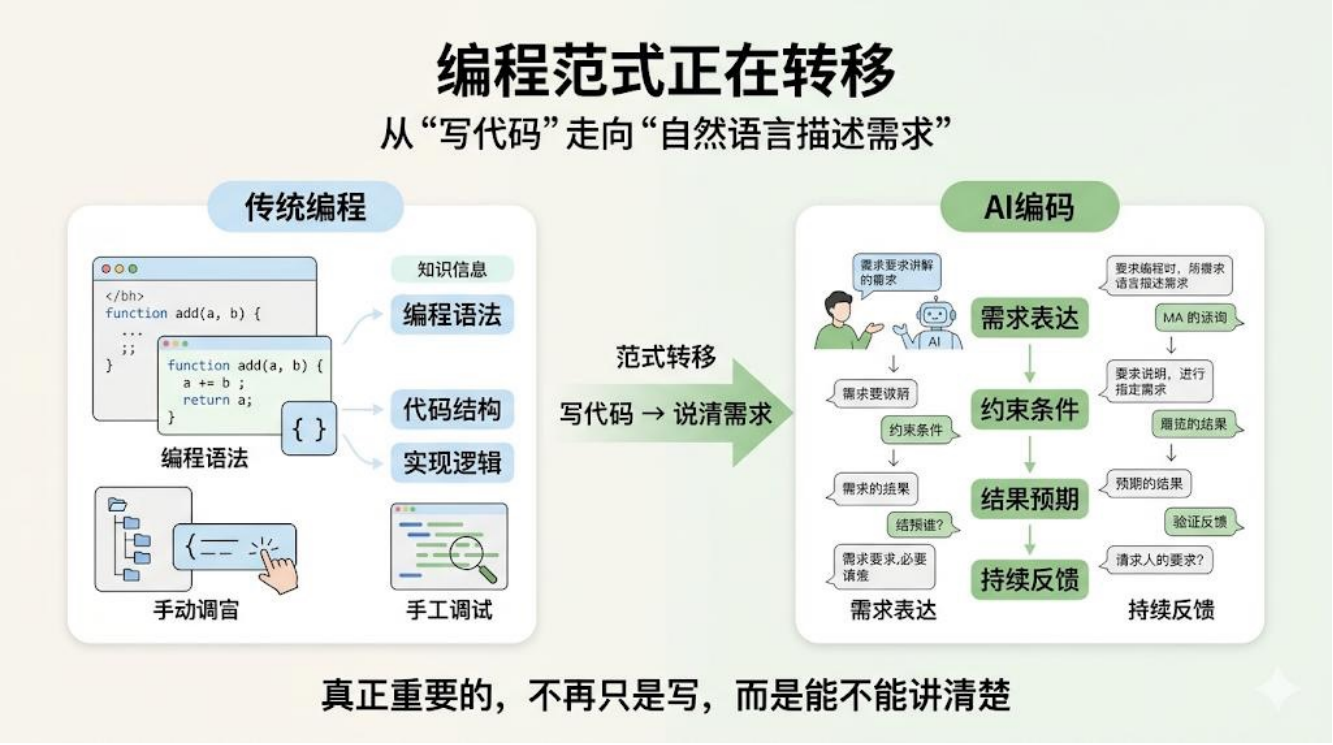
代码生成的技术演进

- 阶段1：基于规则的自动化生成 (1950s - 1990s)
- 阶段2：模板引擎时代（1990s–2010s）
- 阶段3：AST与元数据驱动（2000s–2018）
- 阶段4：DSL与低代码平台（2010s–2022）
- 阶段5：基于大模型的智能生成 (2020s-2024)
- 阶段6：Agent协同与自主编程 (2024+)

AI Coding

AI Coding正从"写代码"转向"说清需求”

——编程语法不再重要，清晰表达需求、约束条件和预期结果才是关键。



代码生成技术

- 早期研究主要采用程序综合（program synthesis）方法
  - 通过形式化规约推导可验证正确的程序。
  - 由于规约编写困难， 该方法长期局限于定义明确的特定任务。
- 基于深度学习的数据驱动范式
  - 为提升泛化能力，将代码生成视为概率性序列学习问题。 但这类方法生成的代码片段往往功能有限，且经常存在语法或语义错误，导致编译或执行失败。
- 大语言模型 （LLM）
  - 以GitHub为代表的开源社区贡献了海量的预训练语料库，使得模型得以掌握程序语言的语法与语义乃至编程算法和范式。
  - 基于LLM的代码生成技术在生成孤立 (standalone) 的程序上表现出色。
- 基于大语言模型的代码生成智能体（ LLM-based Agents）
  - 以LLM为“大脑”，构建自主规划、行动、观察并迭代优化的代码智能体。
  - 模拟人类程序员“分析需求、编写代码、运行测试、分析错误、修复代码”的完整工作流。

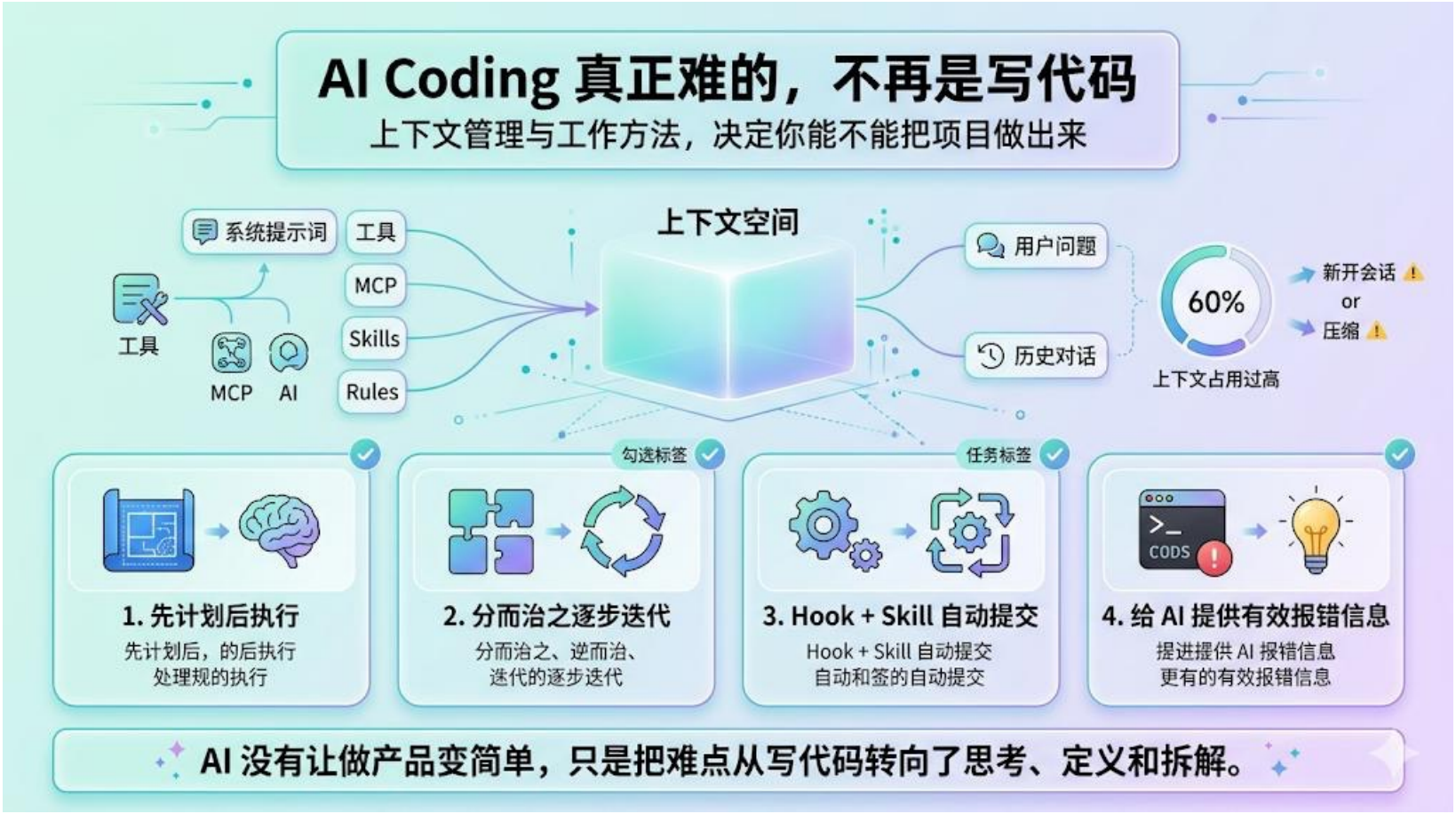
智能体

- 智能体（Agent）是一种能够自主感知环境、自主决策并采取行动以实现特定目标的计算实体。
- 智能体具备以下关键能力：
  - 自主性 (Autonomy)：能在无人干预下独立运行和决策。
  - 反应性 (Reactivity)：能实时感知环境变化并做出响应。
  - 主动性 (Proactiveness)：不仅能被动反应，还能主动设定目标并规划行动。
  - 社会性 (Social ability)：能与其他智能体或人类进行交互协作。
  - 学习进化 (Learning)：能从经验中学习，持续优化自身性能。
- 代码生成智能体功能：自动化代码生成与实现、自动化调试与程序修复、自动化测试代码生成、代码重构与优化以及代码生成的需求澄清。

智能体核心能力的局限性

- 领域知识和复杂专业推理存在欠缺
- 意图理解和上下文感知能力不足
- 长上下文建模、跨文件依赖分析以及整体软件架构理解方面的能力有限
- 无法充分理解和利用多模态信息进行代码生成

AI编程工具



AI编程工具的产品形态：编辑器插件、编辑器（IDE）、命令行(CLI)、桌面端APP

代码生成智能体的主流工具

- GitHub Copilot
  - 作为一款集成式编程智能体，其技术核心在于利用检索增强生成 (RAG) 动态构建上下文，并在由GitHub Actions驱动的云端沙盒中安全执行任务。
- Devin
  - 被宣传为“首位 AI 软件工程师”。它通过赋予大模型智能体使用终端工具、代码编辑器和Web浏览器的能力，期望完全自动化各类复杂的软件工程任务，涵盖从规划、执行到调试和部署的整个工作流程。
- Cursor
  - 是一款AI原生的独立IDE，是“协同伙伴”理念的深度实践。它通过对VS Code的深度定制，实现了本地优先的智能体架构。
  - 其关键技术是对代码库进行持久化向量嵌入索引，从而获得低延迟的全局上下文感知。这使其智能体能直接与本地文件系统和终端交互，在处理跨越多文件的复杂代码重构任务时，展现出与开发者进行高效、深度交互与协作的能力。
- 通义灵码 (Tongyi Lingma)
  - 通义灵码是基于CodeQwen大模型的“协同伙伴”。
  - 它通过构建代码库知识图谱，并创新性地采用蒙特卡洛树搜索 (MCTS) 进行系统性导航与故障定位，实现了对复杂软件仓库的深度理解。
- Claude Code
  - 是一款终端原生的编程智能体，其设计显著地朝“自主开发团队”方向迈进。
  - 其技术核心是利用高达200k token的超长上下文窗口 (“Codebase Lens”)，对整个代码库进行一次性的整体语义理解。

Codex

- Codex是OpenAI研发的一款专门面向代码场景的大语言模型，基于GPT-3架构改进，针对编程语言做了深度专项优化。
- 核心能力

① 代码生成：

自然语言描述需求，自动输出完整代码，支持 Python、Java、JavaScript、Go、SQL 等上百种编程语言；

② 代码翻译：

不同编程语言之间互相转换代码；

③ 代码解读：

读懂现有代码，解释逻辑、注释代码；

④ Bug 修复：

识别代码漏洞、报错，给出修复方案；

⑤ 项目重构：

优化老旧、冗余代码，调整代码架构。

智能体核心能力的局限性

- 领域知识和复杂专业推理存在欠缺
- 意图理解和上下文感知能力不足
- 长上下文建模、跨文件依赖分析以及整体软件架构理解方面的能力有限
- 无法充分理解和利用多模态信息进行代码生成

使用AI生成代码的法律风险

- **风险一：AI生成代码的可能不受版权保护**
  - 最根本的风险。
  - 版权保护“以人为本”，只有人类创作的作品才能获得版权保护。
- **风险二：潜在的侵权风险（AI可能会“抄袭”）**
  - 最直接的法律风险。
  - LLM在训练时“学习”了受版权保护或特定许可证（如GPL）约束的代码。它输出的代码，有可能是这些代码的“变形”或“拼凑”。
  - 许可证“传染”风险，如Copyleft laundering。
- **风险三：代码的“责任”问题——出了事谁来负责？**
  - 责任归属模糊，目前法律对此尚无定论。
  - 开源社区的集体“说不”：重要开源项目限制或禁止AI生成的代码。

规避风险的方法

- **记录“人的贡献”：**
  - 保留详细的开发日志，记录你的提示词、修改思路、代码审查过程等。这是未来主张你对代码拥有版权的重要证据。
- **进行代码“溯源审查”：**
  - 使用专门的代码溯源与许可证合规性检测工具，对AI生成的代码进行扫描，检查其中是否包含受版权保护或特定许可证约束的代码片段。
- **区分使用场景：**
  - 对核心商业项目保持高度谨慎，严格记录人类贡献并进行溯源审查。对于个人、实验性或一次性的“周末项目”，可以相对放松要求。
- **寻求专业法律意见：**
  - 在将AI生成的代码大规模用于商业产品前，有必要咨询专业知识产权律师。

思考题

1. 对负责提取需求描述的工程师，如何搞清楚功能需求和非功能需求之间的关系？给出你的建议。
2. 当系统必须紧急变更的时候，系统中软件可能必须在需求变更被批准之前就修改。给出一个执行这些修改的过程模型，要确保需求文档和系统实现不会产生不一致。
3. 解释为什么在给出系统描述之前设计系统体系结构是必要的？
4. 讨论下面两个概念的区别：作为有效模块化特性的信息隐蔽和模块独立性。
5. 讨论在需求分析、系统设计和软件实现中的复用问题，如：什么可以复用、如何提高复用等？
6. 开发一个字处理软件，使用结构化分析为系统建立模型。